

学校代码: 10284
分类号: TP311.5
密 级: 公开
U D C: 004.41
学 号: 502023330072



南京大學

硕士学位论文

论文题目	面向 Serverless 工作流 的投机执行优化研究
作者姓名	吴晟昊
专业名称	计算机科学与技术
研究方向	软件方法学与自动化
导师姓名	马骏 副教授

2026 年 4 月 3 日

答辩委员会主席_____

评 阅 人_____

论文答辩日期 2026 年 4 月 3 日

研究生签名:

导师签名:

Speculative Execution Optimization for Serverless Workflows

by
Shenghao Wu

Supervised by
Associate Professor Jun Ma

A dissertation submitted to
the graduate school of Nanjing University
in partial fulfilment of the requirements for the degree of

MASTER

in

Computer Science and Technology



School of Computer Science
Nanjing University

April 3, 2026

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：面向 Serverless 工作流的投机执行优化研究

计算机科学与技术 专业 2023 级硕士生姓名： 吴晟昊

指导教师（姓名、职称）： 马骏 副教授

摘 要

随着云计算的快速发展, Serverless 计算凭借按需计费、自动伸缩和低运维成本等优势, 已成为构建在线服务与数据处理应用的重要方式。然而, 在 Serverless 工作流中, 后继步骤需等待上游完成后才能触发, 导致分支判定、跨函数交互和冷启动等开销在关键路径上累积, 增加端到端时延。现有方法已证明投机执行能够有效降低工作流时延, 但在分支决策受输入影响且负载分布动态变化的场景下, 其预测准确性和优化稳定性仍显不足。

为此, 本文研究面向 Serverless 工作流的投机执行优化方法, 提出投机执行方案 SpecSW, 在不改变应用对外语义的前提下, 通过分支预测、投机触发以及副作用隔离与恢复机制, 减少保守等待造成的性能损失, 并针对输入相关和分布动态变化场景下预测精度与优化稳定性不足的问题进行优化。本文的主要工作如下:

- 提出了面向 Serverless 工作流的投机执行模型, 对控制依赖与数据依赖、分支预测、门控预算以及验证恢复等关键机制进行了统一抽象。
- 设计了支持投机执行的编程框架与原型系统, 能够以声明式方式描述工作流结构与运行策略, 并支撑预测、隔离、验证和恢复等机制的实现。
- 提出了面向分支预测的关键优化方法, 引入输入感知机制和分布漂移适应机制, 并探讨二者的联合使用方式, 以提升预测精度和投机收益稳定性。
- 在 OpenWhisk 平台上实现并评估了 SpecSW 原型系统。与已有方案相比, SpecSW 的平均端到端时延降低了 22.73%, P95 尾延迟下降了 10.3%, 错误投机率平均降低了 5.78 个百分点。

关键词: Serverless 计算; 工作流; 投机执行; 分支预测; 系统优化

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Speculative Execution Optimization for Serverless Workflows

SPECIALIZATION: Computer Science and Technology

POSTGRADUATE: Shenghao Wu

MENTOR: Associate Professor Jun Ma

ABSTRACT

With the rapid development of cloud computing, Serverless computing has emerged as an important paradigm for building online services and data-processing applications, due to its advantages of pay-as-you-go pricing, automatic scaling, and low operational overhead. However, in Serverless workflows, downstream functions are usually triggered only after upstream functions have completed, which causes the overheads of branch resolution, inter-function communication, and cold starts to accumulate along the critical path, thereby increasing end-to-end latency. Existing studies have shown that speculative execution can effectively reduce workflow latency. Nevertheless, when branch decisions are highly dependent on input characteristics and workload distributions change dynamically over time, existing approaches still suffer from limited prediction accuracy and unstable optimization gains.

To address these issues, this thesis investigates speculative execution optimization for Serverless workflows and proposes SpecSW, a speculative execution scheme that reduces the performance loss caused by conservative waiting through branch prediction, speculative triggering, and side-effect isolation and recovery, while preserving the externally observable semantics of applications. In addition, SpecSW is further optimized for scenarios involving input-dependent branching and dynamic workload distribution changes, so as to improve prediction accuracy and enhance the stability of speculative performance gains. The main contributions of this thesis are summarized as follows:

- A speculative execution model for Serverless workflows is proposed, which pro-

vides a unified abstraction of key mechanisms, including control and data dependencies, branch prediction, gating and budget control, as well as validation and recovery.

- A programming framework and a prototype system supporting speculative execution are designed, enabling workflow structures and execution policies to be specified in a declarative manner, and supporting the implementation of prediction, isolation, validation, and recovery mechanisms.
- Several key optimization methods for branch prediction are proposed, including an input-aware mechanism and a distribution-drift adaptation mechanism. Their joint usage is further explored to improve prediction accuracy and the stability of speculative benefits.
- A prototype of SpecSW is implemented and evaluated on the OpenWhisk platform. Experimental results show that, compared with existing approaches, SpecSW reduces the average end-to-end latency by 22.73%, decreases the P95 tail latency by 10.3%, and lowers the average false speculation rate by 5.78 percentage points.

KEYWORDS: Serverless Computing; Workflow; Speculative Execution; Branch Prediction; System Optimization

目 录

目 录	V
插图目录	IX
表格目录	XI
第一章 绪论	1
1.1 研究背景	1
1.2 研究现状	3
1.3 本文工作	4
1.4 本文组织结构	6
第二章 背景知识	7
2.1 Serverless 计算	7
2.2 Serverless 工作流与执行特性	9
2.2.1 组织模式	9
2.2.2 控制结构	10
2.2.3 数据流与状态管理	11
2.2.4 性能执行特性	12
2.3 Serverless 系统性能瓶颈与优化技术概览	12
2.3.1 面向典型任务的系统优化	13
2.3.2 沙箱环境与冷启动优化	13
2.3.3 I/O 与通信优化	14
2.3.4 资源调度优化	14
2.3.5 小结与系统层优化边界	14
2.4 投机执行思想与工作流推测优化	15

2.5 本章小结	16
第三章 Serverless workflow 投机执行优化方法	17
3.1 方法概述	17
3.2 Serverless workflow 投机执行模型	19
3.2.1 模型定义	19
3.2.2 模型的运行时行为	24
3.3 Serverless workflow 投机执行的编程框架	26
3.3.1 工作流过程描述	27
3.3.2 投机运行时配置接口	29
3.4 投机执行的关键优化方法	34
3.4.1 输入感知的分支预测	35
3.4.2 分布漂移适应	42
3.4.3 输入感知与分布漂移适应的联合使用	48
3.5 本章小结	50
第四章 Serverless workflow 投机执行原型系统设计与实现	51
4.1 运行平台与系统实现环境	51
4.1.1 OpenWhisk 平台及其 workflow 基础	51
4.1.2 系统与 OpenWhisk 的交互方式	52
4.2 系统总体架构设计	53
4.3 系统核心模块设计	54
4.3.1 工作流描述与运行组织模块	54
4.3.2 投机执行控制器	55
4.3.3 分支预测与模型状态管理模块	59
4.3.4 系统运行支撑模块	61
4.4 本章小结	63
第五章 实验设计与结果分析	65
5.1 实验配置	65
5.2 核心算法仿真实验	66

5.2.1	输入感知分支预测效果评估	66
5.2.2	分布漂移适应效果评估	68
5.2.3	联合优化效果评估	70
5.3	功能验证实验	72
5.4	性能评估实验	73
5.4.1	实验设置	74
5.4.2	平均端到端表现分析	75
5.4.3	尾延迟分析	77
5.4.4	投机情况分析	78
5.4.5	本节小结	79
5.5	消融实验	79
5.5.1	输入感知分支预测模块消融	79
5.5.2	分布漂移适应模块消融	80
5.5.3	联合优化效果消融	81
5.6	本章小结	82
第六章 总结与展望		85
6.1	工作总结	85
6.2	研究展望	86
参考文献		87
致 谢		93
简历与科研成果		95

插图目录

2-1	几种云服务模式的区别比较 ^[28]	7
2-2	Serverless 工作流的四类基本模式	10
3-1	面向 Serverless 工作流的投机执行优化方案 SpecSW 示意图	17
3-2	面向 Serverless 工作流的投机执行优化方案架构示意图	19
3-3	编排式 Serverless 工作流示意图	20
3-4	投机执行模型总体视图	22
3-5	分支预测与候选集合构造示意图	23
3-6	函数实例状态机	24
3-7	工作流中控制/数据依赖与投机执行对关键路径的影响示例	26
3-8	工作流过程描述中的核心抽象	28
3-9	工作流描述文件与函数文件组织示例	29
3-10	WorkflowRunner 接口对象与实现类型	31
3-11	输入感知的分支预测方法流程图	36
3-12	路径级统计与输入桶级统计的回退混合示意图 ($k = 32$)	39
3-13	香农熵随分支确定性变化的示意图 (二分支)	39
3-14	输入感知预测的自适应启用条件示意图	41
3-15	指数衰减与滑动窗口对比示意图	44
3-16	漂移强度 $D_t(c)$ 与指数衰减估计、滑动窗口估计之间的关系示意图	46
3-17	基于双阈值滞回的分布漂移适应分支预测方法流程图	47
3-18	输入感知与分布漂移适应联合使用的分层分支预测流程图	49
4-1	OpenWhisk 编程模型 ^[31]	51
4-2	SpecSW 原型系统总体架构	53
4-3	投机执行控制器的运行逻辑示意图	57

4-4	系统运行支撑模块的典型时间线可视化结果	61
5-1	不同输入建模策略在输入驱动分支场景下的对比结果	67
5-2	长记忆统计与分布漂移适应策略在分布漂移场景下的对比结果 . .	69
5-3	静态路径统计与联合优化策略在输入-漂移耦合场景下的对比结果	71
5-4	功能测试执行结果	73
5-5	不同工作流与负载条件下三种方法的端到端平均时延比较	75
5-6	不同工作流与负载条件下投机方法的平均加速比	76
5-7	不同工作流与负载条件下三种方法的 P95 归一化时延比较	77
5-8	SpecSW 相对 BaseSpec 的 P99 变化热力图	78
5-9	四种策略在分布漂移场景下的最大预测概率变化情况	81
5-10	四种方法在两类场景下的全样本误预测率、投机率和平均撤销数 .	82

表格目录

3-1	SpecRuntimeConfig 公开方法	30
3-2	WorkflowRunner 公开方法	31
3-3	WorkflowRun 公开方法	32
4-1	工作流定义中的关键字段及其作用	55
4-2	投机执行控制器维护的关键运行对象与状态	56
4-3	开发者侧受控副作用接口约定	58
4-4	控制器侧副作用管理接口约定	58
4-5	分支预测器关键配置参数、默认值及其作用	59
4-6	specx 主要命令及其功能说明	62
5-1	实验软硬件环境配置	65
5-2	仅路径统计与输入感知策略在输入驱动分支场景下的结果对比 . . .	68
5-3	长记忆统计与分布漂移适应在分布漂移场景下的结果对比	68
5-4	静态路径统计与联合优化策略在输入-漂移耦合场景下的结果对比	70
5-5	功能验证实验中使用的测试工作流	72
5-6	端到端应用工作流的适配情况	73
5-7	三类工作流的高、中、低并发场景设置	74
5-8	不同工作流场景下两种投机方法的错误投机率与额外触发开销比较	78
5-9	输入感知分支预测模块消融结果	79
5-10	按漂移强度合并后的分支预测统计模块消融结果	80

第一章 绪论

1.1 研究背景

近年来，随着云计算基础设施的不断成熟以及事件驱动应用形态的广泛普及，Serverless 计算逐渐成为构建在线服务与数据处理应用的重要范式^[1-4]。与传统依赖预置服务器或集群资源的部署方式不同，Serverless 以函数即服务（Function as a Service, FaaS）为核心，向开发者屏蔽底层资源供给与运维管理细节，并提供按需计费、自动伸缩和托管运行时等能力。这种更高抽象层次的云计算模式，使应用能够以较低的管理成本应对动态负载，并推动云上应用由单函数调用向多步骤组合与流程化执行持续演进^[5]。目前，AWS Lambda、Google Cloud Functions、Azure Functions 以及阿里云函数计算等平台的发展，也进一步促进了 Serverless 技术在工业界的落地与普及^[6-9]。

在这一背景下，Serverless 工作流逐渐受到研究界与工业界的广泛关注^[10]。随着应用逻辑复杂度不断提高，越来越多的任务已难以由单个函数独立完成，而是需要将端到端处理过程拆分为多个函数节点，并通过控制流与数据流依赖组织成可执行流程。相比单函数调用，工作流能够更好地支撑多阶段处理、条件路由、并行计算与结果汇聚等复杂业务场景，这与经典工作流模式研究中对顺序、分支、并发与同步结构的抽象是一致的^[11]。因此，Serverless 工作流在在线服务、日志处理、媒体处理、在线推理以及自动化运维等场景中展现出良好的适配性^[12-13]。与此同时，AWS Step Functions、Google Workflows 与 Azure Durable Functions 等工业界编排系统也表明，围绕函数编排构建复杂应用流程已成为 Serverless 生态的重要发展方向^[14-16]。

然而，Serverless 工作流在获得灵活性与可扩展性的同时，也暴露出更加突出的端到端性能问题^[17]。传统工作流执行通常遵循严格的依赖驱动语义，即只有当前驱节点完成并产生确定结果后，后继节点才可被触发执行；这种保守执行方式虽然保证了语义正确性，却也使关键路径中累积了较多等待开销。具体而

言，函数实例创建、运行时初始化与依赖加载等冷启动开销会在多级调用链路中反复出现；跨函数触发、数据传递与状态交互会带来额外的平台开销；而在包含条件分支与循环结构的工作流中，下游路径必须等待上游控制判定揭示后才能确定，进一步拉长了关键路径并放大尾延迟问题^[18]。因此，工作流的端到端时延往往不再仅由单个函数的执行效率决定，而更多取决于跨函数交互、同步约束以及控制流不确定性共同造成的等待成本。

围绕 Serverless 性能瓶颈，现有研究已从冷启动缓解、I/O 与通信优化、资源调度与实例预热等多个角度进行了探索。这些方法在降低函数启动延迟、改善数据传递效率以及提升资源利用率方面取得了明显成效^[19-23]，但大多仍建立在“依赖就绪后触发”的保守执行原则之上，对于分支、循环等控制不确定性所导致的关键路径等待问题缓解有限。相比之下，投机执行通过预测可能被执行的后继路径并提前推进相关计算，为缩短关键路径提供了新的优化思路；这一思路与传统体系结构和处理器领域中通过分支预测减少等待的基本思想具有一致性^[24]。然而，在 Serverless 环境中，投机执行的落地仍面临多方面挑战，例如平台黑盒约束下统一抽象与工程实现困难，以及复杂工作流中分支决策受输入影响、负载随时间变化而引起的预测准确性与适应性不足。如何围绕这些问题设计有效的投机执行优化方法，是本文关注的核心内容。

基于上述背景，本文主要研究问题包括：

1. 如何在底层 Serverless 平台的约束下，建立面向工作流投机执行的统一模型与编程框架，从而为投机执行在真实平台上的组织与实现提供基础支撑？
2. 如何针对复杂工作流中分支决策与输入相关、且分支分布随时间变化的问题，设计高准确率、可自适应的分支预测方法，以提升投机触发决策的稳定性与有效性？

我们认为，开展面向 Serverless 工作流的投机执行优化研究具有重要意义。一方面，该研究有助于缓解传统工作流在多级调用和控制不确定性下的关键路径瓶颈，进一步降低端到端时延并提升资源利用效率；另一方面，也能够为预测驱动的云上执行优化提供具有推广价值的方法论与工程经验，推动 Serverless 工作流在更复杂、更严苛应用场景中的落地与发展。

1.2 研究现状

Serverless 工作流是 **Serverless** 多函数应用的重要形态，其运行效率直接影响应用的端到端性能与资源利用率。围绕 **Serverless** 工作流的组织方式、执行机制与性能优化问题，工业界与学术界已经开展了大量研究，提出了一系列工作流编排方案与系统优化方法。

在工业界，主流云厂商和开源社区已经提供了多种面向 **Serverless** 工作流的编排与运行支撑系统。**AWS Step Functions**、**Google Workflows** 以及 **Azure Durable Functions** 等平台提供了较为成熟的工作流描述、状态维护与函数触发能力，使开发者能够以更高层次的抽象组织多函数应用流程^[14-16]。这类系统在工程上显著提升了 **Serverless** 工作流的可构建性、可观测性与可维护性。然而，这些方案的核心目标主要仍是保证严格依赖语义下的正确执行，即后继步骤通常需要在当前节点完成、控制判定揭示或状态更新完成后才能被触发。因此，在复杂工作流场景中，跨函数触发、状态持久化、同步等待以及分支判定等因素仍会在关键路径上持续累积开销，限制端到端性能的进一步提升^[17]。

在学术界，研究者围绕 **Serverless** 工作流的性能瓶颈提出了多种系统优化方案。现有研究一部分聚焦于冷启动与运行时初始化开销优化，通过快照复用、按需恢复、实例预热与预载入等方式降低函数启动延迟。例如，**Catalyzer** 通过复用运行时状态缩短函数启动关键路径^[19]，**ORION** 与 **Kraken** 则进一步结合工作流结构信息，对容器准备、实例预热与资源供给策略进行优化，从而减少多阶段链路中的启动等待^[21,25]。另一部分研究聚焦于函数间数据传输与通信开销，例如 **SONIC** 针对 **DAG** 应用中不同依赖边选择更合适的数据传递方式，并结合通信感知的函数放置优化端到端时延^[20]；**MXFaaS** 则通过共享资源与合并远程访问降低远程调用和外部存储访问成本^[22]。此外，也有研究从资源调度与实例管理角度出发，通过执行时间预测、并发控制与资源回收等策略提升平台利用率并减轻尾延迟，如 **Libra** 等工作便在这一方向取得了较好效果^[23]。

现有工作在缓解冷启动、优化通信路径以及改进资源调度方面已经取得了较为显著的效果，并在一定程度上提升了 **Serverless** 工作流的运行效率。但这些方法大多仍建立在“依赖就绪后触发”的保守执行原则之上，其优化重点主要是减少既定执行路径上的系统开销，而对分支、循环等控制不确定性引起的关键

路径等待关注相对不足^[18]。尤其是在包含条件分支和复杂控制结构的工作流中，下游后继路径通常必须等待上游判定结果出现后才能启动，这使得判定等待本身成为端到端时延的重要来源。

为进一步缩短关键路径，已有研究开始尝试将投机执行思想引入 Serverless 场景。以 SpecFaaS 为代表的工作表明，通过对潜在后继路径进行预测，并在真实分支结果尚未确定时提前启动可能被执行的后继函数，可以在一定程度上实现等待隐藏与执行重叠，从而降低端到端时延与尾延迟^[24]。这为后续研究提供了重要启发。然而，现有方法对于常见的特殊场景适应能力仍然有限。

综合来看，现有研究存在以下两点不足：

- **面向 Serverless workflow 投机执行的研究仍较少，且缺乏统一的抽象与编程规范。**现有研究主要集中于冷启动缓解、通信优化、资源调度与实例预热等一般性系统优化，而针对 Serverless workflow 投机执行的研究相对有限。对于控制依赖、数据依赖、门控约束、副作用隔离、验证提交与误投机恢复等关键机制，尚未形成统一的模型抽象与编程规范。
- **现有代表性工作对投机执行的建模仍较粗略，分支预测方法也有待深化。**以 SpecFaaS 为代表的工作虽然验证了投机执行在 Serverless 场景中的可行性，但其对 workflow 投机执行过程的建模仍较粗粒度，分支预测器也主要依赖基础的历史统计信息。对于输入相关性、路径差异性以及负载变化引起的分布漂移等问题，现有工作考虑仍不充分。

针对上述不足，需要进一步开展面向 Serverless workflow 投机执行优化的研究。一方面，应在底层平台约束下构建统一的投机执行模型与编程框架，为 workflow 中的控制依赖、数据依赖和验证恢复等机制提供清晰的描述基础；另一方面，应围绕复杂 workflow 中的分支预测问题开展更深入的研究，重点增强预测器对输入相关性与时间变化的建模能力，以提高投机触发决策的准确性、稳定性和工程可落地性。

1.3 本文工作

为解决上述研究问题，弥补现有工作在 Serverless workflow 控制不确定性优化方面的不足，本文提出了面向 Serverless workflow 的投机执行优化方案 SpecSW。首

先，本文在已有推测函数执行研究的基础上，引入了一个面向 Serverless 工作流的投机执行模型，对工作流中的控制依赖、数据依赖、分支预测、候选集合构造、门控与预算约束、状态缓冲以及验证恢复等关键要素进行统一、抽象的描述，为 Serverless 环境下的工作流投机执行提供一致的分析与优化框架。在此基础上，本文进一步设计了一套面向 Serverless 工作流的编程框架与运行支撑机制，使用户能够以声明式方式描述工作流拓扑、节点关系和必要的投机执行策略注解，同时为控制器侧的预测、隔离、验证与恢复等机制提供清晰的接口边界，从而降低投机执行优化的使用复杂度并增强方案的工程可落地性。

在投机执行模型和编程框架的支撑下，本文围绕分支预测提出了两类优化方法：一是引入输入感知机制，以提升预测器对输入相关分支行为的辨识能力；二是引入分布漂移适应机制，以增强预测器对负载时间变化的响应能力。在此基础上，本文进一步讨论了二者结合使用方式，从而提高投机触发决策的可靠性，为缩短关键路径和提升执行效率提供更稳定的支撑。

本文的主要贡献总结如下：

1. **Serverless 工作流投机执行模型。**针对现有 Serverless 工作流优化研究中缺乏统一投机执行抽象的问题，本文提出了一个面向 Serverless 工作流的投机执行模型，对控制依赖、数据依赖、分支预测、门控预算、状态缓冲以及验证恢复等关键机制进行了统一描述，为后续的方法设计与系统实现提供了理论基础。
2. **面向投机执行的编程框架。**针对现有方案工程可用性不足、用户难以方便描述投机执行语义的问题，本文设计了面向 Serverless 工作流的声明式编程框架与运行配置接口，使用户能够方便地描述工作流结构、依赖关系与投机策略，并为控制器侧机制提供了清晰、可替换的接口抽象。
3. **分支预测优化方法。**针对复杂工作流场景下预测精度与适应性不足的问题，本文提出了输入感知的分支预测方法和分布漂移适应方法，并进一步实现二者的联合使用机制，从而提升了预测器对输入相关分支行为和时间变化特征的建模能力，提高了投机执行触发决策的可靠性。
4. **SpecSW 原型系统。**为验证所提出模型与优化方法的可实现性，本文基于 OpenWhisk 实现了 SpecSW 原型系统。实验结果表明，相比已有工作的基础投机方法，SpecSW 的平均端到端时延降低了 22.73%，P95 尾延迟下降

了 10.3%，错误投机率平均降低了 5.78 个百分点。

1.4 本文组织结构

本文按照以下结构组织各章节内容：

第二章为背景知识，围绕本文研究的 Serverless 工作流投机优化问题，介绍 Serverless 计算的基本概念与关键特征、Serverless 工作流的组织方式与执行特性，以及 Serverless 系统的主要性能瓶颈与相关优化技术。在此基础上，本章进一步引入投机执行思想及工作流推测优化框架。

第三章详细阐述本文提出的 Serverless 工作流投机执行优化方法。首先给出整体方案与架构设计，然后依次介绍面向 Serverless 工作流的投机执行模型、配套的编程框架，以及支撑该框架的关键优化方法——输入感知的分支预测、分布漂移适应，以及二者的联合使用机制。

第四章介绍基于第三章方法实现的 Serverless 工作流投机执行原型系统。首先说明系统所依赖的运行平台与实现环境，包括 OpenWhisk 平台及系统与平台之间的交互方式；然后阐述系统总体架构设计；最后详细介绍系统核心模块，包括工作流描述与运行组织模块、投机执行控制器、分支预测与模型状态管理模块，以及系统运行支撑模块的设计与实现。

第五章对本文提出的方法与原型系统进行实验设计与结果分析。在说明实验配置后，首先通过核心算法仿真实验评估输入感知分支预测、分布漂移适应及其联合优化的效果；然后通过功能验证实验检验系统在不同测试工作流和端到端应用工作流上的正确性；接着通过性能评估实验分析原型系统的运行收益；最后结合消融实验，对各项关键机制的作用进行进一步讨论。

第六章总结全文工作，并对未来研究方向进行展望。

第二章 背景知识

本章围绕本文研究的 Serverless workflow 投机优化问题，介绍四个方面的背景知识：首先，回顾 Serverless 计算的演进脉络与关键特征；其次，阐述 Serverless workflow 的组织方式、控制结构、性能执行特性与可预测性等；然后，从系统开销视角归纳主要性能瓶颈与优化方向并总结其边界；最后，引入投机执行及推测函数执行框架，为后续章节的系统设计与改进方法奠定背景。

2.1 Serverless 计算

Serverless 计算是近年来兴起的一种重要云计算形态。过去十余年里，云计算产业经历了高速演进，企业上云已成为常态^[26]。与此同时，云平台本身也沿着从 IaaS（托管基础设施）、PaaS（提供运行平台）到 SaaS（直接交付应用）的路径，不断迈向更高抽象、更强虚拟化与更低运维门槛的发展阶段^[27]。随着抽象层级持续上移，用户越来越无需关注底层硬件与集群管理的细节，转而更专注于按需使用计算与存储能力^[2-3]。正是在这一持续演进的趋势下，服务器无感知计算——即 Serverless 计算，亦常被称为“无服务器计算”——应运而生，下文将统一简称为 Serverless。图 2-1 直观地对比了这几种云服务模式在管理边界与抽象层次上的差异。

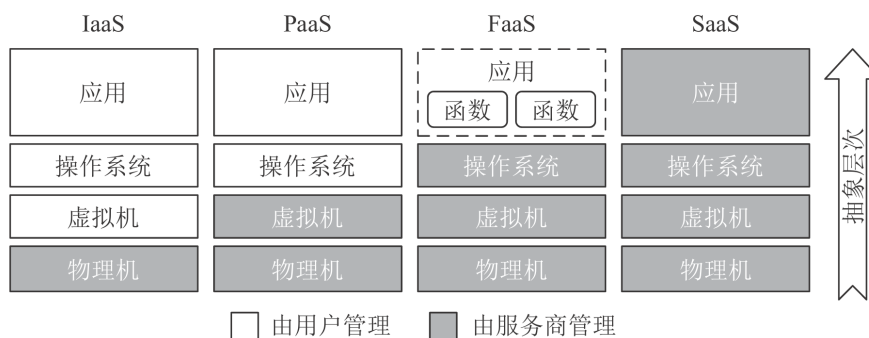


图 2-1 几种云服务模式的区别比较^[28]

2014 年，亚马逊推出 AWS Lambda^[6]，将 Serverless 概念真正带入主流视野。

与传统的 IaaS 或 PaaS 模式不同，Serverless 主要采用 FaaS（函数即服务）架构，以“函数”作为部署与调度的基本单元，并通常基于事件驱动模型运行。用户只需提交函数代码并绑定相应的触发事件（例如 HTTP 请求、对象存储的文件新增、数据库的记录更新等），其余所有环节，包括资源分配、弹性扩缩容、部署运行、监控日志乃至安全隔离，均由平台自动接管^[2]。与此同时，Serverless 普遍采用更细粒度的计费方式，按函数实际执行的消耗（如运行时长短、占用内存等）进行收费，而非按预先分配的资源规模长期计费，从而在成本层面实现了更精准的“按需使用”^[29]。

总体而言，Serverless 的典型特征体现为函数生命周期短暂且强调无状态设计、用户侧运维负担大幅减轻以及按实际使用量计费三大方面。具体而言，其函数实例通常存活时间较短，且被设计为无状态运行，业务状态需存储于外部持久化服务中。这一约束虽然在一定程度上限制了开发的自由度，却也显著提升了平台调度与弹性伸缩的可行性^[12]。对于用户而言，其运维负担得以大幅减轻，只需聚焦于业务代码与少量配置，而底层资源的管理、调度与维护工作完全由平台负责。在计费模式上，Serverless 以函数执行时长、内存配置等实际消耗作为计费依据，计费粒度可精细至毫秒级别，从而使得成本更加贴合真实业务需求。

得益于上述优势，Serverless 在工业界得以快速普及：继 AWS Lambda 之后，谷歌 Cloud Functions^[7]、微软 Azure Functions^[8]、IBM Cloud Functions^[30] 等主流云厂商相继推出类似服务；国内云服务商如阿里云函数计算^[9]等也提供了成熟的函数计算能力。在开源生态中，OpenWhisk^[31]、OpenLambda^[32]、OpenFaaS^[33]、Fission^[34]、Knative^[35]等项目相继涌现，推动了 Serverless 技术的标准化与多样化发展。

在应用层面，Serverless 早期主要服务于突发性强、并发量波动的 Web 业务（如图片实时处理、API 后端等），随后其应用场景不断拓展，已逐步延伸至大数据处理、机器学习推理、视频转码、持续集成与科学计算等更为复杂的领域，应用边界持续扩大^[28]。值得注意的是，随着业务复杂度提升，越来越多的 Serverless 应用不再由单个函数独立完成，而是由多个函数按控制逻辑与数据依赖关系组成端到端处理链路，表现为多步骤的函数组合与流程化执行。此类“多函数、多阶段”的运行形态使得系统性能不再仅取决于单次函数调用的执行效率，还与跨函数编排、事件触发、状态管理以及关键路径的依赖结构密切相关。因而，在

讨论 Serverless 的性能瓶颈与优化机制之前，有必要首先明确 Serverless 工作流的组织方式与执行特性。下一节将围绕 Serverless 工作流展开介绍。

2.2 Serverless 工作流与执行特性

随着 Serverless 平台从“单函数触发式任务”逐步进入业务主链路与数据处理管道，越来越多的应用不再由单个函数独立完成，而是由多个函数按控制逻辑与数据依赖关系组成端到端流程，即 **Serverless 工作流 (Serverless Workflow)**^[10]。

与传统单体应用或常驻服务不同，Serverless 工作流以函数为最小执行单元，依赖事件触发与平台调度完成跨函数的串联与并发。其结构可抽象为带控制依赖与数据依赖的函数调用图，工程上常使用有向无环图 (DAG) 或带循环结构的有向图加以描述^[11,36]。这一抽象在后续分析中尤为关键：工作流端到端性能往往由关键路径上的等待与交互开销主导，而这些开销通常与控制流不确定性及依赖就绪时机紧密相关。

2.2.1 组织模式

从系统组织方式看，Serverless 工作流通常可分为**编排 (Orchestration)**与**协同 (Choreography)**两类^[37]。

编排模式下，存在中心化的工作流引擎或编排器，负责维护状态机、决定下一步执行分支并触发对应函数；典型实现包括 AWS Step Functions^[14]、Google Workflows^[15]等托管编排服务，以及基于持久化状态的函数编排框架（如 Durable Functions^[16]）。该模式的优势在于控制逻辑集中、可观测性较强，失败恢复与重试策略易于统一管理；但其代价在于引入中心控制面的额外交互与状态持久化开销，在长链路、细粒度步骤场景下可能显著增加端到端时延^[10,17]。另一方面，中心编排器也天然提供了“全局控制点”：其对分支选择、循环退出条件等控制决策的维护方式，将直接影响后续可并行化程度与关键路径长度，为后续讨论“面向控制不确定性的优化”提供了系统语境。

协同模式下，系统不依赖单一中心编排器，而是由函数之间通过事件总线、消息队列或触发器规则进行松耦合联动；每个函数在完成自身逻辑后发布事件，驱动后继函数执行^[13]。其优势是组件自治、扩展性强、对中心控制面依赖较弱；

但缺点是全局控制逻辑分散，跨步骤的状态追踪、补偿事务（saga）以及端到端调试更具挑战^[38]。在实际系统中，两种模式常被混合采用：主流程使用编排服务以保证可管理性，局部子流程使用事件驱动协同以提高扩展性。

2.2.2 控制结构

从控制流结构看，Serverless 工作流通常由四类基本模式构成^[39]，如图 2-2 所示。

顺序（Sequence）描述线性依赖的多步骤处理；分支（Branch/Choice）根据条件选择不同后继路径；并行（Parallel/Fan-out & Fan-in）将输入拆分为多个并发任务并在后续汇聚；循环（Loop/Iteration）重复执行某一子流程直至满足终止条件。这些结构可进一步组合形成更复杂的流程，例如“并行-汇聚-分支”或“分支-循环-并行”等。对于本文关注的工作流加速问题，尤其需要强调两点：其一，分支与循环引入**控制流不确定性**，后继函数集合在判定之前并不确定；其二，顺序与汇聚点形成**同步约束**，会将慢分支（straggler）或上游判定延迟放大为端到端尾延迟。

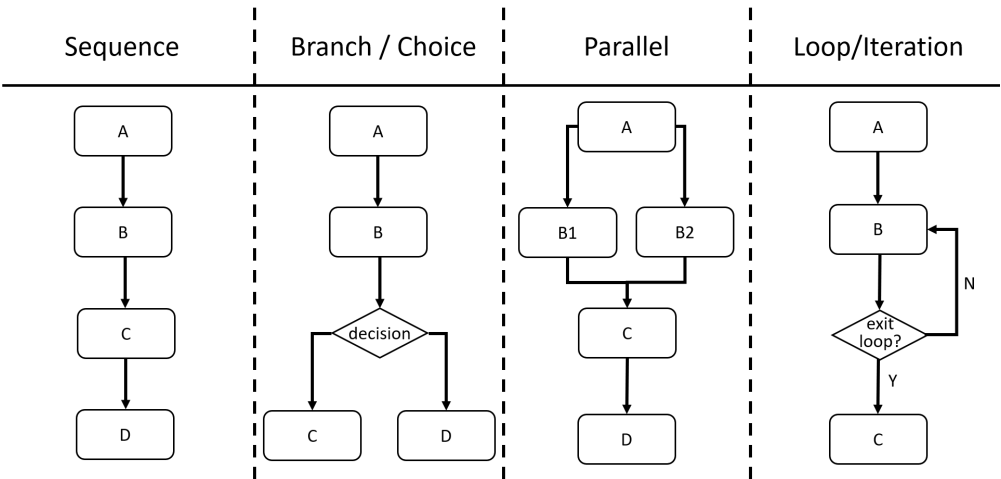


图 2-2 Serverless 工作流的四类基本模式

顺序结构的端到端时延可近似视为各步骤执行时间与跨步骤平台交互开销之和；并行结构可通过 fan-out 提升吞吐并缩短阶段耗时，但在 fan-in 汇聚点形成同步屏障（barrier），其阶段完成时间受最慢分支影响^[36,40]。分支结构则体现为“先判定、后触发”的执行逻辑：分支条件通常由上游函数计算产生，在判定结果出现之前，平台往往采取保守策略以避免无效计算；循环结构进一步带来

执行次数与运行时间的不确定性，使得超时策略、重试策略与成本控制更为复杂^[10]。这些现象共同指向一个事实：在工作流中，端到端性能瓶颈往往与“后继步骤何时可确定、何时可启动”密切相关，而不仅是单步函数本身的计算效率。

除控制结构本身外，工作流在真实生产负载下还呈现出与投机执行高度相关的统计特性：其一，分支路径往往具有显著的概率偏向，即少数路径在多数输入上被高频选择；其二，许多函数在给定输入下具有较强确定性，输出随输入变化的模式相对稳定，从而使得“基于输入/历史”的路径预测具备可行性。有分析指出，Serverless 应用中存在大量重复/相似调用与可预测的下游依赖关系，这为在控制决策尚未产生时提前推进潜在后继步骤提供了经验依据^[24]。

上述特性对端到端性能具有直接含义：当关键路径上存在“判定一触发”的等待空窗期时，若后继路径高度偏向，则对高概率路径进行选择性投机更可能命中并获得稳定收益；相反，若路径分布接近均匀或输入扰动导致预测不稳定，则投机错误率上升并可能引发资源竞争，从而抬升尾延迟^[18]。

2.2.3 数据流与状态管理

Serverless 平台普遍倡导函数无状态运行，函数实例的本地内存与临时存储不保证跨调用持久，业务状态通常外置于对象存储、数据库、KV 存储或编排器的状态存储中^[12,41]。在工作流场景下，这一特征带来两方面影响：其一，跨步骤的数据传递通常需要经过外部介质的序列化与反序列化，增加数据搬运与网络交互开销；其二，状态外置使得失败恢复与重试具备可行性，但也要求应用满足幂等性（idempotency）与可重放（replay）等工程约束。从系统设计角度看，外部持久化与幂等约束不仅是可靠性机制的基础，也为后续在“保持语义正确”前提下开展更激进的执行策略提供了实现条件（例如允许重复触发、补偿与回滚等）。

实际系统中，平台通常提供“至少一次（at-least-once）”的事件投递与触发语义，以提高可靠性，但这意味着同一函数可能被重复触发，进一步强化了幂等性要求。此外，工作流还常配置超时（timeout）、重试（retry）、补偿（compensation）等容错机制^[13]。这些机制增强了鲁棒性，但也会在链式调用中累积控制开销，并在发生错误或抖动时放大尾延迟^[17,40]。

2.2.4 性能执行特性

综上，Serverless 工作流呈现出若干与性能密切相关的执行特性。

首先，工作流执行通常遵循**依赖驱动**原则：只有当前置步骤完成并产出必要数据后，后继函数才会被触发执行。该保守策略保证了正确性，但在存在分支选择不确定或数据依赖较强时，会显著限制并行机会并拉长关键路径^[36,42]。其次，函数作为细粒度单元带来**更频繁的调度与交互**：每一次跨函数边界都可能涉及事件投递、运行时就绪、权限校验、日志与监控等平台开销^[17]。最后，由于工作流包含多次触发与状态交互，单次函数层面的开销（如冷启动、排队等待、数据序列化与网络传输）会在端到端链路中**多次叠加并被放大**，使性能问题更集中地体现为整体延迟与尾延迟上升^[43]。

上述特性表明：工作流性能瓶颈并非仅由单函数执行效率决定，更与**控制流决策何时产生、依赖何时就绪以及平台是否能减少保守等待**相关。若能在不破坏语义正确性的前提下，对关键控制决策（例如分支走向、循环迭代趋势）进行更高精度的判断，并据此提前准备乃至提前执行部分后继计算，则有望缩短关键路径、提升并行度并降低尾延迟。下一节将从系统开销视角归纳性能优化方向，并在小结中指出其在控制不确定性下的局限，为后续引入投机执行奠定动机。

2.3 Serverless 系统性能瓶颈与优化技术概览

Serverless 平台通常以“短生命周期函数 + 沙箱隔离 + 外部化状态/通信”为主要运行形态。这一形态在降低用户侧运维与提升弹性方面具有优势，但也带来了启动、数据搬运与调度等额外系统开销，并在多函数组合场景下沿调用链路叠加放大^[4]。

围绕上述开销，相关研究工作可以归纳为四类方向：面向典型任务的系统优化、沙箱环境与冷启动优化、I/O 与通信优化以及资源调度优化^[28]。本节据此概述主要瓶颈与代表性思路，并结合本文关注的多函数组合与流程化执行形态，说明这些优化在端到端链路中可能产生的叠加效应。

2.3.1 面向典型任务的系统优化

许多传统应用并非天然适配函数级、无状态与外部化通信的约束。如果将原有程序机械拆分为细粒度函数，并主要依赖远程存储/远程调用完成跨步骤协作，则同步等待与数据搬运开销往往成为主导，进而导致端到端性能下降^[1]。

因此，常见做法是围绕任务形态进行结构化改造：通过调整函数粒度、合并调用、引入批处理或阶段化执行来减少跨函数边界次数，从而降低调度与通信开销并提升资源利用率。例如，INFless 面向推理服务将批处理作为平台内建能力，并采用非均匀批处理与伸缩策略以兼顾低延迟与高吞吐^[44]。总体而言，此类方法以工程改造换取系统开销下降，对端到端链路中的“边界次数”与“同步点”更为敏感，但其收益也取决于任务可重构空间以及业务对时延/成本的约束边界^[28]。

2.3.2 沙箱环境与冷启动优化

冷启动时延通常由运行时创建、依赖加载与初始化等环节叠加产生，是 Serverless 平台中最典型且被广泛研究的延迟来源之一^[43]。现有方法主要沿“重用”与“预载入”两条路径降低该开销。

在“重用”路径中，快照/检查点技术是代表性思路：Prebaking 基于 CRIU 保存并恢复已初始化进程状态，以跳过启动关键路径^[45]；SEUSS 借助 unikernel 级环境快照实现更高密度缓存与更快部署^[46]；Catalyzer 进一步提出按需恢复与 sfork 等机制，以复用运行中沙箱状态实现更快启动^[19]。

在“预载入”路径中，系统更强调“提前准备”而不是“被动等待”。当平台能够利用函数调用图或历史信息时，可在关键阶段提前预热/预载入：ORION 面向 Serverless DAG 同时考虑 sizing、bundling 与 prewarming，以降低初始化延迟并控制资源利用率^[25]；Kraken 通过面向动态 DAG 的容器供给策略减少不必要的容器准备与冷启动开销^[21]。在多步骤组合链路中，冷启动往往具有累积效应，任一环节的启动抖动都可能抬升端到端尾延迟^[43]。

2.3.3 I/O 与通信优化

在 Serverless 平台中，函数本地持久存储受限且函数间直接通信常被限制，数据交换与状态共享通常依赖外部存储与中间服务；因此 I/O 与通信开销在许多应用中成为端到端性能的重要决定因素^[47]。现有工作常从“降低远程访问成本”和“减少交互次数/选择更优路径”两方面入手。

在降低远程访问成本方面，Locus 通过组合不同存储层级来支持 serverless analytics 的 shuffle，在成本与性能之间取得折中^[48]。在路径选择方面，SONIC 面向链式（DAG）应用为每条依赖边自适应选择数据传递方式，并结合通信感知的函数放置以降低延迟与成本^[20]。

此外，也有系统通过减少 RPC 次数与合并请求来降低远程交互开销。MX-FaaS 通过共享/复用多种资源，并对远程存储访问与远程函数调用进行合并（coalescing），提升吞吐并降低尾延迟^[22]。对于多函数组合而言，这类优化往往与跨边界数据搬运次数高度相关：边界越多、依赖越密集，数据路径收敛对关键路径缩短越明显^[11]。

2.3.4 资源调度优化

资源调度会影响函数实例的排队等待、复用率与并发扩展速度，是决定吞吐与尾延迟的重要系统因素^[47]。平台侧研究通常以提升资源利用率与保持负载均衡为目标，并逐步引入预测与学习机制。

例如，Fifer 基于执行时间预测与“松弛时间（slack）”思想控制排队与批量执行，以减少资源闲置并提升容器利用率^[49]；Libra 通过对动态资源需求进行画像并安全回收闲置资源，再将其用于加速后续调用，从而同时改善性能与利用率^[23]。在多阶段链路中，调度不均衡还可能放大慢实例拖尾效应，进而影响汇聚点与关键路径完成时间，因此资源调度优化不仅关乎平均时延，也直接决定端到端尾延迟的上界与波动范围^[4]。

2.3.5 小结与系统层优化边界

总体而言，围绕任务适配、冷启动、I/O/通信与资源调度的优化工作，从不同层面降低了系统开销并提升资源利用率，也为多函数组合链路提供了更可控

的运行基础。但这类方法主要针对资源准备与系统开销，在执行语义上通常仍以“依赖就绪后触发”为主：当应用包含分支、循环等控制不确定性时，关键路径上的等待仍可能成为端到端时延的主导因素。

基于此，下一节将引入投机执行思想，为后续从“减少保守等待”角度进一步优化端到端性能奠定背景。

2.4 投机执行思想与 workflow 推测优化

前述优化工作主要从任务适配、冷启动、I/O/通信与调度等角度降低系统开销，但在包含分支、循环等控制不确定性的 workflow 中，端到端关键路径往往仍受“依赖就绪后触发”的保守语义约束：后继函数必须等待控制决策产生后才能启动，从而在链路上引入不可忽略的空窗期。投机执行（Speculative Execution，亦称推测执行）提供了另一类互补思路，其核心是在结果尚未确定前，基于预测提前推进潜在后继计算，以“可能的无效工作”换取关键路径缩短与并行度提升。该思想最早在处理器分支预测与乱序执行等机制中系统化发展，并被广泛用于分布式系统的慢任务与尾延迟治理^[50-51]。

从机制抽象看，投机执行可概括为“**预测—提前执行—验证提交/失败丢弃**”三步：系统通过预测器对未来路径作出判断，提前执行可能被触发的后继步骤；当不确定性被消解后，若预测正确则提交并复用投机结果以缩短关键路径，若预测错误则丢弃无效结果并回到正确执行路径。投机执行能否获得稳定收益，取决于**预测准确率**与**投机代价**之间的权衡：准确率越高、错误代价越低，投机收益越显著；反之则可能因资源浪费与竞争干扰而抬升尾延迟，出现“优化反噬”的现象^[18]。因此，在系统落地层面，通常需要结合隔离、选择性触发与资源约束等手段，以降低错误推测带来的资源浪费与拥塞风险。

将投机执行迁移到 Serverless workflow，需要额外面对外部化状态与副作用带来的挑战。由于函数通常通过数据库、对象存储、消息队列等外部系统读写状态，相比处理器内部状态更难回滚，投机执行必须对副作用进行严格管控，例如限制投机阶段为只读、将写入导向临时命名空间/影子对象，或采用延迟提交在路径确认后再使写入可见，以保证语义正确性与可恢复性。与此同时，workflow 的投机收益往往体现为提前创建运行时或容器、提前加载依赖与数据以及提前执

行计算，从而减少后继步骤在关键路径上的等待；其代价则主要表现为额外资源消耗与可能的资源竞争，这也说明推测优化的有效性在很大程度上取决于错误推测是否能够被控制在可接受范围内。

面向上述特征，已有研究尝试将投机执行思想系统化落地为 Serverless 场景中的推测函数执行框架。典型代表是 SpecFaaS^[24]，其对工作流后继路径进行预测，在真实分支结果尚未确定时提前启动可能被触发的后继函数，并在预测命中时复用推测结果以缩短端到端关键路径，从而降低总体时延与尾延迟。该类框架表明，面向控制不确定性的投机执行能够突破“等待判定再触发”的保守边界，但其效果仍高度依赖于预测精度及错误推测代价的可控性：一方面需要更细粒度、输入相关的预测以提高命中率；另一方面也需关注错误推测可能带来的额外资源消耗与性能扰动。

基于上述背景，后续章节将以工作流级推测函数执行框架为对象，进一步介绍其核心设计与关键机制，并围绕**预测精度提升**这一主线展开改进。在此基础上，本文主要关注如何通过更准确的分支预测减少错误推测与无效执行，从而在保证语义正确性的前提下提升工作流端到端性能。

2.5 本章小结

本章介绍了本文研究所需的背景知识。首先，回顾 Serverless 计算的基本概念与运行形态，明确其以函数为单位、事件驱动与外部化状态/通信等关键特征。其次，讨论 Serverless 工作流的组织模式、控制结构与状态管理机制，指出多函数组合链路中由同步约束与控制不确定性引入的关键路径等待问题。随后，从系统开销视角归纳任务适配、冷启动、I/O/通信与资源调度等主流优化方向，并说明其在“依赖就绪后触发”语义下对控制不确定性的作用边界。最后，引入投机执行思想及其在工作流中的推测函数执行形态（如 SpecFaaS），为后续章节围绕预测精度与推测开销控制开展改进奠定基础。

第三章 Serverless workflow 投机执行优化方法

本章围绕 Serverless workflow 投机执行优化方法展开，内容主要包括三个部分：投机执行模型、相应的编程框架，以及支撑该框架的关键优化策略。章节首先给出整体方案与系统架构，随后分别阐述投机执行模型的定义与运行机制，并说明编程框架的设计思路与实现方式；最后，对优化方法进行详细说明。

3.1 方法概述

图 3-1 展示了本文提出的 SpecSW（Speculative Execution in Serverless Workflows）：面向 Serverless workflow 的投机执行优化方案。

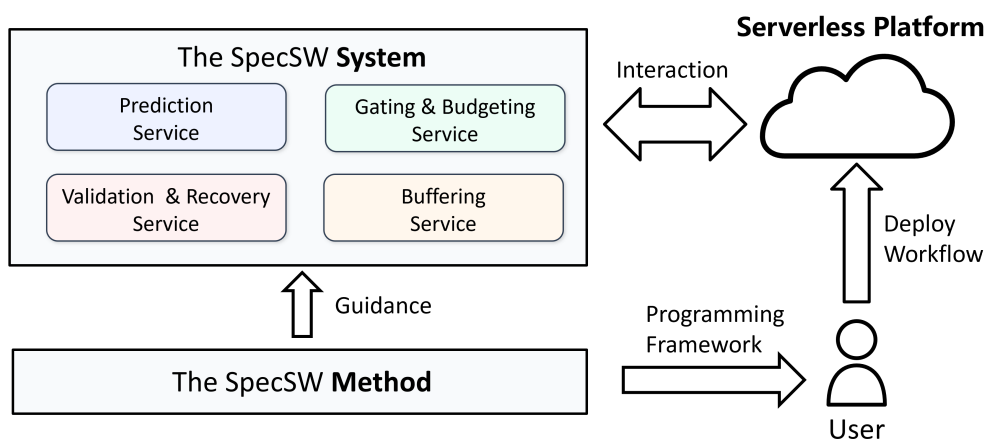


图 3-1 面向 Serverless workflow 的投机执行优化方案 SpecSW 示意图

该方案涉及用户、Serverless 平台与原型系统等多个角色：用户以 workflow 为单位组织应用逻辑，并通过编程框架描述函数节点、依赖关系与必要的运行时注解；Serverless 平台负责函数实例的资源供给与隔离执行，并提供对象存储、数据库、消息队列等外部状态作为跨函数的数据与副作用承载；原型系统作为平台侧增强组件，在不改变应用对外语义的前提下对 workflow 执行过程进行投机化加速。在运行时，系统持续追踪 workflow 拓扑与执行进度，并在分支判定尚未揭示时构造分支点上下文，调用预测器估计后继概率分布，进而形成候选集合并提前触

发投机路径；与此同时，系统对投机执行期间产生的中间结果与外部读写进行隔离缓冲，并在真实分支结果揭示后执行验证与恢复，仅允许与真实执行一致的结果提交为对外可见状态，从而保证外部语义与严格依赖执行一致。

图 3-2 展示了方法的基本架构。本文首先引入一个面向 Serverless workflow 的投机执行模型，以统一、抽象的方式定义投机执行的关键要素；在此基础上，进一步设计配套的编程框架，为 workflow 描述、运行接口与投机控制器组织方式提供规范化抽象；最后，针对复杂 workflow 场景下预测准确性与适应性不足的问题，本文将优化重点聚焦于分支预测环节，主要从输入与分支走向的相关性以及时间变化两个维度对基础预测机制进行改进，并进一步讨论二者的协同建模思路。具体而言，SpecSW 主要包括以下内容：

1. **Serverless workflow 投机执行模型。**本文给出一个形式化的投机执行模型，规范化定义模型中的关键组件与交互关系，包括 workflow 的控制/数据依赖表示、分支预测与候选集合构造、门控与预算约束、投机状态隔离与可见性规则，以及判定揭示后的验证提交与误投机撤销。该模型为后续编程框架设计与优化技术选择提供统一的描述基础。
2. **Serverless workflow 投机执行的编程框架。**本文设计配套的编程框架与运行支撑，面向 workflow 描述、运行接口与投机控制器组织方式提供规范化抽象，使用户能够以声明式方式表达 workflow 拓扑、依赖关系与必要的投机策略注解；同时，为控制器侧的预测、隔离、验证与恢复等机制提供清晰边界与可替换接口，以支持不同平台与不同工作负载下的工程化落地。
3. **投机执行的关键优化方法。**在投机执行模型与编程框架的基础上，本文将优化重点聚焦于分支预测环节，围绕复杂 workflow 场景下预测准确性与适应性不足的问题提出两类改进方法：一是引入输入感知机制，以提升同一路径下对不同输入模式及其分支倾向的区分能力；二是通过强调近期观测，增强预测器对分布漂移的响应能力。在此基础上，本文进一步探讨两类机制的协同作用与分层集成思路，使预测器能够同时兼顾输入相关分支行为建模与时间变化适应能力。上述方法在不改变投机执行基本闭环的前提下，提高了投机触发决策的可靠性，为后续并行推进带来更稳定的性能收益。

本文从模型定义、框架设计与预测增强三个层面系统地讨论了如何在 Server-



图 3-2 面向 Serverless 工作流的投机执行优化方案架构示意图

less 环境下实现工作流执行加速。所提出的方法不仅能够为用户描述和实现投机执行提供统一抽象，支撑工作流在真实平台上的工程化落地，也能够通过更准确的分支预测与更稳健的投机控制，提升工作流执行过程中的有效并行度，降低关键路径等待开销，从而缓解 Serverless 工作流场景下面临的高延迟与低资源利用率问题。

3.2 Serverless 工作流投机执行模型

3.2.1 模型定义

在传统的工作流执行引擎中，通常采用严格按控制依赖触发的执行模型。该模型在运行时仅当上游节点完成并产生明确的控制判定后，才触发其后继函数实例，从而保证执行语义简单、资源开销可控^[14,16]。

以典型的编排式 Serverless 工作流为例，编排器（Orchestrator）会维护工作流状态机，并在每个分支点等待判定函数返回后再选择后继路径执行；对于并行-汇聚结构，汇聚节点需等待所有并行分支完成后才能继续推进，该执行模型示意如图 3-3 所示。

这一严格依赖模型能够提供清晰的因果顺序与一致性保障，但在包含分支与循环等控制结构的工作流中，判定等待与同步约束往往会被放大为端到端尾延迟：一方面，分支点在判定完成前无法启动后继，导致下游执行时间无法被隐藏；另一方面，汇聚点的完成时间由最慢分支决定，慢分支（straggler）会对关

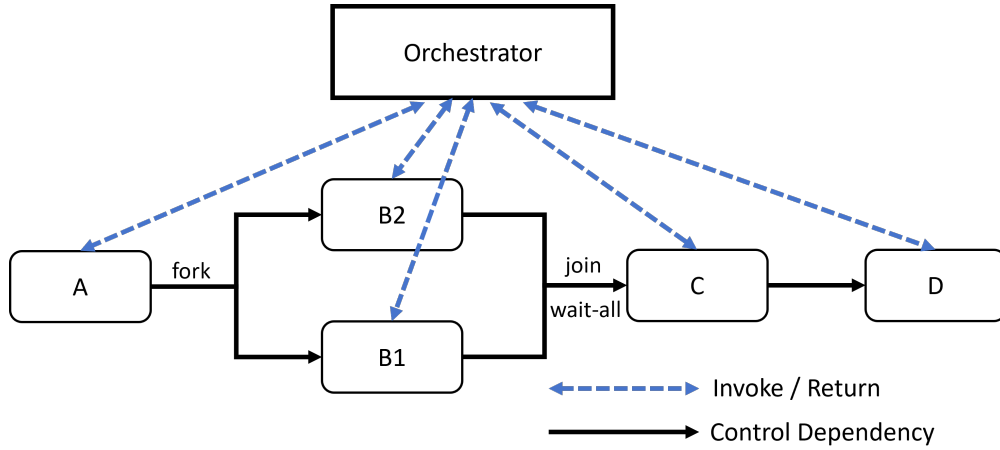


图 3-3 编排式 Serverless workflow 示意图

键路径产生显著拖累^[36,40]。更重要的是，在 Serverless 平台中，函数实例的排队等待、冷启动与并发配额等系统因素会进一步放大上述问题，使得严格依赖执行在高负载与长尾延迟场景下难以获得稳定性能。

针对上述局限性，本文在 SpecFaaS 论文工作的基础上^[24]，提出面向 Serverless workflow 的投机执行模型。其核心思想是在控制判定尚未确定时，基于输入与历史信息对后继分支进行预测，并提前触发候选后继的执行，以利用平台弹性并行能力隐藏判定等待并缓解慢分支影响，从而降低端到端延迟。

同时，该投机执行模型将投机过程的基本要素（分支决策、候选集合、门控阈值、预算约束、验证与撤销）及其行为特征进行规范化定义，为后续的编程框架设计与优化技术选择提供理论依据：通过对组件职责与交互方式的形式化刻画，可以指导控制器、预测器与状态机制的设计实现，并为分支预测、门控决策、资源预算与无效投机回收等优化方法奠定基础。

下面给出 Serverless workflow 投机执行模型的形式化定义：

定义 3.1 (Serverless workflow) 一个 Serverless workflow 可表示为一个七元组：

$$\mathcal{W} = (V, E_c, E_d, v_0, V_T, \kappa, \sigma). \quad (3-1)$$

其中：

- V 为 workflow 节点集合，节点 $v \in V$ 对应一个 Serverless 函数。
- $E_c \subseteq V \times V$ 为控制依赖边集合， $(u, v) \in E_c$ 表示 v 的触发受 u 的控制判定约束。

- $E_d \subseteq V \times V$ 为**数据依赖边**集合, $(u, v) \in E_d$ 表示 v 的输入依赖于 u 的输出 (或其派生状态)。
- $v_0 \in V$ 为起始节点, $V_T \subseteq V$ 为终止节点集合。
- κ 为节点类型映射 (TASK / BRANCH / JOIN / LOOP)。
- σ 为状态访问描述, 用于刻画节点对外部状态 (如对象存储/数据库/消息队列等) 的读写集合与键空间约束。

上述定义将工作流的控制结构与数据依赖显式分离: E_c 刻画“何时可以触发”, E_d 刻画“以何种输入执行”。在严格依赖执行中, 节点仅当其控制前驱完成并产生确定判定、且所需输入均可用时才允许触发; 而投机执行将允许在控制判定未决时提前触发候选后继, 但必须通过后续验证以保证对外可见语义与严格依赖执行一致。

定义 3.2 (投机执行模型) 面向 *Serverless* 工作流的投机执行模型可定义为一个六元组:

$$\mathcal{M} = (\mathcal{W}, C, \mathcal{P}, \mathcal{G}, B, \mathcal{R}). \quad (3-2)$$

其中:

- $\mathcal{W}$ 为定义 3.1 的工作流。
- C 为**控制器 (Controller)**, 维护运行时状态并负责触发函数实例, 可抽象为

$$C : (\text{state}, \text{event}) \rightarrow \text{action}. \quad (3-3)$$

- $\mathcal{P}$ 为**分支预测器 (Predictor)**, 用于在分支点未决时估计后继路径的概率分布。
- $\mathcal{G}$ 为**门控与预算策略 (Gating & Budgeting)**, 决定是否投机以及投机的规模。
- B 为**状态缓冲 (Buffer)**, 用于隔离投机执行对外部状态的读写副作用, 并支持依赖转发与冲突检测。
- $\mathcal{R}$ 为**验证与恢复机制 (Validation & Recovery)**, 在判定揭示后提交正确路径并撤销错误投机。

图 3-4 给出了投机执行模型 $\mathcal{M}$ 的总体视图，展示了 $\mathcal{W}$ 、控制器 $\mathcal{C}$ 及其内部组件 $\mathcal{P}/\mathcal{G}/\mathcal{B}/\mathcal{R}$ 与函数实例之间的交互关系。

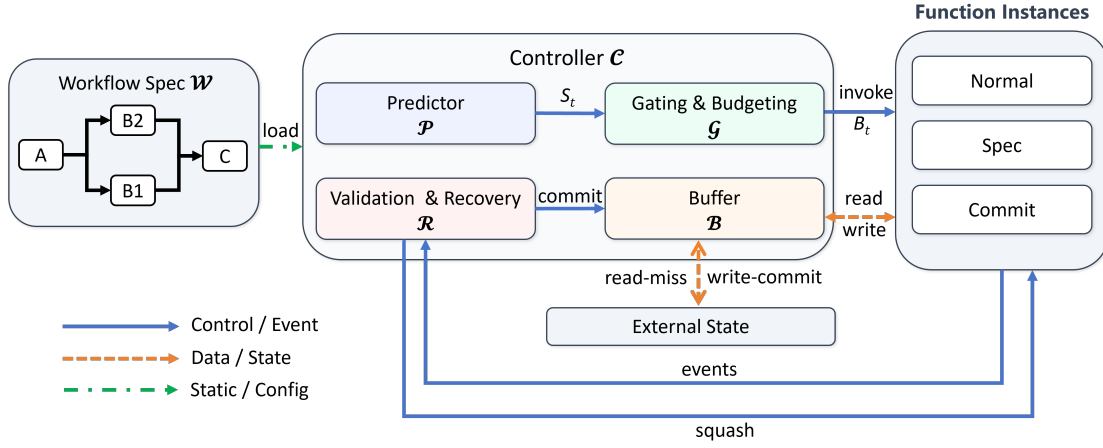


图 3-4 投机执行模型总体视图

定义 3.3 (分支预测与候选集合) 对任意分支节点 $b \in V$, 设其后继集合为 $Succ(b) = \{v_1, \dots, v_m\}$, 预测器输出后继概率分布:

$$\mathcal{P}(b, c_t) = \pi_t, \quad \pi_t(v_i) = \Pr(v_i | b, c_t), \quad \sum_{i=1}^m \pi_t(v_i) = 1. \quad (3-4)$$

据此定义候选投机集合为

$$S_t = \{v_i \in Succ(b) | \pi_t(v_i) \geq \tau\}. \quad (3-5)$$

其中:

- $c_t = \langle x_t, h_t \rangle$ 为分支点运行时上下文; x_t 为当前分支点可观测输入特征, h_t 为路径历史 (如最近 K 个已提交节点的标识序列)。
- $Succ(b)$ 表示分支点 b 的所有可能后继节点集合。
- $\pi_t(\cdot)$ 表示在上下文 c_t 下的后继概率估计, 满足归一化约束 $\sum_{i=1}^m \pi_t(v_i) = 1$ 。
- $\tau \in (0, 1)$ 为门控阈值, 用于控制是否对某一后继进行投机触发。
- S_t 可为单元素集合 (Top-1 投机), 也可包含多个候选 (N-way 投机), 以在不确定性较高时提高命中概率。

如图 3-5 所示, 预测器在分支点 b 的上下文 c_t 下输出后继概率分布 π_t , 并对每个后继 $v_i \in Succ(b)$ 给出估计概率 $\pi_t(v_i) = \Pr(v_i | b, c_t)$ 。

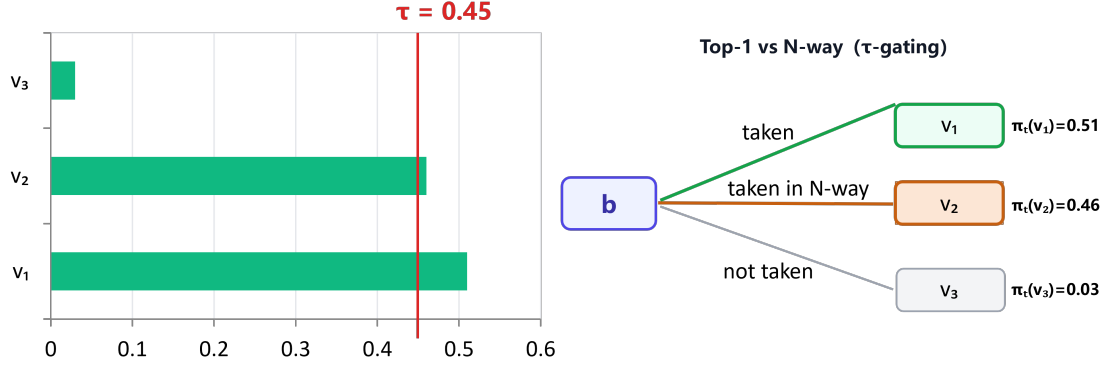


图 3-5 分支预测与候选集合构造示意图

控制器采用单阈值门控 (τ -gating), 若采用 Top-1 策略, 则选择概率最大的 v_1 作为唯一投机目标; 若采用 N-way 策略, 则将所有满足阈值条件的 v_1, v_2 纳入候选集合 S_t 。

定义 3.4 (预算约束) 为限制投机带来的资源膨胀, 设系统在任意时刻 t 的投机并发上限为 B_t , 则投机策略需满足

$$|I_t^{spec}| \leq B_t. \quad (3-6)$$

其中:

- I_t^{spec} 表示时刻 t 正在运行的投机实例集合。
- B_t 可由平台并发配额、租户预算或控制器自适应策略给出 (例如随系统负载动态调整)。

定义 3.5 (状态缓冲与可见性) 为保证投机执行不破坏外部可见语义, 模型引入状态缓冲 B , 将外部写入划分为缓冲写与提交写两阶段。设外部状态键空间为 $\mathcal{K}$, 值空间为 $\mathcal{V}$, 缓冲可形式化为映射:

$$B : \mathcal{K} \rightarrow (\mathcal{V} \times \text{id} \times \text{tag}), \quad (3-7)$$

其中 $\text{tag} \in \{\text{Spec}, \text{Commit}\}$ 标识写入处于投机缓冲或已提交状态。

其中:

- id 标识产生该写入的函数实例。

- 读操作遵循“先缓冲后外部”的规则：当键 k 在缓冲中存在可用版本时优先读取缓冲值，否则读取外部已提交状态。
- 写操作在投机阶段仅写入缓冲，在验证通过后才对外部提交，从而避免错误路径副作用外泄。

3.2.2 模型的运行时行为

投机执行模型的运行时行为刻画了控制器如何在判定未决时提前触发、并在判定揭示后完成验证与恢复。为清晰起见，避免将实现层面的调度细节引入模型定义，本文将函数实例划分为三类状态：NORMAL（严格依赖触发）、SPEC（投机触发）与 COMMIT（验证通过并提交）。

如图 3-6 所示，若严格依赖触发，实例将进入 NORMAL 状态；若推测触发，实例将进入 SPEC 状态。SPEC 实例在判定揭示后若通过一致性验证则晋升为 COMMIT；若发生误投机，则由控制器触发 squash 动作终止实例并丢弃其投机副作用。

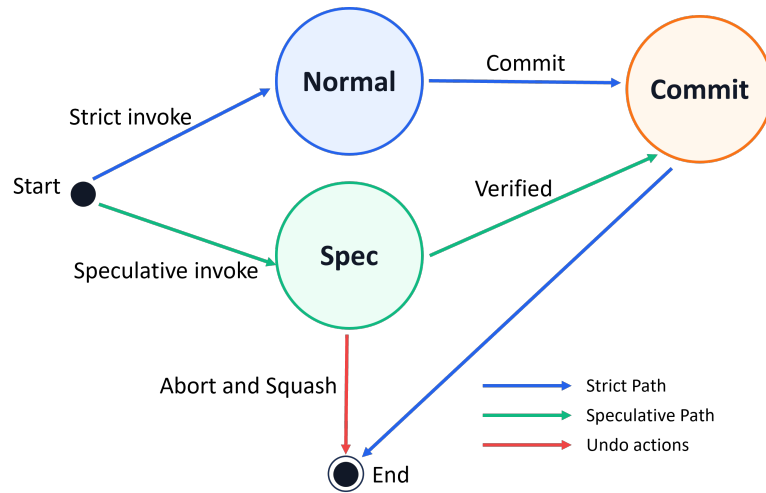


图 3-6 函数实例状态机

据此，控制器在运行时的关键操作可抽象为以下五个环节，分别对应就绪判定、分支预测触发、状态隔离与依赖转发、验证提交/撤销，以及控制投机与全投机两种模式：

(1) 严格依赖触发与就绪集合。设 $\text{Pred}_c(v) = \{u \mid (u, v) \in E_c\}$ 、 $\text{Pred}_d(v) = \{u \mid (u, v) \in E_d\}$ 分别为控制/数据前驱集合。严格依赖执行下，节点 v 在时刻 t 的就绪条件可表示为

$$\text{ready}(v, t) = \left(\forall u \in \text{Pred}_c(v), u \text{ 已完成且控制判定已确定} \right) \quad (3-8)$$

$$\wedge \left(\forall u \in \text{Pred}_d(v), u \text{ 的依赖数据已可用} \right).$$

控制器 C 维护就绪集合 $\mathcal{Q}_t = \{v \mid \text{ready}(v, t)\}$ 并按调度策略触发实例执行。

(2) 分支点的预测触发。当分支节点 b 被触发并开始执行后, 在其判定结果尚未返回时, 控制器构造上下文 c_t 并调用预测器得到 $\pi_t = \mathcal{P}(b, c_t)$, 进而计算候选集合 S_t 。对任意 $v \in S_t$, 若满足预算约束 $|I_t^{\text{spec}}| < B_t$, 控制器可提前触发 v 的投机实例, 并将其实例状态标记为 SPEC。

(3) 投机执行的状态隔离与依赖转发。投机实例在执行过程中可能访问外部状态。为避免错误路径副作用外泄, 投机写入仅进入缓冲 B , 不可直接对外部提交; 投机读遵循缓冲优先规则, 并在存在“生产者—消费者”依赖时进行依赖转发: 即消费者优先读取由其上游(可能为投机实例)产生的缓冲版本, 以隐藏数据等待。

(4) 验证、提交与撤销。当分支节点 b 的判定结果返回并确定真实后继 $v^* \in \text{Succ}(b)$ 后, 控制器执行验证与恢复机制 $\mathcal{R}$:

- **命中:** 若 v^* 已以 SPEC 状态运行并完成, 且其缓冲读写与依赖一致性检查通过, 则将该实例升级为 COMMIT, 并将其缓冲写原子提交到外部状态; 随后继续推进下游节点触发。
- **未命中:** 若 v^* 未被投机触发, 则按严格依赖触发 v^* 的 NORMAL 实例执行。
- **误投机:** 对所有 $v \in S_t \setminus \{v^*\}$ 的投机实例, 执行撤销: 终止实例、丢弃其缓冲写, 并递归撤销由其触发的后继投机实例(若存在)。

对于并行—汇聚结构, 验证点可视为汇聚节点 (JOIN): 当各并行分支的真实集合确定并完成后, 控制器仅允许来自真实分支的提交写对外可见, 从而保证与严格依赖语义一致。

(5) 控制投机与完全投机。根据投机对象的不同, 模型支持两种工作模式:

- **控制投机 (Control-only Speculation):** 仅在分支点提前触发候选后继实例, 不对数据依赖进行预测与转发; 适用于分支判定等待主导的场景。
- **完全投机 (Full Speculation):** 在控制投机基础上, 结合状态缓冲与依赖转发机制, 进一步隐藏数据依赖与外部 I/O 等待; 适用于数据依赖与外部状

态访问显著的工作流。

为直观说明工作流中的控制不确定性如何在关键路径上引入等待空窗，以及投机执行如何将该空窗转化为可并行推进的计算，本节采用图 3-7 的最小示例进行说明。

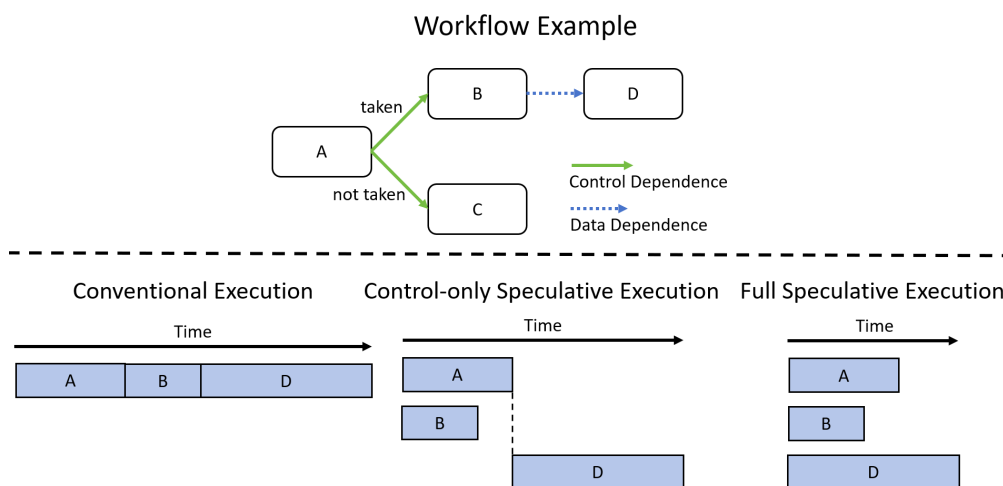


图 3-7 工作流中控制/数据依赖与投机执行对关键路径的影响示例

如图 3-7 所示，常规执行严格遵循“依赖就绪后触发”的保守语义：D 必须等待 A 的分支结果确认并完成 B 的数据产出后才能启动，关键路径上存在不可忽略的等待区间。控制投机通过预测分支走向提前启动 B，能够减少“分支判定—触发”的空窗，但对 D 而言其数据依赖仍构成启动屏障，因此收益受限。进一步地，完全投机在预测控制分支的同时对数据依赖进行提前准备或推测计算，使得 D 可与上游阶段形成更大重叠，从而获得更显著的关键路径缩短；其代价是预测错误时产生的无效计算与潜在资源竞争，这要求系统配合预算与隔离机制以抑制尾延迟反噬。

综上， $\mathcal{M}$ 的运行时行为可归纳为“预测触发 ($\mathcal{P}$)—门控与预算 ($\mathcal{G}$)—缓冲隔离 ($\mathcal{B}$)—验证恢复 ($\mathcal{R}$)”四阶段闭环。该形式化模型为后续章节中分支预测精度提升、门控策略设计、预算自适应与无效投机回收等优化方法提供了统一的描述基础。

3.3 Serverless 工作流投机执行的编程框架

为了方便用户使用 3.2 节定义的 Serverless 工作流投机执行模型，本文提出了一套面向工作流描述与执行控制的编程框架。该框架的目标是为用户提供一

组抽象和对应的 Python API，用于定义 workflow 中的函数节点、依赖关系、结构化控制流以及投机执行所需的关键元信息。编程框架可以视为连接投机执行模型与最终应用之间的桥梁。借助该框架，用户可以便捷地构建和表达端到端 workflow 逻辑，配置分支预测、运行策略与函数属性等信息，并最终实现在 Serverless 平台上的 workflow 投机执行。

本节设计的编程框架由两大部分组成：workflow 过程描述与投机运行配置描述。

workflow 过程描述部分主要关注 workflow 的静态组织形式。该部分提供统一的工作流描述范式，用于声明函数节点、前驱后继关系、分支与循环结构，并支持对不同类型节点施加特定语义约束，从而使系统能够从用户定义中提取可执行的逻辑 workflow 结构。

投机运行配置描述部分则关注模型中的核心运行时机制，包括分支预测、记忆化、纯函数优化、非投机屏障以及运行追踪等内容。本文通过引入多种配置项和接口对象，将这些运行时机制与 workflow 结构建立映射关系。通过这套声明式的配置方式，用户能够灵活定义满足不同场景需求的投机执行策略，并为后续的执行、验证、恢复和性能优化提供基础。

3.3.1 workflow 过程描述

为了支持用户按照 3.2 节定义的投机执行模型构建 workflow，本文在编程框架中提供了统一的工作流过程描述方式。该描述方式面向 workflow 的结构表达，允许用户显式定义函数节点、节点间依赖关系以及必要的语义属性，从而为后续的投机调度、预测与验证机制提供输入。

在本文框架中，workflow 过程描述主要包括以下几个核心要素：

- **workflow 对象**。workflow 对象用于描述一个完整的工作流实例，是过程描述的顶层抽象。它记录 workflow 名称、入口节点以及整体拓扑关系，并组织 workflow 中的全部函数节点。
- **函数节点**。函数节点是 workflow 中的基本执行单元，每个节点对应一个具体的函数实现。除函数名称与绑定关系外，节点描述还需要体现其在工作流中的结构角色。

- **转移关系。**转移关系用于描述节点之间的前驱后继关系。对于顺序结构，它表现为线性连接；对于分支与循环结构，则需要显式给出候选后继或不同转移方向。
- **节点语义属性。**除结构关系外，过程描述还允许为节点附加必要的语义属性，例如纯函数、非投机节点等，以支持后续的记忆化优化与投机控制。

图 3-8 给出了工作流过程描述中几个核心元素之间的关系。其中，Workflow 作为顶层对象描述一个完整工作流，Function 表示工作流中的函数节点，Transition 用于描述节点间的转移关系，Annotation 则用于刻画节点的附加语义属性。工作流对象作为顶层容器组织多个函数节点，节点之间通过转移关系连接形成完整的执行图，而节点语义属性则作为附加信息作用于具体节点。

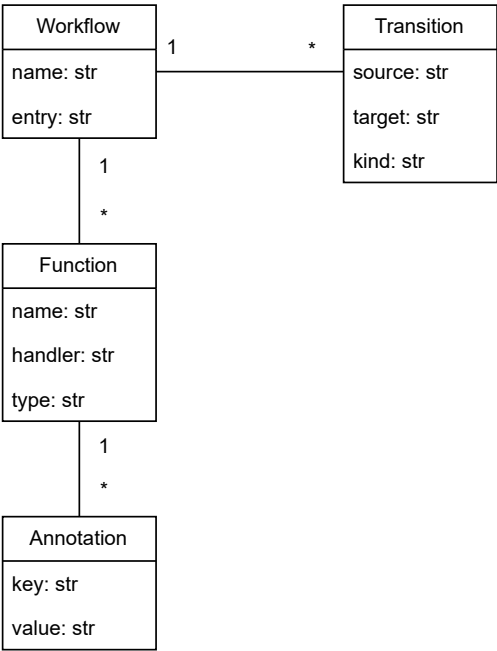


图 3-8 工作流过程描述中的核心抽象

在具体实现上，本文采用声明式方式组织工作流描述。用户需要给出工作流包含的节点集合、节点之间的关系以及必要的节点属性，框架再据此生成系统内部可识别的工作流表示，并用于后续部署与执行。这样一来，用户关注的重点始终是“工作流如何组织”，而不是“投机执行机制如何手工嵌入到每个函数中”。

下面以一个简单的条件工作流为例进行说明。假设工作流首先执行预处理节点 f_1 ，随后由判定节点 f_2 根据输入条件在 f_3 和 f_4 之间选择其一，最后统一

进入节点 f_5 。

在描述上，用户首先在 workflow.json 中使用 entry: "f1" 指定入口节点；随后在 functions 字段中列出五个节点 f_1, f_2, f_3, f_4, f_5 ，并为每个节点给出 name、handler 和 type；再在 transitions 字段中定义边集合，其中包括 $f_1 \rightarrow f_2$ 、 $f_2 \rightarrow f_3$ 、 $f_2 \rightarrow f_4$ 、 $f_3 \rightarrow f_5$ 和 $f_4 \rightarrow f_5$ 。其中，节点 f_2 作为分支节点，其候选后继集合 $\{f_3, f_4\}$ 由两条同源转移边显式表示。若某一节点具有纯函数属性或不参与投机执行，还可在对应节点定义中通过 pure 和 non_spec 字段进一步标注。用户需要编写和配置的工作流描述文件及对应函数文件结构如图 3-9 所示。

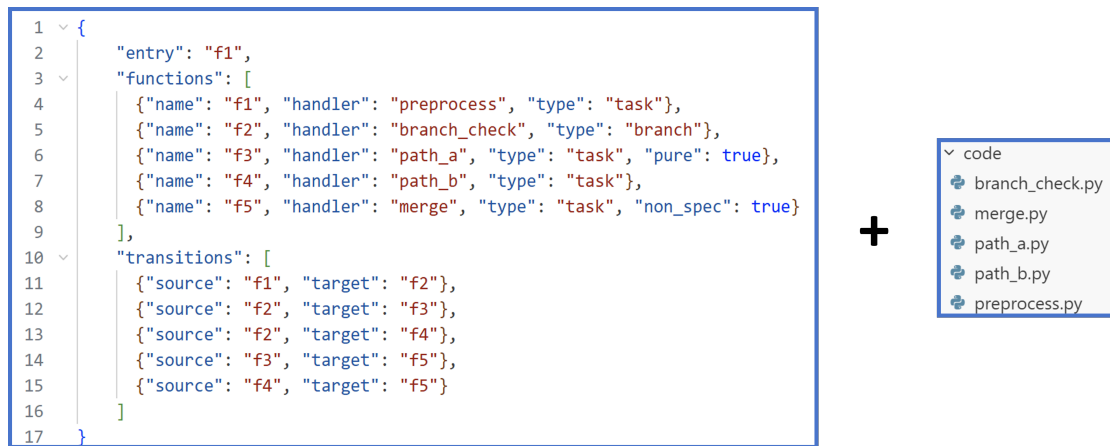


图 3-9 工作流描述文件与函数文件组织示例

在上述描述基础上，系统即可获得一个结构明确的工作流表示：一方面，它能够识别普通顺序依赖与分支依赖；另一方面，它也能够根据节点语义属性决定某些节点是否允许参与记忆化优化或投机执行。这样，3.2 节定义的投机执行模型便获得了面向用户可编写、面向系统可处理的统一描述基础。

综上，工作流过程描述的核心作用在于为工作流提供一套统一的静态表达方式。它将函数节点、结构关系和必要语义组织在同一描述框架下，使工作流能够在保持业务逻辑清晰的同时，为后续的投机运行时配置与执行控制提供输入。

3.3.2 投机运行时配置接口

为了将上一小节给出的工作流静态描述映射为可执行的投机运行过程，本文在编程框架中设计了一组面向运行时控制的配置接口。与工作流过程描述主要关注“如何定义一个工作流”不同，投机运行时配置接口主要关注“如何以特

定策略执行该工作流”。该部分将分支预测、门控阈值、并发预算、记忆化优化以及执行追踪等机制组织为统一的配置抽象，并通过统一的运行入口提交给系统执行。

具体而言，本文在该部分引入了 `SpecRuntimeConfig`、`WorkflowRunner` 和 `WorkflowRun` 三个核心语义对象，分别用于描述运行时配置、工作流运行入口以及运行结果。

`SpecRuntimeConfig` 用于描述一次工作流运行所采用的投机执行配置。每个配置对象对应一组运行时参数，包括分支预测方式、候选集合门控阈值、投机并发预算、运行模式、记忆化开关以及执行追踪选项等。它对应于第 3.2 节投机执行模型中的预测器、门控与预算策略以及部分验证恢复相关配置。

对于同一个工作流，用户可以在一种配置下仅启用控制投机，而在另一种配置下同时启用控制投机、记忆化与执行追踪；也可以通过调整阈值与预算，在投机收益、资源开销与执行风险之间进行权衡。表 3-1 给出了 `SpecRuntimeConfig` 提供的主要方法。

表 3-1 `SpecRuntimeConfig` 公开方法

方法签名	说明
<code>SpecRuntimeConfig()</code>	创建投机运行时配置对象
<code>predictor(name, **kwargs)</code>	指定分支预测器类型及相关参数
<code>threshold(τ)</code>	设置候选集合构造所使用的门控阈值
<code>budget(B)</code>	设置投机并发预算上限
<code>mode(kind)</code>	指定运行模式，如控制投机或完全投机
<code>memo(enable)</code>	设置是否启用记忆化优化
<code>trace(enable)</code>	设置是否记录执行追踪与时间线信息

其中，`predictor` 方法用于声明运行时采用的预测器类型及相关参数。例如，用户可以选择基于路径历史、输入特征或历史统计信息的分支预测器。`threshold` 和 `budget` 分别用于控制候选集合构造与投机规模；`mode` 用于在控制投机和完全投机之间切换；`memo`、`trace` 则分别对应记忆化优化与执行追踪功能。

`WorkflowRunner` 定义了工作流运行器接口，用于将静态工作流描述对象与运行时配置对象组合起来，并提交到具体执行环境中运行。它相当于一个统一的执行入口，负责接收工作流定义与投机运行配置，并将其转换为系统内部可执行的运行实例。

根据运行环境的不同，WorkflowRunner 可以具有不同实现形式。其中，本地运行器 LocalWorkflowRunner 主要用于实验、调试与原型测试；远端运行器 RemoteWorkflowRunner 则负责将 workflow 部署并提交到目标 Serverless 平台，由平台侧控制器执行投机调度、状态隔离与验证恢复过程。图 3-10 展示了 WorkflowRunner 的接口对象及其典型实现类型。

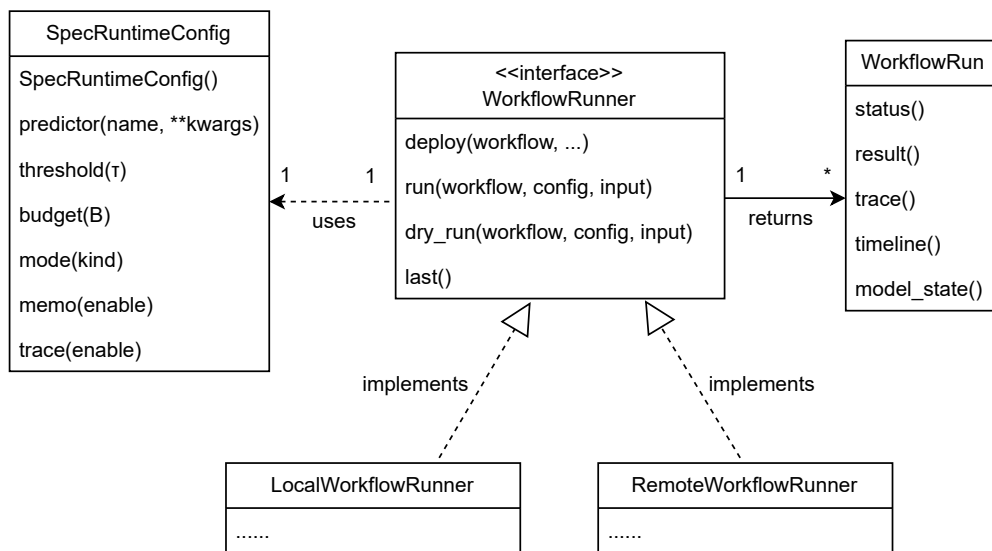


图 3-10 WorkflowRunner 接口对象与实现类型

表 3-2 列出了 WorkflowRunner 提供的主要方法。

表 3-2 WorkflowRunner 公开方法

方法签名	说明
<code>deploy(workflow, ...)</code>	将 workflow 描述部署到目标执行环境
<code>run(workflow, config, input)</code>	以指定配置和输入启动一次 workflow 运行
<code>dry_run(workflow, config, input)</code>	在不真正提交副作用的条件下进行模拟执行
<code>last()</code>	返回最近一次运行对应的句柄对象

`deploy` 方法用于将 workflow 描述及其函数实现注册到目标环境中；`run` 方法用于结合给定输入和 `SpecRuntimeConfig` 启动一次实际运行；`dry_run` 方法用于在不真正提交外部副作用的条件下模拟执行流程，便于用户在开发与调试阶段检查 workflow 拓扑、候选集合构造和门控参数是否合理；`last` 方法则返回最近一次运行对应的 `WorkflowRun` 对象，便于在多次实验中快速定位和比较运行结果。

WorkflowRun 表示一次已经创建的工作流运行实例,它是 WorkflowRunner 调用 run 后返回的结果对象。该对象封装了一次具体运行的状态、输出和追踪信息,并提供进一步的查询与分析接口。用户无需直接接触控制器内部状态机,只需通过该对象暴露的方法即可获取执行结果、查看运行状态或导出事件时间线。表 3-3 给出了 WorkflowRun 提供的主要接口。

表 3-3 WorkflowRun 公开方法

方法签名	说明
status()	获取当前运行状态
result()	获取工作流最终输出结果
trace()	获取本次运行的关键事件追踪信息
timeline()	获取时间线或可视化分析所需的事件序列
model_state()	获取本次运行关联的预测器或控制器状态摘要

其中, status 和 result 主要服务于常规流程; trace 和 timeline 用于记录分支预测结果、候选集合构造、投机实例触发、验证提交和误投机撤销等关键事件; model_state 则用于返回与本次运行相关的模型或控制器状态摘要。

除通过上述编程接口配置和运行工作流外,本文原型系统还提供了统一的命令行工具 specx,作为面向部署、运行、调试与观测场景的补充入口。用户可以通过 specx 完成工作流部署、运行发起、最近一次运行结果查询、模型状态查看以及执行时间线导出等操作。

当命令行工具 specx 与运行时配置接口同时提供参数时,系统的配置优先级为: CLI 显式参数 > 工作流级运行时配置 > 系统默认值。

需要指出的是,并非所有配置项都适合由命令行动态覆盖。预测器类型、门控阈值、并发预算、运行模式以及追踪开关等参数属于运行策略层面的可调项,适合通过 specx 在单次运行时临时修改;而工作流拓扑、节点纯函数声明、非投机屏障以及外部状态访问约束等内容属于工作流定义本身的语义约束,不应由命令行在运行时随意改变。

为了更直观地展示投机运行时配置接口的使用方式,下面给出一个简单的条件工作流运行示例。假设用户已经按照上一小节定义了一个工作流对象 wf,其中包含分支节点 f2 及其两个候选后继 f3 和 f4。用户可以首先创建一份运行时配置对象,并显式设置预测器、门控阈值、预算和追踪选项:


```

1 config = SpecRuntimeConfig() \
2     .predictor("path", history=4) \
3     .threshold(0.4) \
4     .budget(2) \
5     .mode("control") \
6     .memo(True) \
7     .trace(True)

```

上述配置表示：系统采用基于路径历史的分支预测方式，使用长度为 4 的最近路径作为上下文；门控阈值设置为 0.4，最多允许 2 个投机实例并发运行；运行模式为控制投机，并开启记忆化优化与执行追踪。

在此基础上，用户可以实例化运行器并启动工作流运行：

```

1 runner = RemoteWorkflowRunner(endpoint="http://controller
2     :8080")
3
4 runner.deploy(wf)
5
6 run = runner.run(
7     workflow=wf,
8     config=config,
9     input={"user_id": 42, "temp": 31}
10 )

```

运行开始后，系统在分支节点判定尚未揭示时，会依据 config 中的预测参数构造候选集合并触发投机路径；而当判定真正返回后，控制器再执行验证提交或误投机撤销。用户可通过返回的 WorkflowRun 对象进一步获取结果与追踪信息：

```

1 status = run.status()
2 result = run.result()
3 events = run.timeline()

```

如果用户希望在不修改原始工作流代码和配置对象的前提下，对单次运行进行临时调整，还可以通过 specx 提供等价的命令行入口。例如，用户可以先部署工作流，再通过命令行发起一次带有门控阈值和预算覆写的运行：

```
1 specx deploy workflow_x
2 specx run workflow_x --predictor path --history 4 \
3   --threshold 0.4 --budget 2 --mode control --trace
```

在该示例中，若 workflow 代码中的 `SpecRuntimeConfig` 已给出部分默认配置，则 `specx run` 中显式出现的参数将覆盖对应字段，而其余字段仍沿用原有配置。这样，编程接口与命令行接口便能够在同一套语义之下协同工作，既支持面向应用开发的嵌入式调用，也支持面向实验与系统评估的外部控制。

综上所述，投机运行时配置接口通过三类语义对象 `SpecRuntimeConfig`，`WorkflowRunner` 和 `WorkflowRun`，将工作流的静态描述进一步映射为可控制、可执行、可观测的运行过程。其中，`SpecRuntimeConfig` 负责描述“以何种策略运行”，`WorkflowRunner` 负责描述“在何种环境中启动运行”，而 `WorkflowRun` 则负责表示“运行之后如何查询与分析”。在此基础上，命令行工具 `specx` 进一步提供了面向实验、部署与调试场景的统一外部入口。三者共同构成了本文编程框架中面向投机执行控制的接口基础。

3.4 投机执行的关键优化方法

在前文给出的 Serverless workflow 投机执行模型与编程框架基础上，本文进一步对投机执行中的关键环节进行优化。本文将优化内容主要聚焦于分支预测，并围绕复杂 workflow 场景下预测准确性与适应性不足的问题，对基础预测机制进行改进。整体来看，本文首先分别从输入与分支走向的相关性以及时间变化两个维度提出针对性的预测增强方法，随后进一步讨论二者在同一预测器中的融合策略。

输入感知的分支预测。在路径相关建模的基础上进一步引入输入信息，使预测器能够区分同一路径下由输入差异引起的不同分支行为，从而提高复杂输入场景下的预测精度。

分布漂移适应。针对工作负载和输入模式随时间变化导致历史统计逐渐失效的问题，通过强调近期观测、抑制陈旧统计的影响，提高预测器对分布变化的响应能力。

在此基础上，本文进一步讨论如何将上述两类机制进行分层集成，使预测器

在保持统计稳定性的同时，兼顾输入细化带来的额外判别能力与对时间变化的响应能力。

3.4.1 输入感知的分支预测

在基础投机执行流程中，分支预测器需要在分支判定尚未揭示时，根据当前上下文估计各后继路径的概率分布，并为后续投机触发提供依据。3.2 节已经将分支点上下文表示为 $c_t = \langle x_t, h_t \rangle$ ，其中 x_t 表示当前可观测输入特征， h_t 表示到达当前分支点之前的路径历史。该建模方式为分支预测提供了统一的形式化基础。

然而，在实际工作流中，仅依赖路径相关信息仍然可能无法准确刻画真实分支行为。其原因在于：对于某些分支点，即使到达该分支的执行路径相同，不同输入仍可能触发明显不同的后继结果。例如，在订单处理、内容审核或推荐服务等工作流中，请求类型、用户属性、地区标识、对象类别等输入字段可能对分支选择产生直接影响。此时，若预测器仅基于路径签名进行统计，则会将“同一路径、不同输入”的多种行为模式混合在一起，导致预测结果过于粗粒度，进而降低投机命中率。

针对上述问题，本文在路径相关建模的基础上，进一步提出输入感知的分支预测方法。其核心思想是：以路径签名作为基础上下文，在此基础上利用输入特征对路径状态进行进一步细化；同时，为避免输入细分带来的状态爆炸与冷启动不稳定问题，引入回退混合与自适应启用机制，在路径级统计与输入细化统计之间进行平滑切换。整体方法示意如图 3-11 所示。

系统首先根据已提交节点序列构造当前分支点的路径签名，并结合当前输入提取与分支决策相关的关键字段；随后，通过输入桶映射将高维输入压缩到有限个离散桶中；在此基础上，预测器分别维护路径级统计与输入桶级统计，并通过样本数驱动的回退混合得到最终分支概率分布；最后，再根据输入细化所带来的额外信息增益决定是否启用输入感知预测。通过这一过程，系统能够在保持预测状态规模可控的前提下，更准确地区分同一路径下由输入差异引起的多模态分支行为。

(1) 路径签名作为基础上下文。本文沿用 SpecFaaS 论文的思想，将路径签名作为输入感知预测的基础上下文^[24]。具体而言，设当前分支节点为 b ，系统维

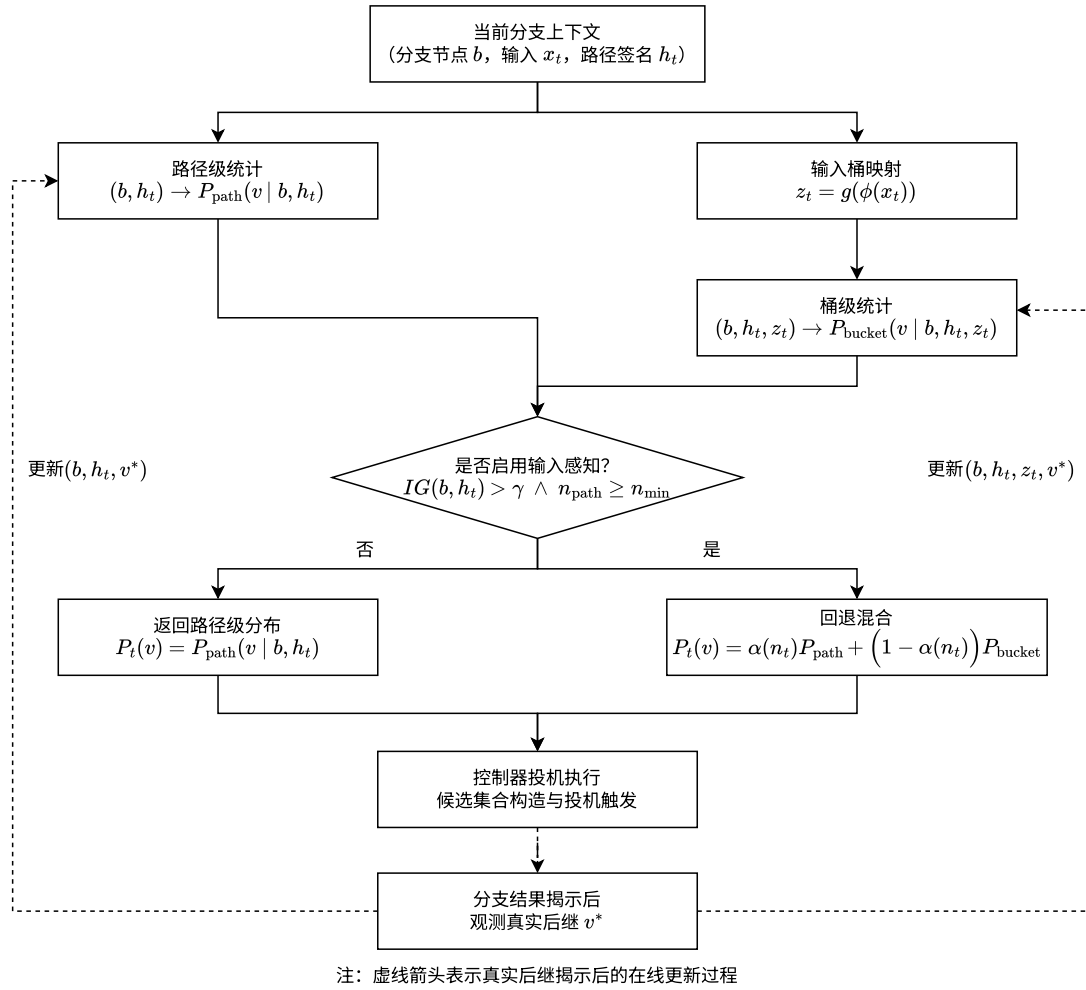


图 3-11 输入感知的分支预测方法流程图

护最近 K 个已提交节点构成的路径签名

$$h_t = \langle u_{t-K+1}, u_{t-K+2}, \dots, u_t \rangle. \quad (3-9)$$

在最基础的路径相关预测中，可根据历史观测得到路径级后继分布

$$P_{\text{path}}(v_i | b, h_t) = \frac{N(b, h_t, v_i)}{\sum_{v_j \in \text{Succ}(b)} N(b, h_t, v_j)}, \quad (3-10)$$

其中 $N(b, h_t, v_i)$ 表示在分支点 b 且路径签名为 h_t 的历史运行中，真实后继为 v_i 的观测次数。该分布描述了“在给定路径上下文下，分支更可能走向何处”，是后续输入细化与概率混合的基础。

(2) 输入桶细化。在路径级统计的基础上，本文进一步考虑输入与分支走向

之间的相关性对预测结果的影响。直接将完整输入作为统计键虽然能够最大化保留输入信息，但会导致状态数量快速膨胀，不利于在线学习与状态维护。为此，本文采用“特征抽取 + 输入桶映射”的方式，将输入细化为有限个离散状态。设当前输入为 x_t ，系统首先通过特征抽取函数 $\phi(\cdot)$ 选择与分支决策相关的关键字段，再通过桶映射函数 $g(\cdot)$ 将其压缩为离散桶标识：

$$z_t = g(\phi(x_t)). \quad (3-11)$$

其中， $\phi(\cdot)$ 的作用是提取与当前分支决策最相关的输入信息， $g(\cdot)$ 则用于控制状态空间规模。在本文原型系统中， $\phi(\cdot)$ 采用轻量的字段选择与规范化方式实现， $g(\cdot)$ 则采用有界分桶映射实现，例如分箱或哈希取模等方式；更具体的实现细节将在 4.3.3 节中进一步说明。与直接使用完整输入相比，输入桶细化的目标不是保留全部输入细节，而是在可控状态规模下尽可能保留对分支决策最有影响的输入差异。

于是，在路径签名与输入桶共同作用下，可得到更细粒度的输入感知后继分布：

$$P_{\text{bucket}}(v_i | b, h_t, z_t) = \frac{N(b, h_t, z_t, v_i)}{\sum_{v_j \in \text{Succ}(b)} N(b, h_t, z_t, v_j)}, \quad (3-12)$$

其中 $N(b, h_t, z_t, v_i)$ 表示在分支点 b 、路径签名为 h_t 且输入桶为 z_t 的历史运行中，真实后继为 v_i 的观测次数。

通过上述细化，预测粒度从“分支节点 + 路径签名”进一步扩展为“分支节点 + 路径签名 + 输入桶”。这样一来，对于同一路径下由输入差异引起的多种分支行为，预测器能够分别学习其后继分布。例如，在相同调用链到达某个风控分支点时，高风险用户与低风险用户可能分别对应不同后继路径；若不引入输入桶，这两种模式会被混合统计，而输入细化后则可显式区分。

(3) 回退混合。 输入桶细化虽然能够提高分支行为的区分能力，但也带来了一个直接问题：输入桶级统计更细，样本数往往更少，在冷启动阶段容易出现估计不稳定，甚至长期停留在接近均匀分布的状态。若系统在样本很少时便完全依赖输入桶级统计，则可能出现预测波动大、误投机率高的问题。为此，本文引入路径级统计与输入桶级统计之间的回退混合机制，使预测器能够在“稳定性”

和“区分能力”之间取得平衡。

设当前桶状态 (b, h_t, z_t) 的样本总数为

$$n_t = \sum_{v_i \in \text{Succ}(b)} N(b, h_t, z_t, v_i), \quad (3-13)$$

则最终用于预测的后继概率分布定义为

$$P_t(v_i) = \alpha(n_t) P_{\text{path}}(v_i | b, h_t) + (1 - \alpha(n_t)) P_{\text{bucket}}(v_i | b, h_t, z_t), \quad (3-14)$$

其中 $\alpha(n_t) \in [0, 1]$ 为样本数驱动的回退系数。本文采用如下简单且可解释的形式：

$$\alpha(n_t) = \frac{k}{n_t + k}, \quad (3-15)$$

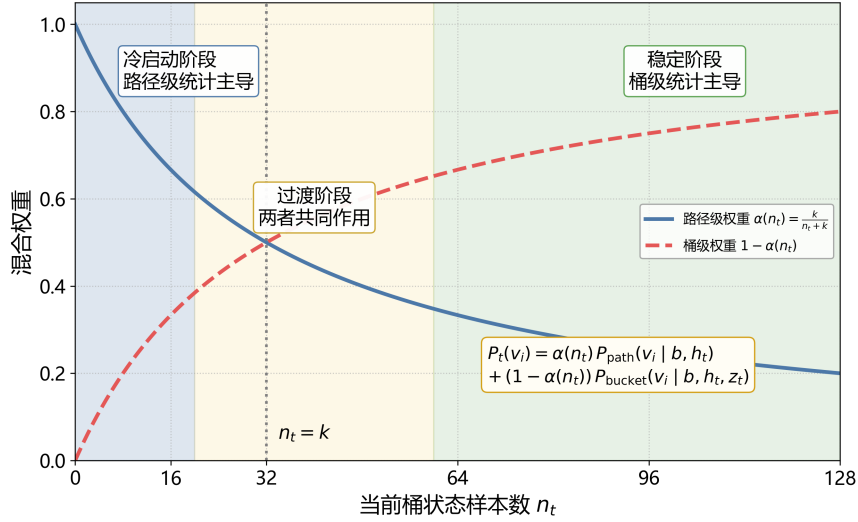
其中 k 为平滑超参数，用于控制输入桶级统计从冷启动过渡到稳定主导的速度。

上述混合机制具有较好的可解释性。当 $n_t = 0$ 时， $\alpha(n_t) = 1$ ，系统完全退化为路径级预测，即在无可靠输入桶统计时优先依赖较稳定的路径级分布；当 n_t 增大时， $\alpha(n_t)$ 逐渐减小，说明输入桶级统计对最终结果的影响逐步增强；当 $n_t \gg k$ 时， $\alpha(n_t) \rightarrow 0$ ，系统主要依赖输入桶级分布，此时输入感知预测的区分能力可以充分发挥。该设计本质上是一种层次化平滑策略：路径级统计提供稳健先验，输入桶级统计提供细粒度修正，从而避免将输入细化直接建立在脆弱的小样本估计之上。

图 3-12 给出了 $k = 32$ 时，回退混合的作用。在冷启动阶段，由于输入桶样本数不足，预测结果主要由路径级分布决定；随着在线观测不断积累，输入桶级统计逐步接管最终分布，预测器由保守稳定逐步过渡到细粒度区分。

(4) 自适应启用。并非所有路径状态都值得引入输入细化。对于某些分支点，在给定路径签名后，其后继分布已经较为稳定，此时进一步按输入桶细分只会增加状态规模，却难以带来明显收益。为此，本文进一步引入输入感知预测的自适应启用机制，仅在输入信息确实能够显著降低分支不确定性时启用输入细化。

具体而言，本文使用信息增益来衡量输入桶相对于路径级统计所提供的额

图 3-12 路径级统计与输入桶级统计的回退混合示意图 ($k = 32$)

外判别能力。记任意离散后继分布 $P(\cdot)$ 的香农熵为

$$H(P) = - \sum_y P(y) \log P(y). \quad (3-16)$$

如图 3-13 所示，随着主分支概率逐步增大，分支结果由接近均匀分布逐渐转向更确定的偏斜分布，其对应的香农熵持续下降，表明分支不确定性不断减弱。

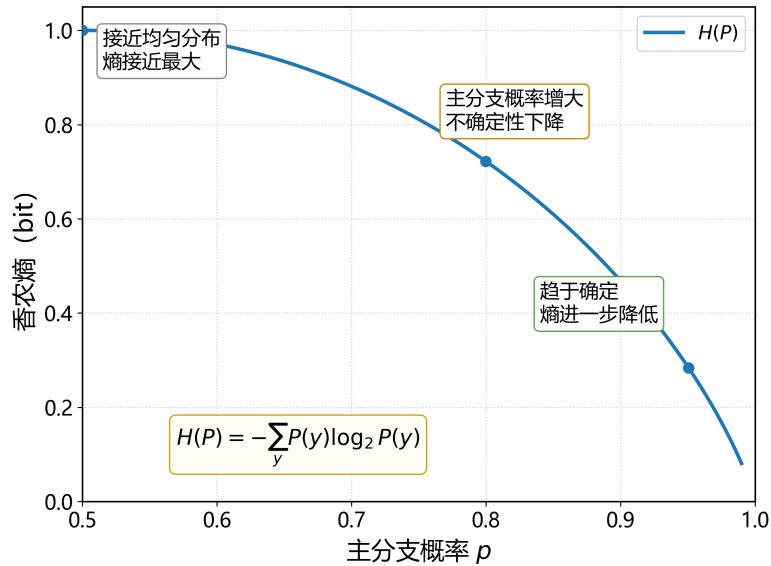


图 3-13 香农熵随分支确定性变化的示意图（二分支）

在此基础上，对于分支节点 b 和路径签名 h_t ，路径级分布的不确定性可表

示为

$$H_{\text{path}}(b, h_t) = H(P_{\text{path}}(\cdot | b, h_t)) = - \sum_{v_i \in \text{Succ}(b)} P_{\text{path}}(v_i | b, h_t) \log P_{\text{path}}(v_i | b, h_t). \quad (3-17)$$

进一步地，在输入桶细化后，给定路径状态 (b, h_t) 的剩余不确定性可表示为各输入桶条件熵的加权平均：

$$H_{\text{bucket}}(b, h_t) = \sum_{z \in \mathcal{Z}_{b, h_t}} p(z | b, h_t) H(P_{\text{bucket}}(\cdot | b, h_t, z)), \quad (3-18)$$

其中， $\mathcal{Z}_{b, h_t}$ 表示在路径状态 (b, h_t) 下出现过的输入桶集合， $p(z | b, h_t)$ 表示在给定分支节点 b 和路径签名 h_t 的条件下，输入落入桶 z 的经验概率。由此，输入桶细化带来的信息增益定义为

$$IG(b, h_t) = H_{\text{path}}(b, h_t) - H_{\text{bucket}}(b, h_t). \quad (3-19)$$

其中， $H_{\text{path}}(b, h_t)$ 反映了仅依赖路径签名时结果的不确定性， $H_{\text{bucket}}(b, h_t)$ 则反映了进一步引入输入桶细化后剩余不确定性的加权平均。若 $IG(b, h_t)$ 较大，则说明在给定路径签名后，引入输入桶能够显著降低分支不确定性，此时启用输入感知预测更有意义；反之，若 $IG(b, h_t)$ 很小，则说明路径级统计已经足以刻画该状态下的分支行为，继续细分输入并不能带来明显收益。

基于此，本文采用如下启用条件：

$$\text{EnableInputAware}(b, h_t) = \mathbf{1}[IG(b, h_t) > \gamma \wedge n_{\text{path}}(b, h_t) \geq n_{\min}], \quad (3-20)$$

其中， γ 为信息增益阈值， $n_{\min}$ 为路径级最小样本数门限， $n_{\text{path}}(b, h_t)$ 表示路径状态 (b, h_t) 的累计观测次数。该策略使系统从“总是启用输入细化”转变为“仅在输入确实提供额外区分能力时启用输入细化”，从而提高了方法的解释性与可控性。

图 3-14 进一步直观展示了输入感知预测的自适应启用区域划分。横轴表示路径状态 (b, h_t) 的累计样本数 $n_{\text{path}}(b, h_t)$ ，纵轴表示输入细化带来的信息增益 $IG(b, h_t)$ 。当样本数与信息增益同时超过阈值 $n_{\min}$ 和 γ 时，系统启用输入感知预

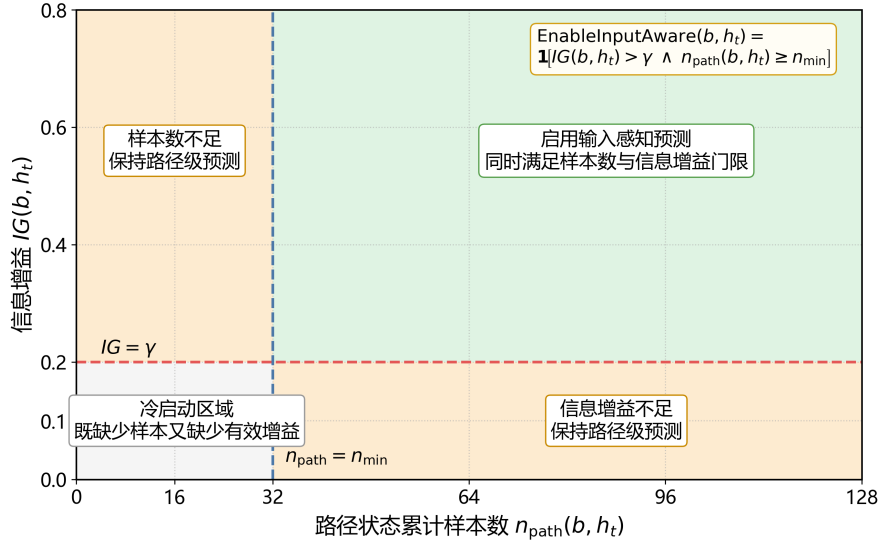


图 3-14 输入感知预测的自适应启用条件示意图

测；否则，仍保持路径级预测。该图也反映了两类典型的保守情形：其一是样本不足导致的冷启动区域，此时即使观察到局部差异，也不宜过早依赖输入细化；其二是信息增益不足区域，说明输入特征未能显著降低分支不确定性，此时继续采用路径级预测更加稳健。

综合上述设计，输入感知的分支预测可概括为算法 3.1。算法输入为当前分支点、路径签名和当前输入，首先估计路径级分布；若当前路径状态满足输入感知启用条件，则进一步构造输入桶并估计桶级分布；随后通过回退混合得到最终后继概率分布，并返回给控制器作为后续候选集合构造与投机触发的依据。

算法 3.1: 输入感知的分支预测流程

输入: 分支节点 b ，其对应的路径签名 h_t ，当前可观测输入 x_t
输出: 最终后继概率分布 $P_t(\cdot)$ ，其中 $P_t(v_i)$ 表示候选后继 $v_i \in \text{Succ}(b)$ 的预测概率

```

1  $P_{\text{path}} \leftarrow \text{PATHPRED}(b, h_t)$  // 根据  $(b, h_t)$  查询路径级统计
2  $n_{\text{path}} \leftarrow \text{PATHCNT}(b, h_t)$  // 计算当前路径状态的累计样本数
3  $IG \leftarrow \text{INFOGAIN}(b, h_t)$  // 估计输入细化带来的信息增益
4 if  $IG \leq \gamma \vee n_{\text{path}} < n_{\min}$  then
5   return  $P_{\text{path}}$  // 若输入细化收益不足，则直接返回路径级分布
6  $f_t \leftarrow \phi(x_t)$  // 提取与分支决策相关的输入特征
7  $z_t \leftarrow g(f_t)$  // 通过桶映射函数计算输入桶标识
8  $P_{\text{bucket}} \leftarrow \text{BUCKETPRED}(b, h_t, z_t)$  // 根据  $(b, h_t, z_t)$  查询桶级统计
9  $n_t \leftarrow \text{BUCKETCNT}(b, h_t, z_t)$  // 计算当前桶状态的样本数
10  $\alpha \leftarrow \frac{k}{n_t + k}$  // 根据样本数计算回退系数
11 foreach  $v_i \in \text{Succ}(b)$  do
12    $P_t(v_i) \leftarrow \alpha P_{\text{path}}(v_i) + (1 - \alpha) P_{\text{bucket}}(v_i)$  // 执行路径级与桶级概率混合
13 return  $P_t(\cdot)$ 

```

综上所述，输入感知的分支预测并非简单地将输入信息附加到已有上下文之上，而是在路径相关建模的基础上，进一步提出了“输入桶细化—回退混合—自适应启用”的完整方法设计。其中，路径签名提供基本的控制流上下文，输入桶细化用于捕捉同一路径下由不同输入模式所引起的分支分化行为，回退混合用于在统计稳定性与细粒度区分能力之间取得平衡，自适应启用则进一步抑制不必要的状态细分。通过这些设计，系统能够在保持状态规模可控的前提下，提高复杂输入场景下的分支预测准确性。

3.4.2 分布漂移适应

在基础路径相关预测中，分支概率通常由历史观测的累计统计得到。这种方式在工作负载相对稳定时能够提供较为平滑的概率估计，但在实际 Serverless workflow 中，分支行为往往并非静态不变，而是会随着输入模式、业务阶段与运行环境的变化而发生漂移。例如，在版本迭代、促销活动、地区切换或上游请求结构变化等场景下，同一分支点在相同路径上下文下的后继概率分布可能在不同时间阶段呈现明显差异。此时，若预测器始终依据长期累计统计进行判断，则旧样本会持续影响当前决策，使得模型在真实分布已经变化后仍然偏向陈旧方向，进而增加误预测与误投机开销。

针对上述问题，本文提出分布漂移适应的分支预测方法。其核心思想是：在保持“分支节点 + 路径签名”这一基础上下文建模方式不变的前提下，不再将全部历史观测等权对待，而是同时维护两种概率视图：一种是基于指数衰减的平滑近期统计，用于在稳定性与适应性之间取得平衡；另一种是基于滑动窗口的短期统计，用于在分布发生明显切换时快速跟踪新模式。在此基础上，系统进一步通过漂移强度检测机制，在两类预测模式之间进行硬自适应切换，从而在稳定阶段保持预测平滑性，在强漂移阶段提高响应速度。

(1) 长记忆统计的局限性。 设当前分支节点为 b ，最近 K 个已提交节点构成的路径签名为 h_t ，则当前分支上下文记为

$$c_t = (b, h_t). \quad (3-21)$$

对于任意候选后继 $v_i \in Succ(b)$ ，传统长记忆预测器通常基于累计计数

$$N_t(c_t, v_i) \quad (3-22)$$

来估计其条件概率：

$$P_{\text{long}}(v_i | c_t) = \frac{N_t(c_t, v_i)}{\sum_{v_j \in Succ(b)} N_t(c_t, v_j)}. \quad (3-23)$$

其中， $N_t(c_t, v_i)$ 表示截至时刻 t ，在上下文 c_t 下观测到真实后继为 v_i 的总次数。该估计方式隐含地假设各时刻样本对当前决策具有相同参考价值。然而，当真实分支分布随时间变化时，这一假设通常不再成立。若早期样本主要对应旧分布，而近期样本已反映新分布，则长期累计统计会在相当长一段时间内维持对旧主分支的偏好，从而削弱预测器对当前行为模式的刻画能力。

(2) 指数衰减统计。为提高预测器对近期变化的敏感性，本文首先引入指数衰减统计机制。其基本思想是：每当新的分支观测到来时，对已有统计整体施加衰减，再将当前真实结果写入计数，从而使较新样本拥有更高权重。设 $\tilde{N}_t(c, v_i)$ 表示上下文 c 下后继 v_i 的衰减计数，则其递推更新方式为

$$\tilde{N}_t(c, v_i) = \gamma \tilde{N}_{t-1}(c, v_i) + \mathbf{1}[y_t = v_i], \quad (3-24)$$

其中， y_t 表示时刻 t 的真实分支结果， $\mathbf{1}[\cdot]$ 为示性函数， $\gamma \in (0, 1]$ 为衰减系数。

基于衰减计数，可得到指数衰减分布

$$P_{\text{decay}}(v_i | c) = \frac{\tilde{N}_t(c, v_i) + \lambda}{\sum_{v_j \in Succ(b)} (\tilde{N}_t(c, v_j) + \lambda)}, \quad (3-25)$$

其中 λ 为 Laplace 平滑参数，用于避免小样本阶段出现零概率。

当 $\gamma = 1$ 时，系统退化为无衰减的长记忆统计；当 $\gamma < 1$ 时，越早的观测对当前分布的影响越弱。指数衰减本质上是一种“软遗忘”机制：它并不完全丢弃历史样本，而是逐步降低旧样本的贡献，因此在分布缓慢演化时能够在统计稳定

性与适应性之间取得较好平衡。

(3) 滑动窗口统计。虽然指数衰减能够减弱陈旧样本影响，但它仍然保留了全部历史信息。当分支分布出现较明显的阶段性切换时，仅依赖衰减机制仍可能保留过多旧信息，从而限制对新分布的响应速度。为此，本文进一步引入滑动窗口统计，仅基于最近一段观测估计当前分支分布。

设窗口大小为 W ，则上下文 c 下最近窗口内后继 v_i 的计数定义为

$$N_t^{(W)}(c, v_i) = \sum_{\tau=t-W+1}^t \mathbf{1}[c_\tau = c \wedge y_\tau = v_i]. \quad (3-26)$$

由此得到滑动窗口分布

$$P_{\text{win}}(v_i | c) = \frac{N_t^{(W)}(c, v_i) + \lambda}{\sum_{v_j \in \text{Succ}(b)} (N_t^{(W)}(c, v_j) + \lambda)}. \quad (3-27)$$

与指数衰减相比，滑动窗口是一种“硬截断”机制：窗口之外的样本不再参与当前预测。因此，它能够更快捕捉近期分布的显著变化，尤其适用于存在明显阶段切换的场景。但与此同时，窗口统计仅依赖有限样本，其波动性通常高于指数衰减统计，若窗口过小或样本不足，则容易出现估计不稳定的问题。

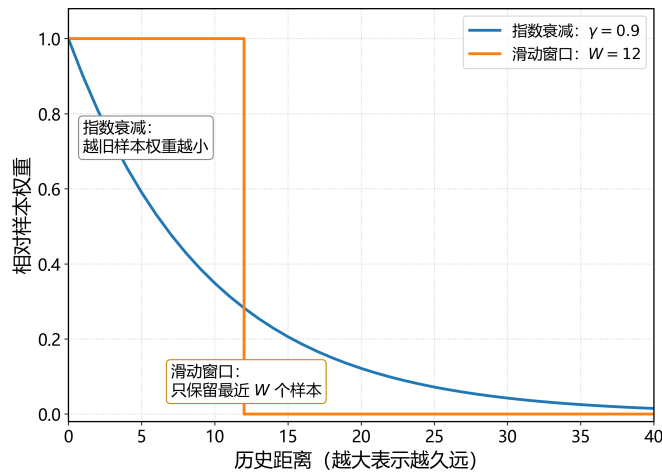


图 3-15 指数衰减与滑动窗口对比示意图

(4) 漂移强度检测。指数衰减统计与滑动窗口统计分别对应“平滑适应”和“快速响应”两种建模偏好，图 3-15 给出了二者的差异。为了根据当前上下文下的真实漂移程度自适应选择合适的机制，本文进一步引入漂移强度检测方法。其

核心思想是：若两类统计视图给出的分支分布较为接近，则说明近期行为与平滑历史之间差异较小，当前不存在明显漂移；反之，若二者差异较大，则说明近期行为已经偏离平滑历史，应更重视短期统计。

本文使用 Jensen–Shannon 散度衡量两类分布之间的差异。记

$$M_t(\cdot | c) = \frac{1}{2} \left(P_{\text{decay}}(\cdot | c) + P_{\text{win}}(\cdot | c) \right), \quad (3-28)$$

则当前上下文下的漂移强度定义为

$$D_t(c) = \frac{1}{2} D_{\text{KL}}(P_{\text{decay}}(\cdot | c) \| M_t(\cdot | c)) + \frac{1}{2} D_{\text{KL}}(P_{\text{win}}(\cdot | c) \| M_t(\cdot | c)), \quad (3-29)$$

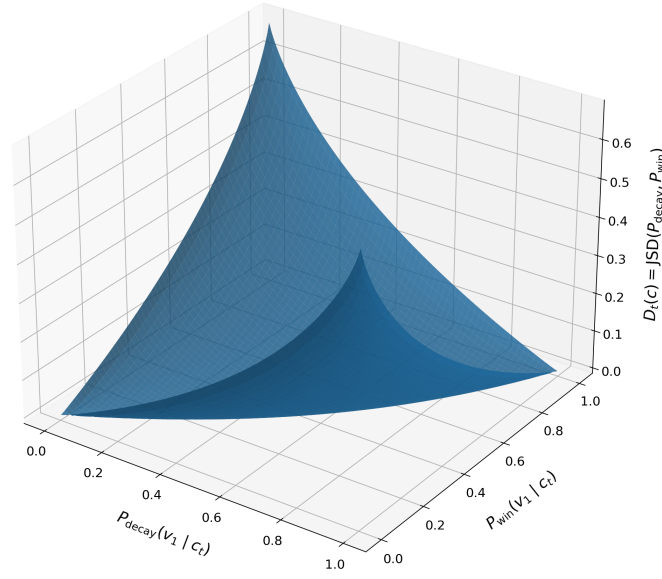
其中 $D_{\text{KL}}(\cdot \| \cdot)$ 表示 Kullback–Leibler 散度，用于衡量一个概率分布相对于另一个概率分布的信息偏离程度。对于定义在同一离散后继集合 $\text{Succ}(b)$ 上的两个分布 $P(\cdot)$ 与 $Q(\cdot)$ ，其 Kullback–Leibler 散度定义为

$$D_{\text{KL}}(P \| Q) = \sum_{v_i \in \text{Succ}(b)} P(v_i) \log \frac{P(v_i)}{Q(v_i)}. \quad (3-30)$$

当 $P(\cdot)$ 与 $Q(\cdot)$ 完全一致时， $D_{\text{KL}}(P \| Q) = 0$ ；两者差异越大，散度通常越大。由于 KL 散度不具有对称性，即一般有 $D_{\text{KL}}(P \| Q) \neq D_{\text{KL}}(Q \| P)$ ，故本文采用 Jensen–Shannon 散度对两类分布差异进行对称化处理，以获得更稳定的漂移度量。

图 3-16 进一步展示了漂移强度 $D_t(c)$ 与两类概率估计之间的关系。当 $D_t(c)$ 较小时，说明指数衰减分布与窗口分布基本一致，当前漂移不明显；当 $D_t(c)$ 较大时，则说明近期行为已明显偏离平滑历史，当前上下文更可能处于分布切换或阶段漂移状态。

(5) 基于双阈值滞回的硬自适应预测。若仅根据单一阈值对漂移强度进行模式切换，则在阈值附近容易因统计波动而频繁来回切换，导致预测器自身不稳定。为此，本文采用双阈值滞回机制，在指数衰减模式与滑动窗口模式之间进行硬自适应切换，并由当前模式唯一确定最终输出的预测分布。


 图 3-16 漂移强度 $D_t(c)$ 与指数衰减估计、滑动窗口估计之间的关系示意图

设 τ_{low} 与 τ_{high} 分别为低阈值与高阈值，且满足

$$\tau_{\text{low}} < \tau_{\text{high}}, \quad (3-31)$$

同时设

$$n_t^{(W)}(c) = \sum_{v_i \in \text{Succ}(b)} N_t^{(W)}(c, v_i) \quad (3-32)$$

表示当前上下文在窗口内的有效样本数， n_{min} 为窗口模式的最小样本门限。则当前上下文 c 下的预测模式定义为

$$\text{Mode}_t(c) = \begin{cases} \text{WINDOW}, & D_t(c) \geq \tau_{\text{high}} \wedge n_t^{(W)}(c) \geq n_{\text{min}}, \\ \text{DECAY}, & D_t(c) \leq \tau_{\text{low}} \vee n_t^{(W)}(c) < n_{\text{min}}, \\ \text{Mode}_{t-1}(c), & \tau_{\text{low}} < D_t(c) < \tau_{\text{high}}. \end{cases} \quad (3-33)$$

在此基础上，系统最终输出的分支概率分布由当前模式唯一决定，即

$$P_{\text{drift}}(v_i | c) = \begin{cases} P_{\text{win}}(v_i | c), & \text{Mode}_t(c) = \text{WINDOW}, \\ P_{\text{decay}}(v_i | c), & \text{Mode}_t(c) = \text{DECAY}. \end{cases} \quad (3-34)$$

上述机制体现了两层约束。其一，只有当漂移强度超过高阈值且窗口样本充足时，系统才切换到滑动窗口模式，以避免小样本噪声触发误切换；其二，当漂移强度处于两个阈值之间时，系统保持上一时刻的模式不变，从而形成滞回区间，抑制阈值附近的来回震荡。

图 3-17 展示了本文分布漂移适应方法的整体流程：系统首先基于当前分支节点与路径签名定位上下文状态；随后并行维护指数衰减统计与滑动窗口统计，并分别计算对应的分支分布；在此基础上，通过 Jensen–Shannon 散度估计当前漂移强度，并结合双阈值滞回规则决定当前模式；最后输出对应模式下的概率分布，供控制器进行候选集合构造与投机触发。分布漂移适应的预测流程可概括为算法 3.2。

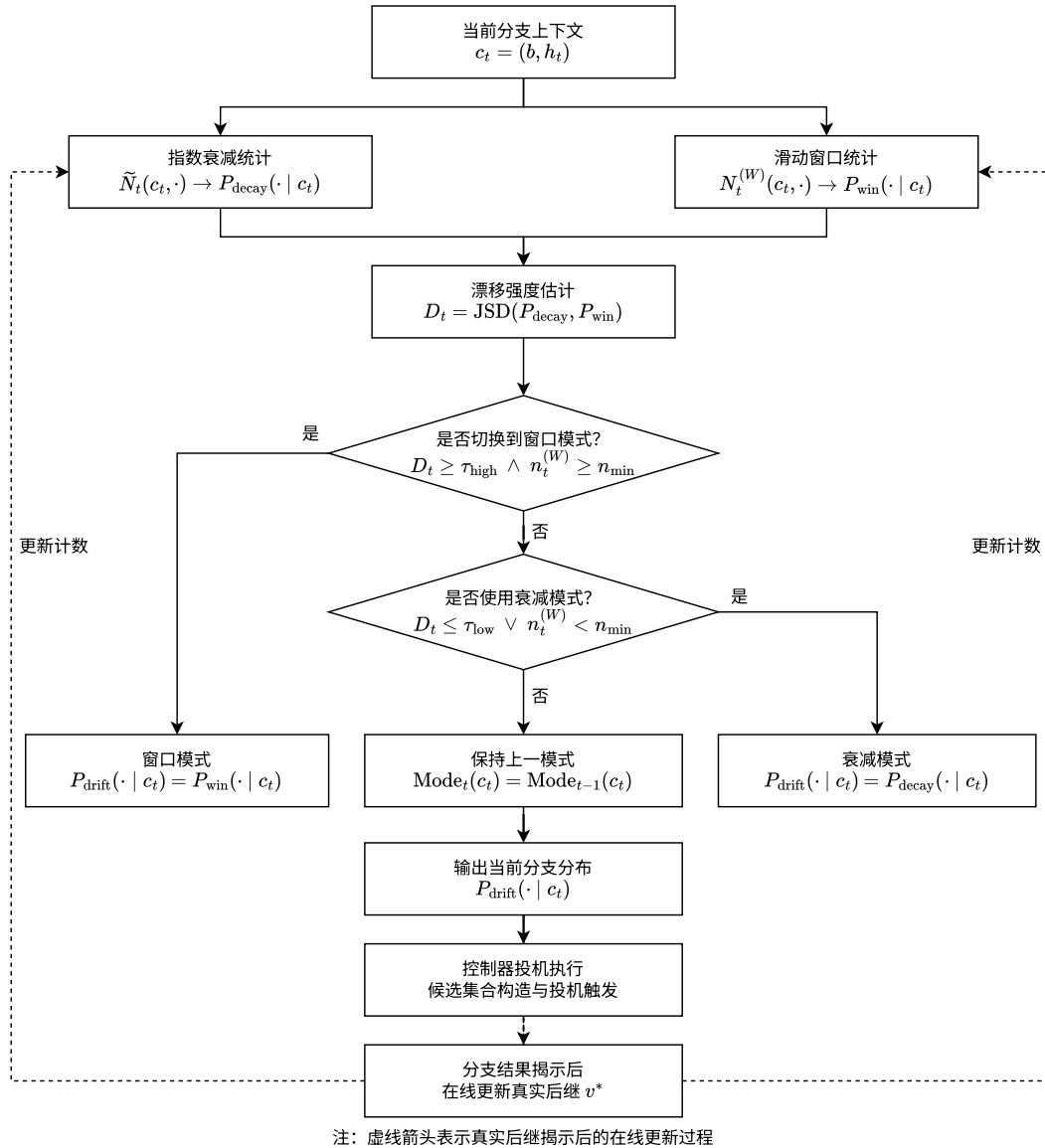


图 3-17 基于双阈值滞回的分布漂移适应分支预测方法流程图

算法 3.2: 基于双阈值滞回的分布漂移适应预测流程

输入: 分支节点 b , 路径签名 h_t , 当前真实观测结果流 (用于在线更新)
输出: 当前上下文下的分支概率分布 $P_{\text{drift}}(\cdot | c_t)$

```

1  $c_t \leftarrow (b, h_t)$  // 构造当前路径上下文
2  $P_{\text{decay}} \leftarrow \text{DECAYPRED}(c_t)$  // 依据衰减计数计算指数衰减分布
3  $P_{\text{win}} \leftarrow \text{WINDOWPRED}(c_t)$  // 依据窗口计数计算滑动窗口分布
4  $D_t \leftarrow \text{JSD}(P_{\text{decay}}, P_{\text{win}})$  // 计算当前上下文下的漂移强度
5 if  $D_t \geq \tau_{\text{high}} \wedge n_t^{(W)} \geq n_{\text{min}}$  then
6    $\text{Mode}_t(c_t) \leftarrow \text{WINDOW}$  // 漂移显著且窗口样本充足, 切换到窗口模式
7 else if  $D_t \leq \tau_{\text{low}} \vee n_t^{(W)} < n_{\text{min}}$  then
8    $\text{Mode}_t(c_t) \leftarrow \text{DECAY}$  // 漂移较弱或窗口样本不足, 使用衰减模式
9 else
10   $\text{Mode}_t(c_t) \leftarrow \text{Mode}_{t-1}(c_t)$  // 处于滞回区间, 保持上一模式
11 if  $\text{Mode}_t(c_t) = \text{WINDOW}$  then
12  return  $P_{\text{win}}$ 
13 else
14  return  $P_{\text{decay}}$ 

```

综上所述, 本文提出的分布漂移适应方法并不改变投机执行的整体闭环, 而是在统计更新与模式选择层面引入“近期优先”的建模思想。由此, 分支预测器能够更及时地贴近当前行为分布, 为后续投机触发提供更加可靠的概率估计基础。

3.4.3 输入感知与分布漂移适应的联合使用

上述两类优化分别从不同维度改进分支预测机制: 输入感知方法主要解决同一路径上下文下由输入差异引起的多模态分支行为建模问题, 分布漂移适应方法则主要解决同一统计状态随时间变化而导致的历史失效问题。前者关注“如何对状态进行细化”, 后者关注“如何对观测赋予时间权重”, 二者在建模作用维度上是相互独立的, 因此可以在同一预测器中联合使用。

在联合使用时, 系统可分别对路径级状态与桶级状态维护近期优先的统计信息, 并得到对应的时间自适应概率估计, 即

$$\hat{P}_t^{\text{path}}(v_i) = P_{\text{drift}}(v_i | b, h_t), \quad \hat{P}_t^{\text{bucket}}(v_i) = P_{\text{drift}}(v_i | b, h_t, z_t). \quad (3-35)$$

其中, 路径级估计对应于在状态 (b, h_t) 上应用分布漂移适应机制, 桶级估计则对应于将同一机制扩展到输入细化后的状态 (b, h_t, z_t) 。

在此基础上，最终分支预测结果仍采用与输入感知方法一致的层次化回退混合形式：

$$P_t(v_i) = \alpha_t \hat{P}_t^{\text{path}}(v_i) + (1 - \alpha_t) \hat{P}_t^{\text{bucket}}(v_i). \quad (3-36)$$

需要说明的是，输入感知与分布漂移适应在联合使用时，并不对应两套完全对称的实现机制。在具体实现上，联合方式采用分层组织而非简单叠加，如图 3-18 所示。

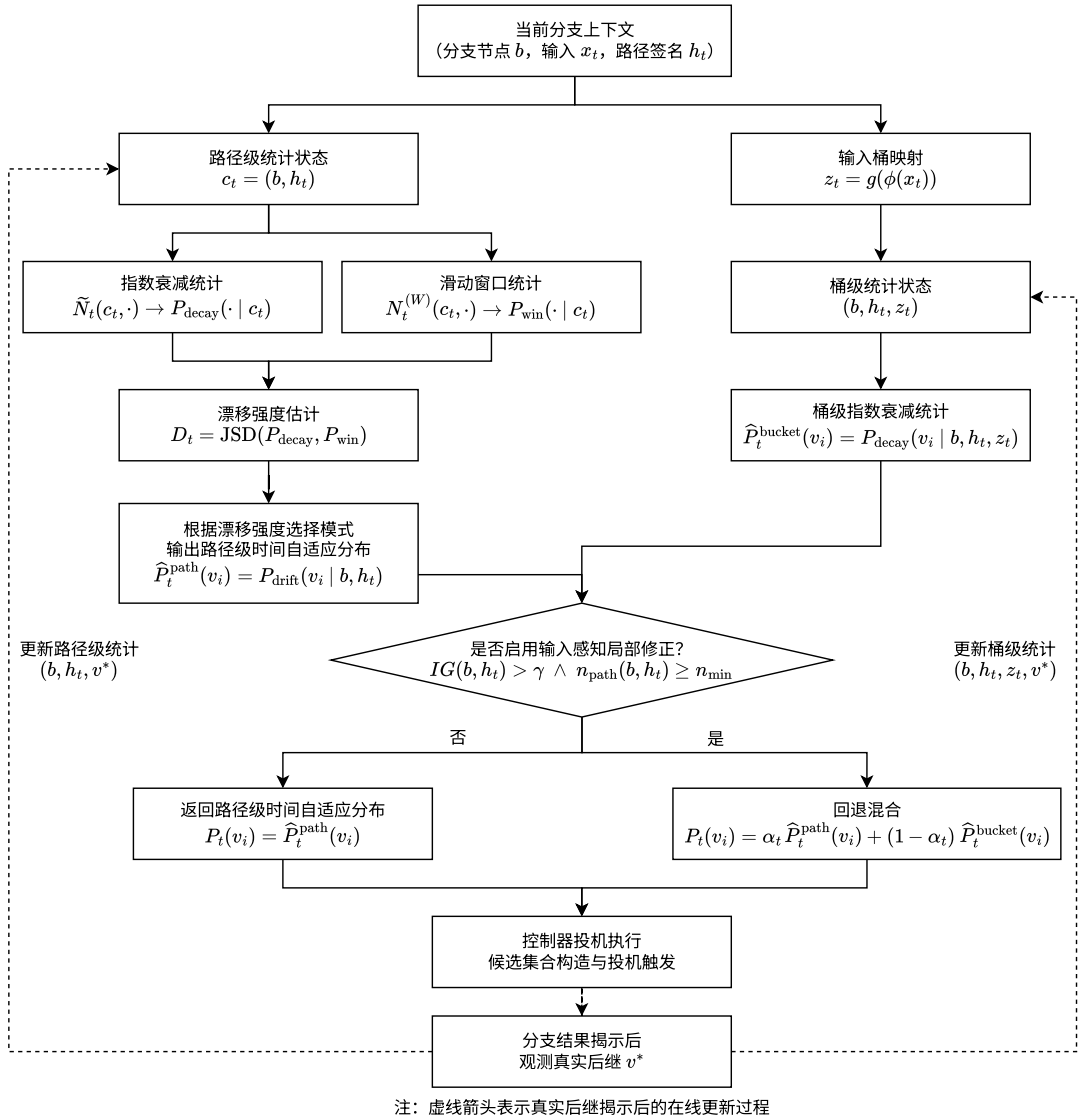


图 3-18 输入感知与分布漂移适应联合使用的分层分支预测流程图

路径级状态使用完整的分布漂移适应机制，以获得较稳定的时间自适应基线分布；而桶级状态由于输入细化后样本更稀疏，不再采用与路径级同等强度的时间窗口、漂移检测与模式切换机制，而是仅保留较为保守的指数衰减统计，并

将其作为对路径级结果的局部修正。这样做的原因在于，若在桶级状态上同样引入完整的窗口切换机制，容易放大小样本噪声，导致预测结果波动过大；相反，采用“路径级完整建模、桶级保守修正”的分层策略，能够在保持稳定性的同时保留输入细化带来的额外判别能力。

通过这种方式，预测器既能区分同一路径下由不同输入模式引起的多模态分支行为，又能及时响应这些行为随时间发生的变化，从而为候选集合构造与投机触发提供更稳健的概率估计基础。

3.5 本章小结

本章探讨了如何在 Serverless 环境下高效实现工作流执行加速。首先，本章设计了面向 Serverless 工作流的投机执行模型，形式化定义了模型中的关键组件，阐述了它们之间的交互关系与运行机制，为优化方法的设计奠定了理论基础。接着，本章提出了工作流过程描述和投机运行时配置接口两部分内容，构成了配套的编程框架，简化了用户使用模型的流程。最后，本章围绕输入感知分支预测、分布漂移适应及二者的协同集成，设计了一系列优化方法，目的是提高分支预测的准确性与适应能力，增强投机执行的有效性，从而提升工作流执行的并行度与整体效率。

第四章 Serverless workflow 投机执行原型系统设计与实现

本章将对本文实现的 Serverless workflow 投机执行原型系统进行介绍。首先介绍系统运行平台 OpenWhisk 及相关实现环境，接着介绍系统总体架构设计，最后介绍各个核心模块的详细设计。

4.1 运行平台与系统实现环境

本文提出的 Serverless workflow 投机执行方法在原理上具有一定普适性，可部署于多种支持函数执行与组合编排的 Serverless 平台之上。基于平台开放性、函数编排能力、运行结果可观测性以及已有基线工作的可比性等优点，本文选择 Apache OpenWhisk 作为原型系统的运行平台。

4.1.1 OpenWhisk 平台及其 workflow 基础

OpenWhisk 是一个开源的 Serverless 计算平台，其基本编程实体包括 action、trigger、rule 和 package 等。其中，action 对应可执行函数，是平台最基本的计算单元；trigger 与 rule 用于表达事件驱动关系；package 则用于组织相关动作与资源。其编程模型如图 4-1 所示。

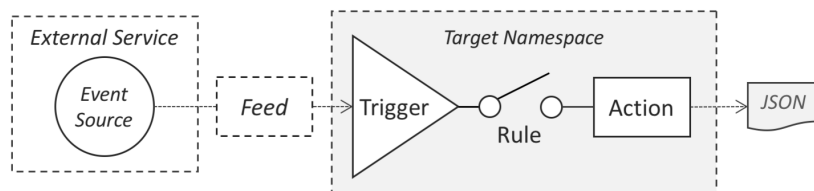


图 4-1 OpenWhisk 编程模型^[31]

对于本文关注的工作流场景而言，最关键的是 OpenWhisk 提供了函数组合能力，使多个函数能够按照一定控制关系构成更大粒度的执行过程。

在原生能力上，OpenWhisk 提供了两类与 workflow 相关的基础机制。第一类是 action sequence，即将多个函数按线性顺序串联，使前一个函数的输出

自动成为后一个函数的输入——这种方式适合描述简单的串行处理链。第二类是基于 Composer 的显式组合能力，允许用户以更高层方式描述顺序、条件、循环等控制结构，并将组合逻辑编译为可在 OpenWhisk 上执行的 conductor action。

需要指出的是，OpenWhisk 原生提供了函数组合的基础能力，但并不直接提供本文所需的工作流级投机执行机制。因此，本文在 OpenWhisk 提供的函数执行与组合基础之上，进一步构建平台外的投机协调能力。

4.1.2 系统与 OpenWhisk 的交互方式

在实现路径上，本文将 OpenWhisk 作为底层函数执行的 Serverless 后端。系统负责维护 workflow 描述、运行配置、节点语义、预测状态以及投机执行相关控制逻辑；OpenWhisk 则负责接收函数部署请求、执行具体动作实例，并返回动作级的运行结果。

基于这一定位，本文系统与 OpenWhisk 的交互主要包括三类操作。第一类是**函数部署**：系统根据 workflow 定义，将各节点对应的函数注册为 OpenWhisk actions，并保存函数名称、节点类型及相关元数据。第二类是**函数调用**：在一次 workflow 运行过程中，控制器根据当前 workflow 状态决定应触发哪个函数，并通过 OpenWhisk 提供的标准调用接口发起动作执行；若遇到分支节点，则由协调器在平台外完成预测与候选集合构造，再提前调用一个或多个候选后继函数。第三类是**运行结果查询**：函数执行完成后，系统通过 OpenWhisk 的 activation 记录获取动作级结果、状态与日志摘要，并在平台外进一步还原为 workflow 级时间线、投机命中情况与恢复过程。

具体而言，投机执行相关的工作流拓扑、分支预测、预算控制、状态缓冲、验证恢复等逻辑均由在平台外维护；而 OpenWhisk 负责按请求执行具体函数节点，并提供统一的动作调用与运行记录接口。这种实现方式的优点在于，一方面无需侵入修改底层平台源码，工程实现更灵活，有利于平台的扩展性和可移植性；另一方面又能够复用 OpenWhisk 已有的函数管理、实例执行与运行结果可观测能力，为本文后续的架构设计、核心模块实现与系统运行流程提供稳定支撑。

在此基础上，本文原型系统采用 Python 实现，向上提供统一的工作流描述

接口、运行配置接口与命令行工具，向下通过 HTTP/REST 与 OpenWhisk 平台交互，完成函数部署、函数调用与 activation 查询。

4.2 系统总体架构设计

基于前一节给出的运行平台与实现环境，本文在第三章所提出的投机执行模型与编程框架基础上，设计并实现了一个面向 Serverless 工作流的原型系统，其总体架构如图 4-2 所示。

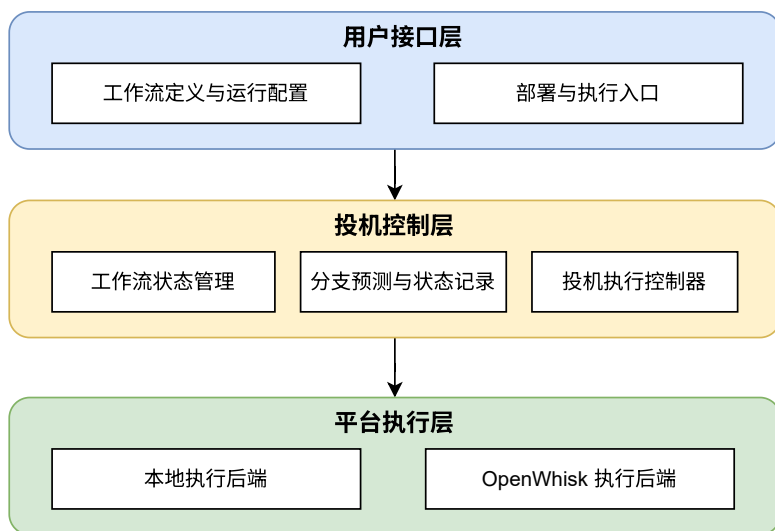


图 4-2 SpecSW 原型系统总体架构

从整体上看，系统采用三层式架构，分别为**用户接口层**、**投机控制层**和**平台执行层**。其中，用户接口层负责接收工作流定义、运行配置以及部署与执行请求；投机控制层负责维护工作流状态，组织任务推进，并完成分支预测、投机触发和状态更新；平台执行层负责承载具体函数任务的运行，并向上返回执行结果。三层之间分工明确、协同配合，共同支撑工作流投机执行的完整过程。

用户接口层是系统面向用户的统一入口。该层主要负责描述工作流的结构信息、组织运行参数，并向系统提交部署和执行请求。对用户而言，这一层屏蔽了底层执行环境与控制逻辑的复杂性，使工作流能够以统一的方式进行定义、配置与调用，从而提升了系统的可用性与实验操作的一致性。

投机控制层是整个系统的核心。该层围绕一次工作流运行维护统一的逻辑状态，并根据当前节点类型和运行上下文决定后续推进方式。对于常规节点，系统按照既定的拓扑关系推进后继任务；对于分支节点，系统结合预测结果决定是

否提前触发候选后继，从而实现投机执行。与此同时，该层还负责维护与投机相关的模型状态，对运行过程中产生的信息进行在线更新，并组织执行记录与结果汇总。因此，投机控制层不仅承担执行推进职责，也承担状态学习和运行管理职责，是连接 workflow 描述与底层执行环境的关键枢纽。

平台执行层负责具体函数任务的承载与执行，是系统与不同运行环境之间的连接部分。该层向上提供统一的任务执行能力，向下适配具体的执行后端，从而保证上层控制逻辑能够以一致方式触发函数、接收结果并推进后续流程。通过这一层，系统能够在不同执行环境下保持相对统一的运行语义，同时避免将上层控制逻辑直接耦合到某一种底层平台之上。

从系统运行过程来看，整体数据流和控制流可概括为如下闭环：首先由用户接口层给出 workflow 定义和运行配置；随后投机控制层在运行开始前完成状态初始化，并在执行过程中根据当前 workflow 状态组织任务推进；平台执行层负责实际执行被触发的函数任务，并将结果返回给控制层；最后，控制层依据执行结果更新内部状态、记录运行信息，并继续推进后续任务。

总体而言，本文原型系统的总体架构以用户接口层、投机控制层和平台执行层为主线，分别承担输入组织、执行控制和任务承载三类职责。这样的三层结构能够清晰反映系统的整体组织方式，为后续核心模块的展开提供结构基础。

4.3 系统核心模块设计

4.3.1 workflow 描述与运行组织模块

3.3 节已经从编程框架角度说明了用户如何描述 workflow 与配置投机执行策略；本节进一步从系统实现角度说明这些描述如何被解析、组织并转化为运行时可消费的内部表示。 workflow 描述与运行组织模块的主要职责，是将 workflow 定义、运行配置和部署结果组织为统一的可执行对象，并为后续控制器、预测器和执行后端提供一致的运行入口。

系统以 `workflow.json` 作为 workflow 的静态定义输入，其中 `entrypoint` 用于确定运行起点，`functions` 给出可执行节点集合，而 `sequence_table` 则统一描述节点之间的顺序与分支后继关系。系统在解析阶段将这些内容转换为内部的函数描述集合与顺序表条目集合。其中，顺序表是系统用于组织 workflow

拓扑的内部结构，每个条目对应一个函数节点，并记录其确定性后继或候选分支后继。通过这一转换， workflow 定义由面向用户的静态配置转化为面向运行时的统一表示。

表 4-1 workflow 定义中的关键字段及其作用

字段	作用说明
name	标识 workflow 名称
entrypoint	指定 workflow 入口节点
functions	给出 workflow 中的函数节点定义
sequence_table	描述节点之间的顺序与分支后继关系

如表 4-1 所示， workflow 定义中的关键字段主要服务于运行时组织。在系统内部，顺序表既承担拓扑组织作用，也为后续执行控制提供直接依据：普通节点通过确定性后继推进，分支节点则显式维护候选后继集合，供后续分支预测与投机控制使用。与此同时，节点语义属性也一并进入内部表示，例如分支属性、纯函数属性和非投机属性分别参与预测触发、记忆表快速路径和投机边界控制。

在部署阶段，系统进一步生成统一的 manifest.json，用于固化 workflow 与执行环境之间的绑定关系，其中保存函数映射、顺序表以及所选执行后端等运行所需信息。运行开始前，系统装载 workflow 定义、运行配置和部署结果，恢复内部 workflow 表示与函数映射，并据此构造后续执行所需的上下文。由此， workflow 描述与运行组织模块完成了从用户定义到系统运行表示的转换。

4.3.2 投机执行控制器

投机执行控制器是原型系统的核心执行组件，负责在一次 workflow 运行期间维护运行时状态，并组织任务提交、并发执行、分支投机、路径确认和结果收敛。本文原型在这一部分沿用了 SpecFaaS 的基本控制思路^[24]：控制依赖通过分支预测提前展开，数据依赖通过记忆表支持后继任务的提前启动；在此基础上，结合本文的 workflow 描述与运行组织方式完成统一实现。

(1) 任务组织。为支持并发执行，控制器内部采用线程池管理任务生命周期，并为每一次函数触发构造统一的任务句柄。任务句柄保存函数标识、输入、投机标记、执行句柄以及是否跳过底层调用等信息，用于统一管理真实路径任务和投机路径任务。与此同时，控制器还维护当前已启动任务、各实例绑定输入、已确

认真实路径以及记忆表等运行状态。表 4-2 总结了控制器维护的关键运行对象与状态，这些状态共同为路径确认、结果汇总和运行追踪提供支撑。

表 4-2 投机执行控制器维护的关键运行对象与状态

对象/状态	作用说明
任务句柄	表示一次函数触发实例，记录其基本执行信息
已启动任务集合	记录当前已经提交或正在执行的任务
输入缓存	保存各任务实例绑定的输入
已提交路径	记录当前已经确认为真实路径的节点序列
记忆表	保存函数历史输入与输出的对应关系
任务状态标记	记录任务是否已完成、已提升或已撤销

(2) 常规执行。在常规执行模式下，控制器按照 workflow 顺序表推进任务。若当前节点是普通节点，系统提交该节点对应的函数实例，等待其执行完成，再依据返回结果确定后继节点及其输入，并继续沿拓扑关系推进。该过程对应严格依赖执行，其重点在于保证节点输入与路径推进的一致性。

(3) 控制依赖投机。当控制器遇到分支节点时，执行逻辑转入投机处理阶段。系统先收集该分支节点的候选后继集合，再基于当前已提交路径构造预测上下文，并调用分支预测器输出预测结果。若预测结果满足触发条件，控制器会在真实分支尚未揭示前提前启动预测命中的后继任务；在允许更深层扩展时，系统还会继续沿预测路径扩展投机链，从而将部分后继计算前移到分支判定之前，减少关键路径上的等待时间。

(4) 数据依赖投机。除控制依赖投机外，控制器还通过记忆表支持数据依赖投机。系统为函数维护历史输入与输出之间的映射；当控制器准备启动某个函数时，若在其记忆表中找到匹配输入，则可直接取出预测输出，并据此提前启动后继函数。对于一般函数，命中记忆表后仍继续执行当前函数，其真实输出在后续提交阶段用于校验和更新记忆表。对于标记为纯函数的节点，系统在命中记忆表时可直接复用已有输出，并跳过该函数的实际执行。这样，记忆表一方面支持后继任务的提前启动，另一方面也为纯函数提供了快速执行路径。

(5) 任务状态转换。图 4-3 展示了控制器的核心运行逻辑。系统在运行过程中首先根据当前状态获取可执行节点及其输入，并结合记忆表判断当前节点是否可以复用已有结果。对于普通节点，系统沿确定性后继继续提交任务；对于分支节点，系统先调用预测器生成候选后继，并在满足阈值条件时启动预测后继任务。待真实分支结果返回后，控制器再对已启动任务进行路径确认：位于真实路

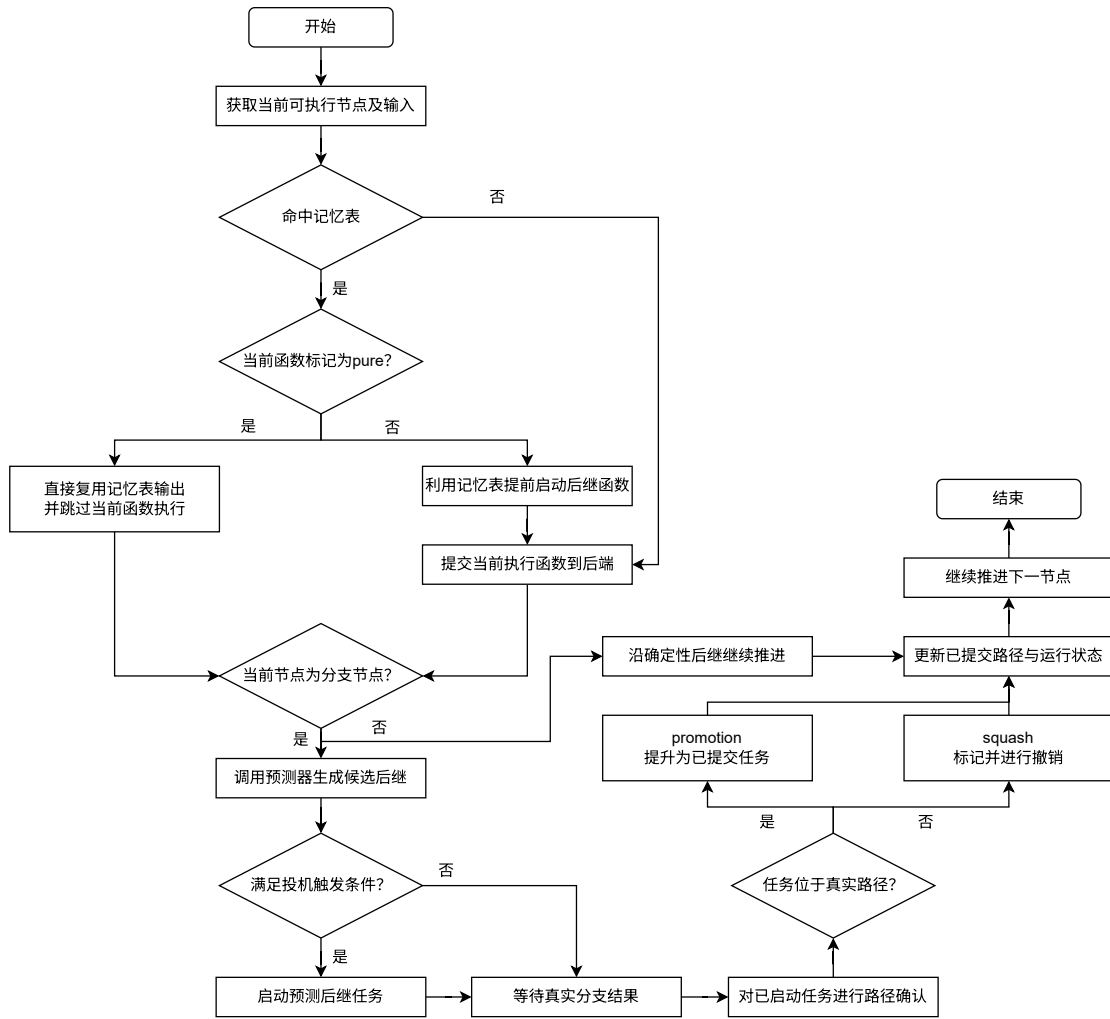


图 4-3 投机执行控制器的运行逻辑示意图

径上的投机任务被提升为已提交任务，其余任务被标记为撤销。随后，系统更新已提交路径与运行状态，并继续推进后续节点，直至整个 workflow 执行结束。

在真实路径与投机路径的衔接上，系统采用提升和撤销两种状态转换语义。提升表示某个投机任务在真实路径确认后被纳入正式提交链路；撤销表示某个已启动任务不属于真实可达路径，其结果不再参与后续推进。系统允许错误路径任务继续运行到完成，再通过状态标记完成撤销处理，以便稳定记录投机命中、误投机开销和时间线重叠情况。

(6) 副作用处理。副作用处理通过受控接口和两阶段可见性机制完成。在不修改 OpenWhisk 源码的条件下，系统无法透明拦截任意函数内部的外部 I/O，因此要求开发者在编写带副作用的函数时遵循统一的受控编程方式。具体而言，函数不直接对真实外部系统执行读写操作，而是通过框架提供的接口显式表达副作用意图。对于读操作，系统记录被访问资源及其上下文；对于写操作和消息发

送操作，系统先将其登记到与当前任务绑定的暂存区域，而不立即提交到真实外部环境。表 4-3 给出了开发者侧需要遵循的受控副作用接口约定。

表 4-3 开发者侧受控副作用接口约定

接口	输入参数	作用说明
effect_read	资源标识、读取参数、上下文标识	声明一次受控外部读取请求
effect_write	资源标识、写入内容、上下文标识	登记一次待提交写入
effect_emit	通道标识、消息内容、上下文标识	登记一次待发送消息或事件

控制器在提交任务时会为其分配运行标识、任务标识和路径标识，并将这些元信息一并传入函数执行上下文，从而使副作用记录能够与具体任务实例绑定。在路径确认之后，控制器再统一决定这些副作用意图的处理方式。若对应任务被提升为真实路径任务，则将其暂存写入和消息发送提交到真实外部环境；若对应任务被撤销，则丢弃其暂存结果，并在需要时执行清理。控制器侧的副作用管理接口如表 4-4 所示。

表 4-4 控制器侧副作用管理接口约定

接口	输入参数	作用说明
effect_commit	任务标识、路径标识	提交与真实路径任务绑定的副作用
effect_discard	任务标识、路径标识	丢弃与误投机任务绑定的副作用

这样，函数执行阶段与副作用对外可见阶段被显式区分开来，副作用的提交或丢弃与路径确认保持一致，误投机路径上的写入不会提前影响真实执行结果。

(7) 非投机屏障。为进一步限制投机扩展范围，控制器支持非投机屏障机制。对于节点类型属于 `non_speculative` 的节点，系统将其视为投机扩展的停止边界，不再对其后继执行控制投机或数据投机；若在扩展投机链过程中命中此类节点，则链式扩展立即停止。此类节点通常直接产生不可安全重放、不可安全撤销或强依赖外部环境状态的副作用，因此系统要求其在真实输入和真实路径确定后再执行。

总体而言，投机执行控制器负责组织真实路径与投机路径的并发运行，并通过输入绑定、状态标记、结果可见性控制和边界约束保证整个投机执行过程可控、可观测且具有一致的工作流语义。

4.3.3 分支预测与模型状态管理模块

分支预测与模型状态管理模块负责为投机执行控制器提供分支决策依据，并维护跨多次运行持续演化的模型状态。3.4 节已经从方法层面详细给出了输入感知、分布漂移适应及其联合建模的优化机制，包括相应的统计对象、启用条件与组合方式；因此，本节不再重复展开这些优化策略本身的推导与判定细节，而是重点说明它们在系统中如何被组织为统一的预测器状态、预测流程与持久化机制。总体上，该模块承担两项职责：一是根据当前分支节点的路径上下文和可选输入信息输出预测决策，二是在每次运行结束后依据真实执行结果更新预测器和记忆表，从而逐步提升后续执行质量。

(1) 预测器的内部表示。系统将分支预测实现为一个路径敏感的 N-way 预测器。对于任意分支节点，预测器直接在其全部候选后继之间输出概率分布，而不是像已有工作一样，仅进行二分类判断。这样能够自然适配工作流中的多路分支场景，并与顺序表条目中的候选后继集合保持一致。实现上，预测器内部维护一个键值映射：键由分支节点标识、路径签名以及可选输入桶标识组成，值则保存候选后继的观察计数以及窗口或衰减所需的附加状态。其中，路径签名由控制器根据当前实例最近若干个已确认节点构造，用于表示当前分支点的局部执行上下文。系统预测模式配置项支持基础模式、输入感知模式、漂移适应模式以及联合模式。预测器的关键配置及默认值如表 4-5 所示。

表 4-5 分支预测器关键配置参数、默认值及其作用

符号	作用说明	默认值
M	预测模式	base
τ	投机触发阈值	0.60
K	路径上下文长度	8
λ	Laplace 平滑系数	1
γ_{decay}	衰减系数	0.99
W	滑动窗口长度	32
$ Z $	输入桶空间大小	64
k	回退平滑系数	32
γ_{input}	输入启用阈值	0.05
$n_{\text{min}}^{\text{input}}$	输入最小样本量	16
τ_{low}	漂移回退阈值	0.01
τ_{high}	漂移进入阈值	0.05
$n_{\text{min}}^{\text{drift}}$	漂移最小样本量	16

(2) 输入特征提取与输入桶映射。具体实现 3.4.1 节的输入特征提取和桶映

射时，预测模块首先需要从当前请求输入中抽取目标字段；对于数值型字段，可按预设区间进行离散化；对于枚举型或类别型字段，则映射为离散类别标识；对于多字段组合场景，系统进一步将多个字段的离散结果拼接后映射到固定数量的桶空间中。经过该过程后，原始输入被统一压缩为桶标识 z_t ，并与路径签名共同形成桶级状态键 (b, h_t, z_t) 。这样，预测器即可在不直接保存原始输入的前提下，对同一路径上下文下由输入差异引起的分支行为变化进行更细粒度建模，同时将状态空间控制在可管理范围内。

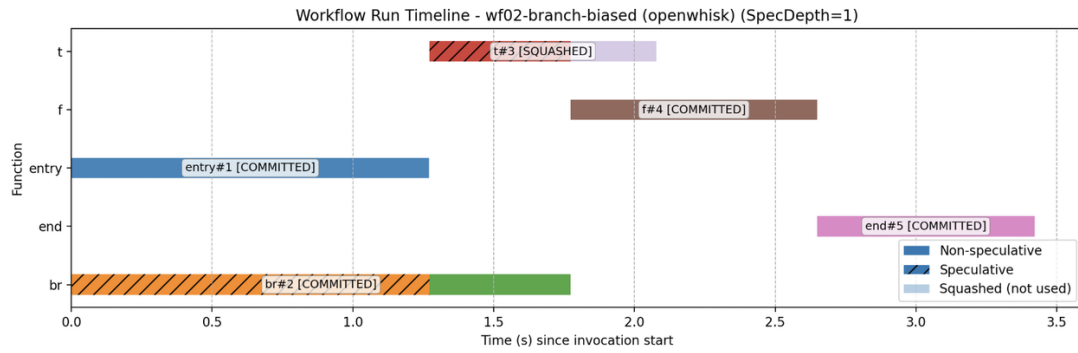
(3) 预测流程与分支决策生成。当控制器在运行过程中到达某一分支节点时，会向预测模块发起一次预测请求。该请求至少包含当前分支节点标识、路径签名以及可选输入上下文。预测模块首先根据分支节点和路径签名定位对应的路径级状态；若当前预测模式包含输入感知机制，则进一步抽取输入特征并映射为桶标识 z_t ，查询对应的桶级状态 (b, h_t, z_t) 。随后，模块依据当前配置的预测模式选择相应的统计信息与组合方式，对全部候选后继进行概率估计，并按概率降序排序。最终，预测结果被统一封装为分支决策对象 `BranchDecision`，其中除预测后继及其概率外，还包括是否建议触发投机、当前采用的预测模式以及必要的解释信息，供控制器直接消费。

(4) 模型状态更新机制。分支预测器并非静态模型，而是一个随 workflow 多次运行持续演化的在线状态化预测器。因此，在每次分支节点的真实执行结果揭示后，控制器都会将真实后继、路径签名以及可选输入桶标识回传给预测模块，触发一次状态更新流程。需要说明的是，预测状态的更新始终以真实执行结果为准，而不会因为投机路径的提前展开而直接修改预测状态，从而保证模型学习过程与 workflow 真实执行语义保持一致。

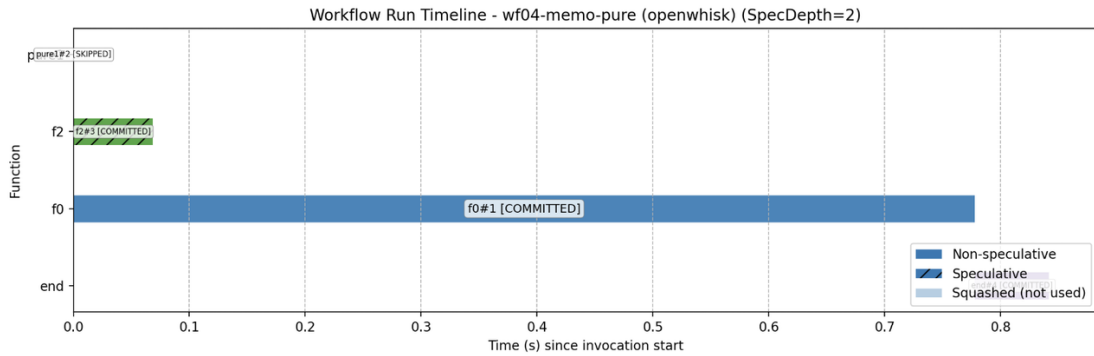
(5) 状态持久化与恢复。由于分支预测器需要跨多次运行持续积累统计信息，原型系统支持对预测状态进行周期性持久化，并在后续启动时重新恢复。持久化内容主要包括路径级观察计数、桶级观察计数、指数衰减统计、窗口缓存、模式标记以及与输入启用判定相关的辅助指标等。在实现上，预测模块将内存中的状态表按照统一格式序列化写入外部存储，并在系统初始化阶段完成反序列化恢复。

4.3.4 系统运行支撑模块

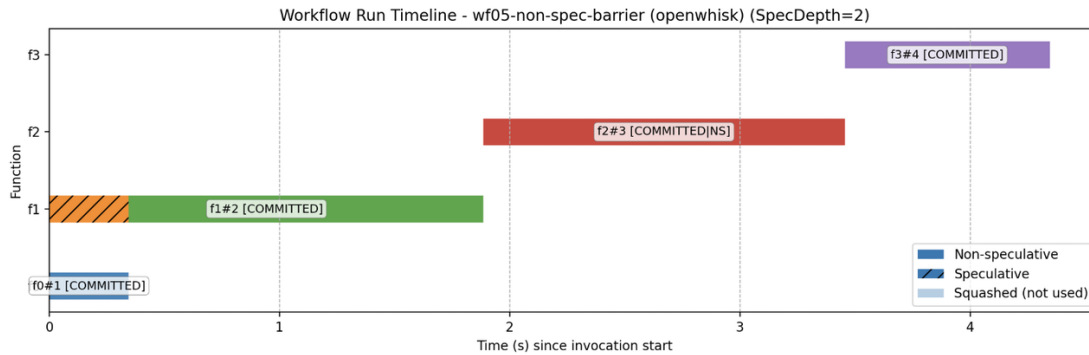
除工作流描述、分支预测与投机控制等核心逻辑外，原型系统还实现了一组面向运行支撑的配套模块，用于完成运行记录归档、执行过程追踪、时间线可视化、后端环境适配以及命令行交互等工作。这些模块虽然不直接参与单次分支决策，但为系统的持续学习、实验复现、运行分析和工程化使用提供了必要支撑，因此也是原型系统能够稳定运行的重要组成部分。



(a) 误预测撤销



(b) Memo 命中跳过执行



(c) 非投机屏障约束

图 4-4 系统运行支撑模块的典型时间线可视化结果

(1) 事件追踪与时间线可视化。为提升系统运行过程的可观测性，本文实现

了任务级事件追踪机制。具体而言，系统中的 Tracer 会为每个被提交的任务分配唯一的任务标识，并在任务开始、结束、状态切换以及投机执行任务被提升等关键时刻记录事件。Tracer 支持生成任务编号、记录任务时间、设置状态、记录投机执行任务在中途被确认进入真实路径时的时间点等。

在可视化层面，系统使用 matplotlib 将每个任务实例绘制为一条横向时间条。图 4-4 给出了系统运行支撑模块生成的典型时间线可视化结果，分别覆盖了三类具有代表性的场景——强偏置分支下的误预测撤销场景、纯函数跳过执行场景和非投机屏障约束场景。

(2) 执行后端适配。为避免投机控制逻辑与具体运行环境发生直接耦合，系统设计了统一的后端调用抽象，即通过 `Provider.invoke(fn_spec, inp)` 向上层提供一致的函数触发语义。基于这一抽象，原型系统实现了本地执行后端以及 OpenWhisk 执行后端。本地后端适合开发调试与单元测试。OpenWhisk 后端并负责输入封装、输出解析、分支函数结果转换以及常见瞬时错误下的重试与退避控制：当输入为字典时可直接作为 JSON 参数传递，非字典输入则会进行包装；对返回结果则先尝试做 JSON 解析，再对 OpenWhisk 风格的 `envelope` 进行防御性解包；对于分支 `action`，系统要求返回对象中包含 `next_func` 字段，并将其规范化为内部统一的 `BranchResult` 形式。通过这一适配层，控制器可以在不修改执行逻辑的前提下切换本地调试、模拟验证和真实平台调用环境。

表 4-6 specx 主要命令及其功能说明

子命令	功能说明
deploy	将工作流源码目录部署为可运行对象，解析 <code>work-flow.json</code> ，生成或写出对应的 <code>manifest</code> ，并根据 <code>provider</code> 类型选择本地或 OpenWhisk 部署方式。
run	运行指定工作流，是系统最核心的执行入口。该命令支持通过 <code>--spec</code> 或 <code>--no-spec</code> 控制是否启用投机执行，并通过 <code>--spec-depth</code> 等参数设置链式投机深度及预测器配置。
last	查询最近一次或若干次历史运行记录，用于查看最终输出、运行路径以及对应的运行记录文件。
model	查看持久化模型状态，包括预测器计数表与 <code>memo</code> 表内容，用于分析在线学习后的统计状态。
plot	根据历史运行记录中的 <code>timeline_events</code> 绘制时间线图，用于展示 <code>speculative</code> 、 <code>promotion</code> 、 <code>squash</code> 、 <code>skip</code> 等任务状态。

(3) 命令行接口与统一运行入口。在用户交互层面，系统通过统一的命令行

工具 `specx` 封装了 workflow 部署、运行、历史记录查询、模型状态查看和图形绘制等常见操作。表 4-6 对系统中几个主要命令及其功能进行了归纳。

总体而言，系统运行支撑模块为原型系统提供了运行结果归档、任务级事件追踪、执行过程可视化、后端环境切换以及统一实验入口等关键能力。正是这些支撑能力的存在，才使本文提出的 workflow 投机执行机制不仅能够在功能上实现，而且能够以可重复、可分析、可验证的方式进行系统化评估。

4.4 本章小结

本章介绍了 Serverless workflow 投机执行原型系统的设计与实现。首先介绍了系统所使用的 Apache OpenWhisk 平台及相关实现环境，说明了系统与底层执行平台的交互方式。接着阐述了系统整体的架构设计，说明了各层及各组件在系统中的功能。最后详细介绍了系统中的各个核心模块，描述了它们如何实现前文提出的投机执行方法、分支预测优化机制以及相关运行支撑能力。

第五章 实验设计与结果分析

本章将力图客观地评估本文提出的 Serverless workflow 投机执行优化方法和系统 SpecSW 的有效性。实验分为四个部分：核心算法仿真实验、功能验证实验、端到端性能评估实验和消融实验，分别回答“核心机制是否有效”、“系统功能是否完备”、“实际性能是否优越”、“关键设计是否必要”四个关键问题。

5.1 实验配置

本文实验平台采用单节点容器化部署方式构建，即在一台本地实验主机上运行完整的 OpenWhisk 平台及本文实现的原型系统。运行的 Serverless 系统组件，包括函数控制组件、消息队列、数据库以及函数执行节点。所有系统组件均以 Docker 容器形式部署，通过容器网络进行通信，从而构建完整的 Serverless 运行环境。实验所使用的主要软硬件环境配置如表 5-1 所示。

表 5-1 实验软硬件环境配置

类别	配置
操作系统	Ubuntu 22.04
CPU	阿里云 vCPU（16 vCPU）
内存	32 GB
本地存储	512 GB ESSD
容器环境	Docker
编程语言	Python 3.12
Serverless 平台	Apache OpenWhisk
消息系统	Apache Kafka
状态数据库	Apache CouchDB

在实验工作负载方面，本文同时采用自构造 workflow 与真实应用 workflow 相结合的方式进行评估。一方面，为便于分析投机执行机制及分支预测优化方法的效果，本文构造了一组具有代表性的 Serverless workflow 基准，这些 workflow 涵盖线性结构、条件分支、多分支以及输入相关分支等典型控制流模式，并通过设置不同输入分布与请求序列，用于验证基础投机执行、输入感知预测以及分布漂移适

应等机制的有效性；另一方面，为评估所提出方法在实际场景中的端到端性能表现，本文选取了常见的 Serverless 应用场景，包括用户请求处理流、图像处理流以及数据分析流，覆盖条件分支、数据依赖及动态负载变化等典型场景。

需要说明的是，本文实验采用单节点部署主要用于验证算法机制与系统实现效果。由于研究重点在于分支预测与投机执行策略对函数执行路径的影响，而非评估大规模集群调度性能，因此单节点环境能够有效减少网络延迟和节点差异带来的干扰，提高实验可控性。同时，由于系统架构采用模块化设计，在多节点环境下仍可直接进行水平扩展，因此实验结果具有良好的可推广性。

5.2 核心算法仿真实验

为了验证 3.4 节中提出的关键优化方法的有效性，本文将进行核心算法仿真实验，对输入感知的分支预测、分布漂移适应以及二者联合使用的效果进行独立评估。

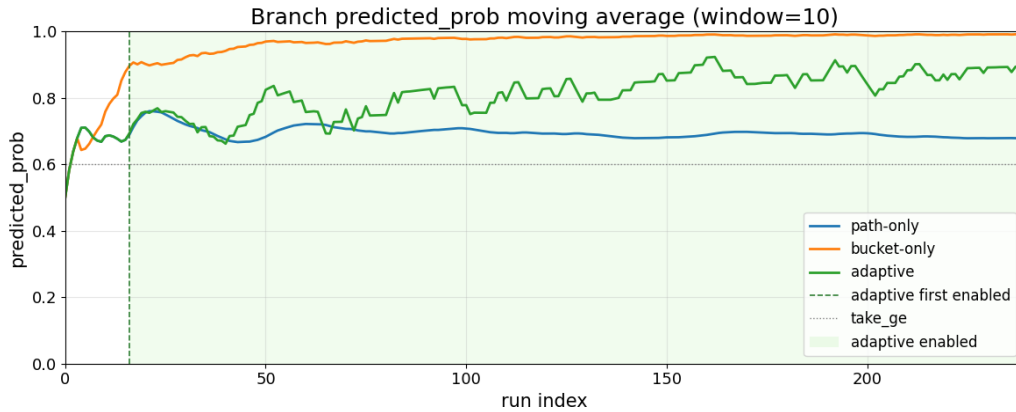
为了生成仿真数据，本文构建了三类具有代表性的模拟工作流，分别用于刻画输入差异主导、分布随时间变化主导以及两种因素共同作用的场景。对于每类工作流，本文进一步设置相应的输入分布、阶段切换方式和请求序列，使工作流在执行过程中呈现出不同的分支行为特征。在此基础上，分别将基线预测方法、核心优化策略以及含自适应机制的优化策略应用于这些工作流，并比较它们在不同场景下的预测效果与执行收益。通过仿真实验，可以在较少受真实平台调度、网络通信和运行时噪声干扰的条件下，更清晰地分析各项优化方法本身的作用机制及适用场景。

5.2.1 输入感知分支预测效果评估

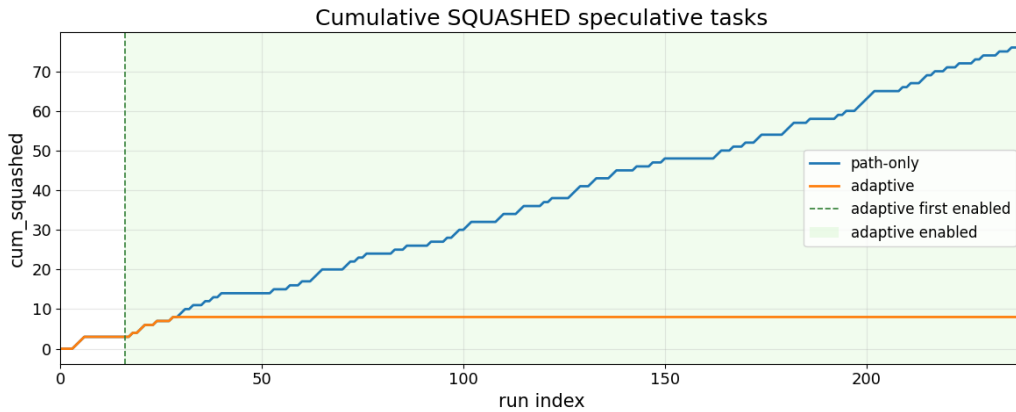
为评估本文提出的输入感知分支预测机制在输入与分支结果相关的场景中的有效性，本实验设计了一个输入驱动型分支工作流——该工作流中，分支走向由输入唯一决定。实验共运行 240 次，其中一种输入的概率为 0.65，另一种为 0.35。预测器推测深度为 2，其余相关参数均采用默认配置，具体同表 4-5。

图 5-1 展示了两策略在输入驱动分支场景下的整体对比结果，绘图的移动平均窗口为 10。可以看出，仅依赖路径上下文的策略虽然能够逐步学习到主导

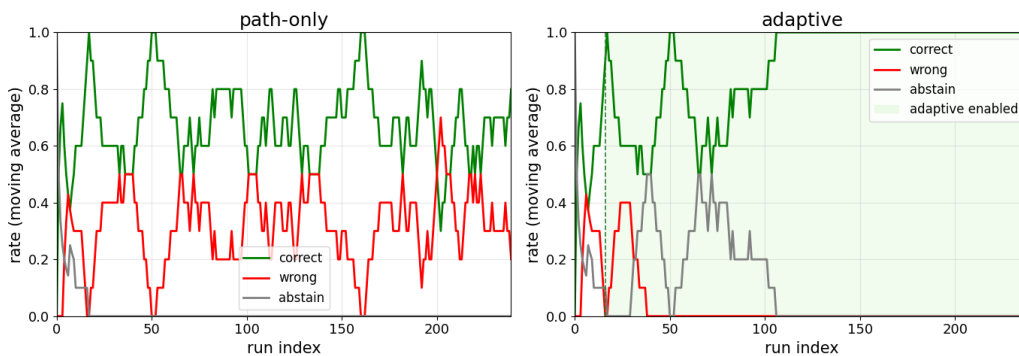
分支偏置，但由于无法区分不同输入对应的真实后继，因此在少数输入上会持续产生错误投机；相比之下，输入感知策略在实验初期与原始策略表现接近，随后随着样本数与判别信息逐步积累，开始启用输入细化，并在后期表现出更稳定的预测效果。



(a) 最大预测概率变化情况



(b) 累计撤销开销对比



(c) 不同结果比例的变化情况

图 5-1 不同输入建模策略在输入驱动分支场景下的对比结果

为便于比较不同策略在输入驱动分支场景下的差异，表 5-2 汇总了两种策略

的主要统计结果。

表 5-2 仅路径统计与输入感知策略在输入驱动分支场景下的结果对比

策略	总投机率	总误预测率	y 类误预测率	平均撤销量	平均浪费时间/ms
仅路径统计	99.17%	31.93%	100.00%	0.317	0.135
输入感知策略	90.83%	3.67%	14.29%	0.033	0.015

可以看出，在输入决定分支后继的场景下，仅依赖路径级统计的策略难以识别相同路径上下文下由不同输入触发的后继差异，因而容易对少数输入持续产生错误投机；相比之下，输入感知策略能够在样本逐步积累后利用输入信息修正路径级偏置，从而更有效地抑制错误投机带来的额外开销。具体而言，输入感知策略的平均撤销量较仅路径依赖策略降低了 89.47%，平均浪费时间降低了 88.98%，表明其在输入驱动分支场景下具有更好的判别能力与稳定性。

5.2.2 分布漂移适应效果评估

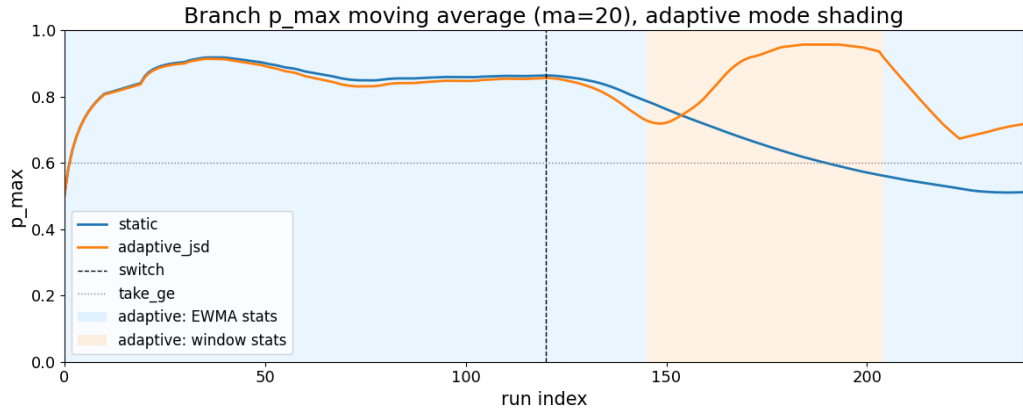
为评估本文提出的自适应分支统计机制在分布漂移场景下的有效性，本实验设计了一个二阶段突变型漂移场景 workflow。该 workflow 通过控制分支节点输入的概率分布，其中，前 120 次运行中，分支有 0.9 的概率选择漂移前主导路径；第 120 次运行后，概率突变为 0.1。预测器推测深度为 2，其余相关参数均采用默认配置，具体见表 4-5。

图 5-2 展示了两种策略在分布漂移场景下的整体对比结果，绘图的移动平均窗口为 20。表 5-3 汇总了长记忆统计与分布漂移适应两种策略的主要统计结果。

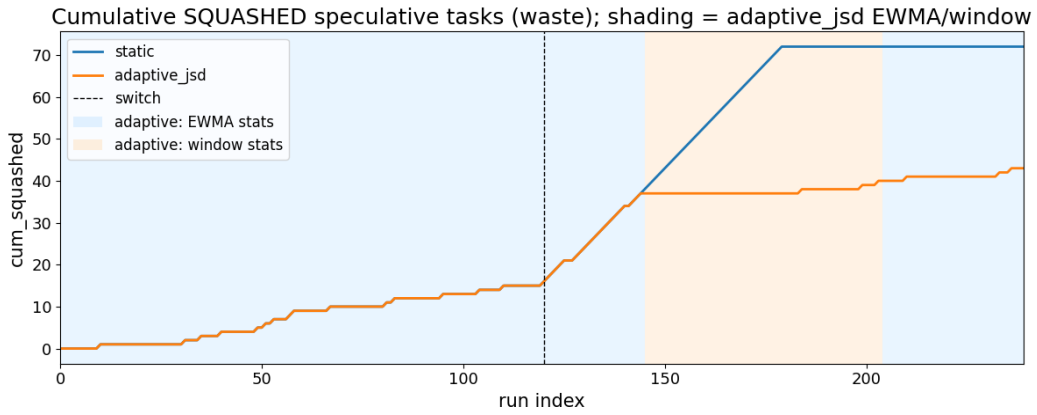
表 5-3 长记忆统计与分布漂移适应在分布漂移场景下的结果对比

策略	总投机率	总误预测率	漂移后投机率	漂移后误预测率	平均撤销量	平均浪费时间/ms
长记忆统计	74.58%	40.22%	50.00%	95.00%	0.300	0.162
分布漂移适应	99.58%	17.99%	100.00%	23.33%	0.179	0.103

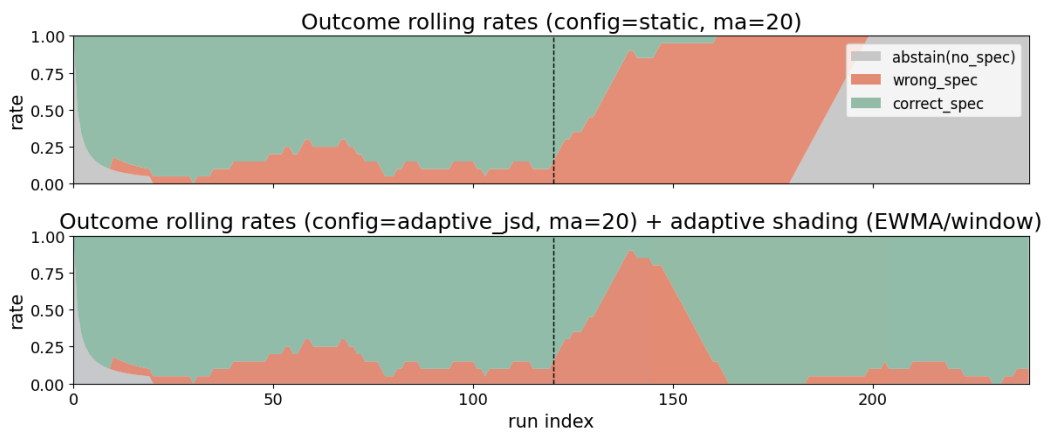
可以看出，前 120 次运行中，两种策略的表现基本一致，都能够较准确地学习分支偏置。分布突变后，长记忆统计由于持续保留全部历史信息，难以及时适应新的分支分布；相比之下，分布漂移适应策略能够根据分布变化及时调整统计视图，从而更有效地抑制错误投机带来的额外开销。具体而言，分布漂移适应策



(a) 最大预测概率变化情况



(b) 累计撤销开销对比



(c) 不同结果比例的变化情况

图 5-2 长记忆统计与分布漂移适应策略在分布漂移场景下的对比结果

略的平均撤销量较长记忆统计降低了 40.28%，平均浪费时间降低了 36.53%，表明其在分布漂移场景下具有更好的适应能力与鲁棒性。

5.2.3 联合优化效果评估

为评估输入感知分支预测与分布漂移适应机制的联合效果，本实验设计了一个输入-漂移耦合场景工作流。在该工作流中，分支走向由输入唯一决定。实验共运行 480 次，并在第 240 次运行后切换输入分布：前一阶段中，两类输入的概率分别为 0.65 和 0.35，后一阶段中概率翻转。此外，为避免场景过于理想化，实验还引入了 0.08 的标签噪声。预测器推测深度为 2，其余相关参数均采用默认配置，具体同表 4-5。

图 5-3 展示了两种策略在该场景下的整体结果，绘图的移动平均窗口为 20。可以看出，静态路径统计策略在前期虽形成了一定投机能力，但整体置信度较低，且在阶段切换后迅速退化为保守执行，后半段几乎不再进行有效投机。相比之下，联合优化策略在两个阶段均保持了较高投机活跃度，仅在切换点附近出现短暂调整，随后即可恢复稳定。

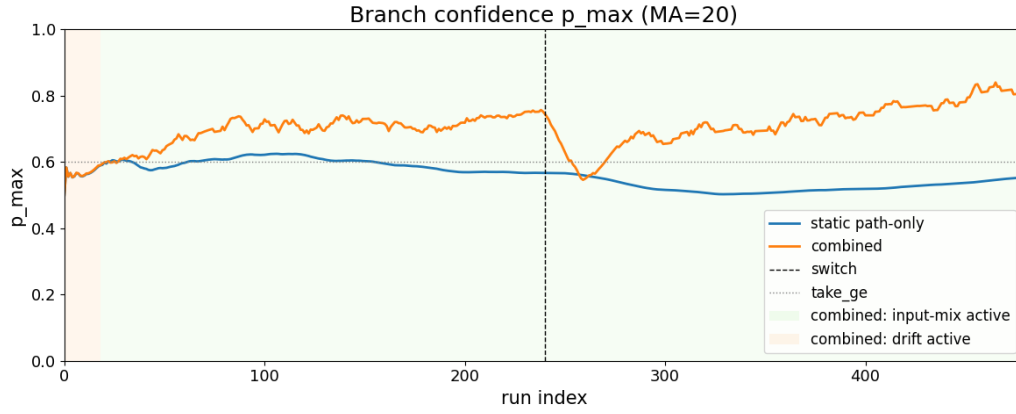
为便于比较两种策略的整体表现，表 5-4 汇总了主要统计结果。

表 5-4 静态路径统计与联合优化策略在输入-漂移耦合场景下的结果对比

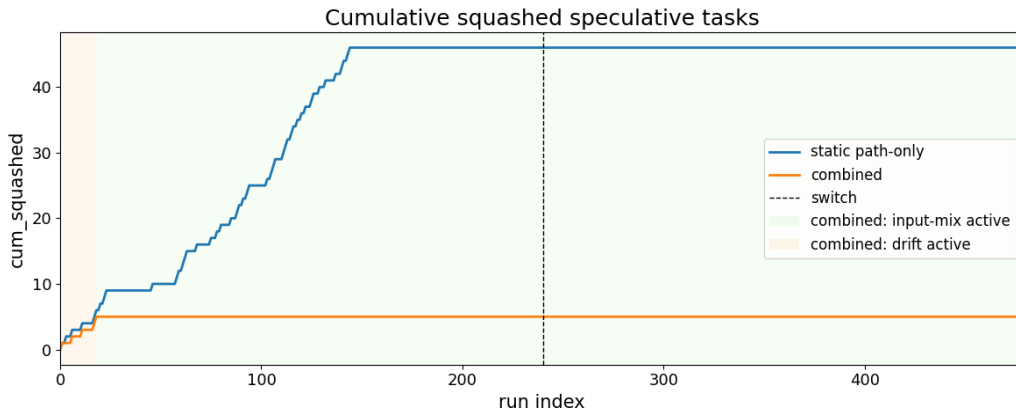
策略	总投机率	投机条件下 误预测率	平均撤销 量	平均浪费 时间/ms	阶段 2 投 机率	阶段 2 误 预测率
静态路径统计	20.83%	46.00%	0.0958	0.0540	0.00%	—
联合优化策略	74.38%	1.40%	0.0104	0.0062	73.75%	0.00%

从表 5-4 可以看出，联合优化策略在复合动态场景中表现出明显优势。其总投机率由 20.83% 提高至 74.38%，投机条件下误预测率由 46.00% 降至 1.40%，平均撤销量和平均浪费时间也分别由 0.0958 和 0.0540 ms 降至 0.0104 和 0.0062 ms。更重要的是，静态路径统计在阶段 2 中已基本失去投机能力，而联合策略仍保持 73.75% 的投机率，且误预测率为 0，说明其能够在分布切换后较快恢复并稳定工作。

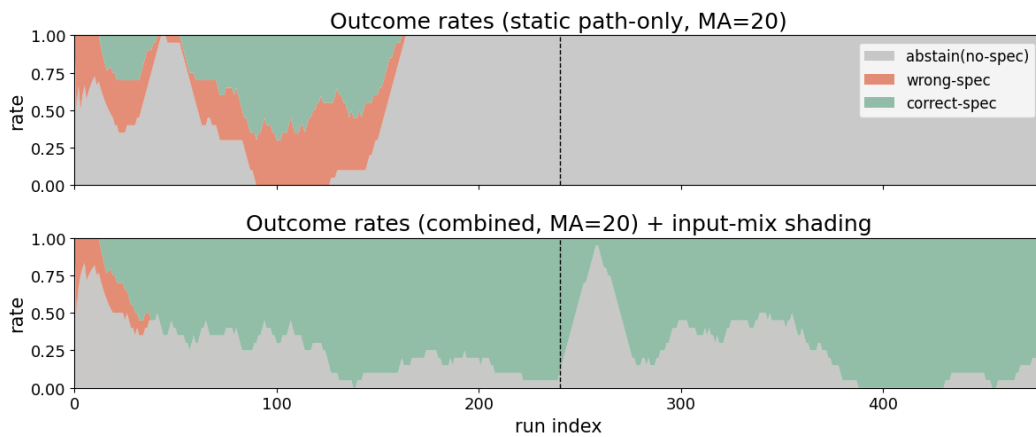
综上，在输入相关性与分布漂移同时存在的场景中，单纯依赖静态路径统计难以兼顾投机积极性、预测准确性和后期适应能力；而联合优化策略能够在保持较高投机率的同时显著降低误预测与撤销开销，体现出更好的综合优化效果。



(a) 最大预测概率变化情况



(b) 累计撤销开销对比



(c) 不同结果比例的变化情况

图 5-3 静态路径统计与联合优化策略在输入-漂移耦合场景下的对比结果

5.3 功能验证实验

在性能评估之前，首先需要进行功能验证实验，即检验系统在不同 workflow 结构和不同运行情形下，是否能够正确完成任务调度、投机启动、结果提交以及错误路径撤销等过程，并保证最终执行结果与非投机执行语义一致。

为尽可能覆盖原型系统可能遇到的典型与边界情形，本文构造了一组用于功能验证的测试 workflow。表 5-5 给出了所有的测试 workflow。

表 5-5 功能验证实验中使用的测试 workflow

workflow 名称	核心验证点
wf00_smoke	workflow 基础功能
wf01_single_node	单节点执行
wf02_short_chain	短链执行
wf03_depth_stress	深链执行
wf04_branch_biased	偏置分支预测
wf05_branch_unbiased_gate	低置信分支门控
wf06_branch_merge_join	分支汇合
wf07_memo_miss_then_hit	记忆表增量构建
wf08_memo_pure	纯函数结果复用
wf09_non_spec_barrier	非投机执行屏障
wf10_data_hazard_raw	RAW 依赖冲突恢复
wf11_data_hazard_war_waw	WAR/WAW 状态一致性
wf12_side_effect_buffered	副作用隔离提交
wf13_squash_cascade	级联撤销恢复
wf14_deep_spec_rollback	深层投机回滚
wf15_multi_run_state_persist	跨次状态积累
wf16_multi_run_state_isolation	跨实例状态隔离

本文采用自动化差分验证的方法对执行结果进行校验。对于每个测试 workflow 及其对应输入，系统分别在非投机执行模式和投机执行模式下运行一次，并在每次运行结束后自动导出三类结果：workflow 最终返回结果、CouchDB 中最终提交的状态数据，以及运行过程中实际对外生效的消息或输出记录。随后，由验证脚本对两种模式下的结果进行规范化比对，过滤时间戳、请求编号等与语义无关的信息；若三类结果一致，则认为该测试样例通过功能验证，否则判定为失败。

除最终结果一致性外，本文还验证系统内部机制是否按预期生效。原型系统在运行过程中记录节点启动、投机来源、分支解析、结果提交及错误路径撤销等关键事件，并由验证脚本自动检查：当分支预测正确或记忆化命中时，后继节点是否能够被提前启动并正确提交；当发生误预测、无效投机或依赖冲突时，错误路径上的任务是否被正确撤销，且不会影响最终返回值、提交状态及对外可见的

消息或输出。如图 5-4 所示，功能测试汇总结果表明，各测试 workflows 均通过了语义一致性与机制有效性验证。本地执行后端和 OpenWhisk 后端均通过了验证。

```
=====
Functional Validation Summary
=====
Total Workflows      : 17
Passed               : 17
Failed               : 0
Pass Rate            : 100.0%

Semantic Checks
  Output Match       : 17/17
  State Match        : 17/17
  Side-effect Match  : 17/17

Mechanism Checks
  Early Start        : 7/7
  Correct Commit     : 17/17
  Correct Squash     : 5/5

Event Statistics
  Total Early Starts : 10
  Total Commits      : 68
  Total Squashes     : 7
  Branch Hits        : 4
  Memo Hits          : 3

Final Verdict        : PASS
=====
```

图 5-4 功能测试执行结果

仅依靠合成或机制导向的测试 workflows，还不足以说明系统在较完整应用场景中的可用性。为此，本文进一步选取并移植了三类具有代表性的端到端应用 workflows，用于验证系统在真实风格任务中的功能正确性，如表 5-6 所示。

表 5-6 端到端应用 workflows 的适配情况

工作流类型	典型流程	适配结果
请求路由与风控流	鉴权、请求特征提取、风险评分/用户分层分支、服务路由、聚合返回	支持
事件日志分析与告警流	日志清洗、特征提取、正常/异常分支、告警等级路由、聚合存储	支持
内容感知媒体处理流	解码/质量分析、内容识别、模型选择分支、后处理、结果返回	支持

在上述应用 workflows 上，本文同样对执行结果的正确性进行了验证，相关测试均顺利通过。后文将进一步基于这三类应用 workflows 开展性能评估实验，以考察系统在较真实应用场景下的优化效果。

5.4 性能评估实验

本节聚焦本文方法在端到端场景下的整体性能表现，主要从端到端时延、尾延迟以及资源开销等方面评估其在真实风格 workflows 中的优化效果。

5.4.1 实验设置

结合前文实验设计，本文选取了三类具有代表性的端到端应用 workflows，分别为请求路由与风控流(request_risk)、事件日志分析与告警流(log_alert)以及内容感知媒体处理流(media_content)。三类 workflows 体现了 Serverless 场景下常见的在线请求处理、运维监控分析与媒体内容处理应用模式。

其中，request_risk 主要模拟请求接入后的风险评估与路径分流过程，输入特征对后继分支选择具有较强影响；log_alert 主要模拟日志解析、异常检测与告警分级过程，其分支决策具有中等输入相关性，且更容易受到阶段性负载变化的影响；media_content 主要模拟媒体内容识别与处理链路选择过程，不同内容特征将触发不同的后续处理路径。

需要说明的是，本文未直接采用公开 benchmark 中的现成实验 workflows，因为它们不能直接给出本文所需的输入驱动分支语义与阶段性分布变化模式^[52-56]。鉴于本文的核心目标在于评估输入感知分支预测与分布漂移适应机制在完整 workflows 中的实际收益，本文围绕研究问题对 workflows 结构、输入字段及阶段切换方式进行了针对性移植和构造，从而形成更适合本文方法验证的端到端实验负载。

在输入设计上，本文为不同 workflows 构造了能够影响后继路径选择的输入字段，并由中间函数完成特征提取后再触发相应分支，以保证实验负载既具有可控性，也更接近真实系统中的决策过程。为覆盖本文关注的输入相关分支与阶段性分布变化两类关键因素，实验进一步设置了 stable 与 drift 两种场景。其中，stable 场景下输入分布及分支规律在整个实验期间基本保持稳定；drift 场景下则在预设阶段切换点前后改变输入分布。

本文还在不同并发度进行了测试，包括 low、medium 和 high 三档并发水平，详细参数如表 5-7。表中的并发度定义为同时处于执行状态的工作流实例数。

表 5-7 三类 workflows 的高、中、低并发场景设置

工作流	低并发	中并发	高并发
request_risk	30	100	200
log_alert	15	50	100
media_content	6	16	32

对比方法包括三种：NoSpec，即不启用投机执行；BaseSpec，即采用基

基础投机策略，仅使用路径统计信息进行分支推进；以及 SpecSW，即本文提出的完整方法，同时启用输入感知分支预测与分布漂移适应机制。主实验共形成 $3 \times 2 \times 3 \times 3$ 的实验矩阵，其中每个工作流重复执行 500 次；参数均采用默认参数。

评估指标主要包括端到端平均时延、P95/P99 尾延迟、错误投机率以及每请求平均额外函数触发数。从数据质量上看，各组合成功率整体接近 1.0，未出现系统性的执行失败，因此本节后续分析主要聚焦于不同方法之间的性能差异与开销变化。

5.4.2 平均端到端表现分析

平均端到端时延结果如图 5-5 所示，对应的平均加速比如图 5-6 所示。总体来看，采用投机执行的两种方法在全部 18 个场景中均优于保守执行的 NoSpec。从平均加速比来看，BaseSpec 和 SpecSW 的总体平均加速比分别约为 2.42 $\times$ 和 2.97 $\times$ 。此外，若直接比较两种投机方法，SpecSW 相对于 BaseSpec 的加速比提高了 22.73%。这表明投机执行本身已经能够显著改善工作流性能，而本文方法又在此基础上进一步提升了投机执行的有效性。

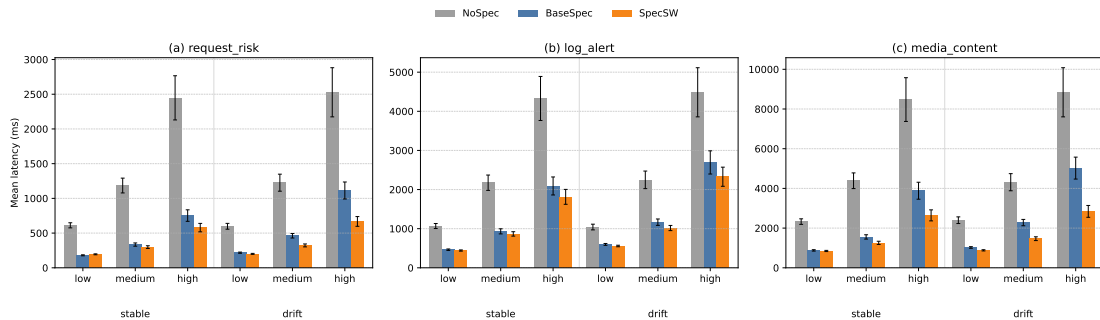


图 5-5 不同工作流与负载条件下三种方法的端到端平均时延比较

进一步比较两种投机方法可以看到，SpecSW 在 18 个场景中的 17 个场景下均优于 BaseSpec。仅在 request_risk 的 stable-low 场景下，SpecSW 的平均时延略高于 BaseSpec (194.3 ms vs. 179.9 ms)，这一现象可能是低并发稳定场景下投机执行收益有限，以及额外控制开销在轻负载稳定分支中抵消潜在收益所导致的。不过，随着负载升高或分布发生变化，SpecSW 的优势会迅速显现。

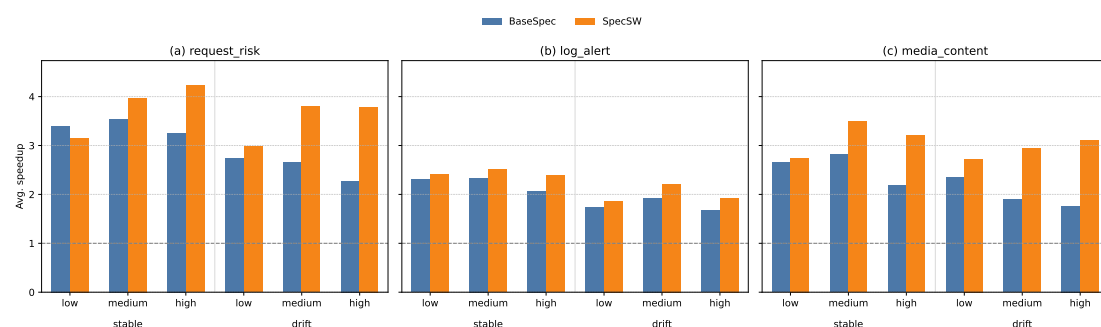


图 5-6 不同工作流与负载条件下投机方法的平均加速比

从不同应用的比较来看, `log_alert` 中 SpecSW 相对 BaseSpec 的附加收益最小, 这与该工作流仅具有中等输入相关性的特征是一致的。定量来看, `log_alert` 中 SpecSW 相对 BaseSpec 的平均时延附加降幅约为 11.59%; 相比之下, `request_risk` 和 `media_content` 中这一指标分别约为 26.06% 和 32.14%。这说明当输入与分支决策的耦合程度较弱时, 输入感知预测所能带来的边际收益会相应减小。

从不同漂移场景的比较来看, `drift` 下 BaseSpec 的退化更为明显。对全部场景分别求平均后, BaseSpec 在 `stable` 与 `drift` 下的平均加速比分别约为 2.73x 和 2.11x, 下降约 22.71%; 而 SpecSW 则分别约为 3.12x 和 2.82x, 下降约 9.62%。这说明在输入分布发生变化时, 仅依赖较弱自适应机制的 BaseSpec 更容易因误投机增加而损失性能, 而 SpecSW 能够更有效地缓解由输入漂移带来的性能退化。

从不同并发水平的比较来看, 随着并发由 `low` 提升至 `medium` 和 `high`, 三类工作流的平均时延整体上均有所增加, 但 SpecSW 在中高并发下表现出更明显的相对优势。对全部工作流取平均后, SpecSW 相对 BaseSpec 的平均时延附加降幅在 `low`、`medium` 和 `high` 三种并发下分别约为 7.08%、22.32% 和 30.11%。这说明在高并发场景下, SpecSW 相对 BaseSpec 具有更高的有效投机比例, 从而能够将更多潜在并行性转化为端到端性能收益。

综合而言, 图 5-5 和图 5-6 共同反映出三个较为清晰的结论: 第一, 投机执行总体上能够显著降低端到端平均时延并提升平均加速比; 第二, 输入-分支相关性和输入分布漂移越强, SpecSW 相对 BaseSpec 的附加收益越明显; 第三, 在存在较高系统负载时, SpecSW 通常能够维持更稳定的性能表现。这些结果说明, 本文方法不仅能够在稳定场景下取得额外收益, 而且在更具挑战性的漂移

与中高并发场景下也具有较好的鲁棒性。

5.4.3 尾延迟分析

尾延迟结果如图 5-7 和图 5-8 所示。总体来看, P95 与平均时延的趋势基本一致: 两种投机方法整体均优于 NoSpec, 且 SpecSW 在全部 18 个场景中的 P95 均低于 BaseSpec。对全部场景取平均后, BaseSpec 和 SpecSW 的 P95 归一化时延分别约为 0.78 和 0.70, 即相对于 NoSpec 分别下降约 21.6% 和 30.0%。SpecSW 相对于 BaseSpec 下降 10.3%。

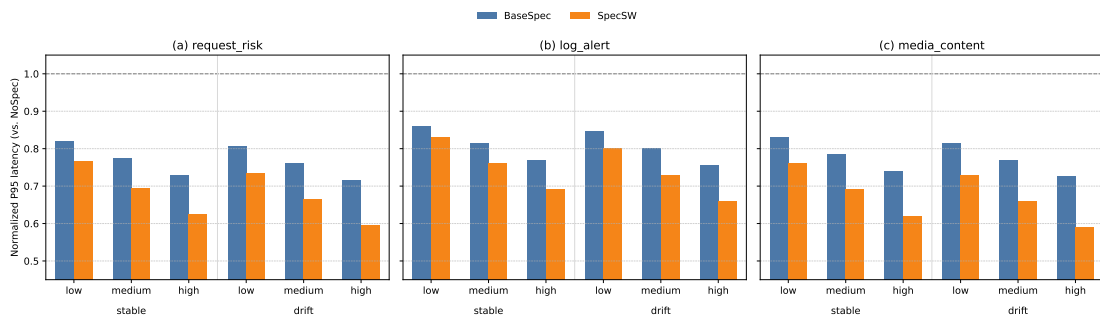


图 5-7 不同 workflow 与负载条件下三种方法的 P95 归一化时延比较

从不同场景的比较来看, P95 上 SpecSW 的优势在 drift 与中高并发场景下更为明显。例如,在 media_content-drift-high 场景下, SpecSW 的 P95 归一化值约为 0.59, 而 BaseSpec 约为 0.73; 在 request_risk-stable-medium 场景下, 两者分别约为 0.70 和 0.78。相比之下, 在 stable-low 这类轻载稳定场景下, 两种投机方法之间的差距相对有限, 如 request_risk-stable-low 中二者分别约为 0.77 和 0.82。这一结果与平均时延中的观察基本一致, 即当输入相关性增强、分布发生变化或系统负载升高时, 更强的输入感知与漂移适应机制更容易转化为稳定的尾延迟收益。

相比之下, P99 呈现出更强的波动性。由图 5-8 可以看到, 在全部 18 个场景中, SpecSW 相对 BaseSpec 的 P99 有 8 个场景下降、10 个场景上升, 整体平均变化仅约为 -0.39% 。这可能是因为实验轮数仅为 500, 意味着 P99 基本上仅由最慢的 5 个请求决定, 两种方法预测结果类似 (均为错误或投机), 受平台抖动影响较大。

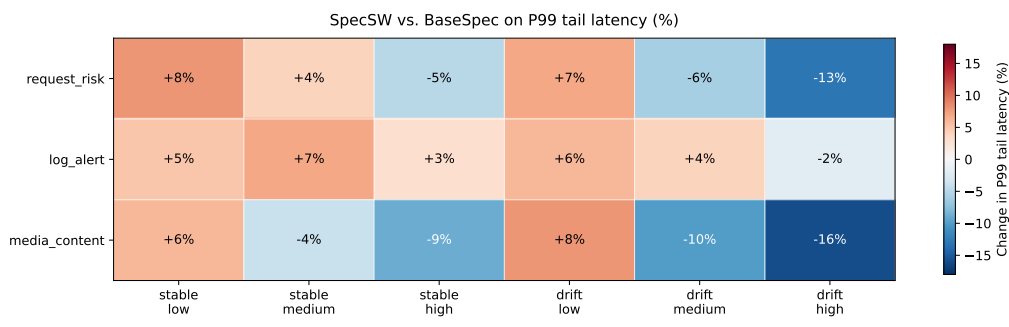


图 5-8 SpecSW 相对 BaseSpec 的 P99 变化热力图

5.4.4 投机情况分析

为进一步解释性能收益的来源，本文统计了两种投机方法在不同场景下的错误投机率以及每请求平均额外函数触发数，结果如表 5-8 所示。总体来看，SpecSW 在全部六类 workflow 场景中的错误投机率均低于 BaseSpec。对各场景取平均后，BaseSpec 与 SpecSW 的错误投机率分别约为 19.76% 和 13.98%，平均下降约 5.78 个百分点。

表 5-8 不同 workflow 场景下两种投机方法的错误投机率与额外触发开销比较

场景	BaseSpec 误投机率 (%)	SpecSW 误投机率 (%)	下降 (pct)	BaseSpec 额外触发	SpecSW 额外触发
RR-S	14.14	9.65	4.49	0.613	0.571
RR-D	20.02	12.55	7.47	0.647	0.555
LA-S	19.46	16.05	3.41	0.547	0.505
LA-D	22.54	18.18	4.36	0.580	0.529
MC-S	18.06	11.92	6.14	0.680	0.597
MC-D	24.36	15.53	8.83	0.733	0.620
平均	19.76	13.98	5.78	0.633	0.563

注：RR 表示 request_risk，LA 表示 log_alert，MC 表示 media_content；S 表示 stable，D 表示 drift。

从不同场景的比较来看，drift 场景下两种方法的错误投机率普遍高于 stable 场景，但 SpecSW 的下降幅度更为明显。例如，在 request_risk-drift 和 media_content-drift 中，错误投机率分别下降约 7.47 和 8.83 个百分点；相比之下，log_alert 中下降幅度相对较小，这与其输入-分支相关性较弱的特征基本一致。同时，SpecSW 的每请求平均额外触发数并未增加，反而由 BaseSpec 的 0.633 降至 0.563。这表明 SpecSW 的性能改善主要来源于更高质量的投机。

5.4.5 本节小结

本节从平均端到端时延、尾延迟以及投机行为开销，对本文方法在三类端到端 workflows 中的性能表现进行了系统评估。实验结果表明，投机执行总体上能够显著改善工作流性能，而本文提出的 SpecSW 在绝大多数场景下进一步优于 BaseSpec。尤其在输入强相关、分支分布发生漂移以及中高并发条件下，SpecSW 表现出更稳定的时延收益与更好的鲁棒性。结合错误投机率与额外触发开销分析可以看出，本文方法的性能提升主要来源于更高质量的投机。

5.5 消融实验

为考察各关键机制的独立贡献，本文将消融实验放在核心算法仿真场景中进行。原因在于，端到端场景中存在较多平台抖动，容易掩盖单一模块的真实效果；而仿真实验便于控制输入分布、路径偏置和阶段切换等条件，更适合进行细粒度、可复核的机制分析。

5.5.1 输入感知分支预测模块消融

为比较仅基于路径统计、仅基于输入桶统计以及本文提出的自适应路径-输入混合机制在误预测控制与无效投机开销上的差异，本节沿用前文输入感知场景的实验设定，进一步开展消融实验。

实验设置了两类场景：一类为无噪声场景，用于观察较干净条件下三种方法的基本表现；另一类为稀疏噪声场景，用于考察输入桶统计在稀疏条件下的稳定性。表 5-9 给出了两类场景下三种方法的主要结果。

表 5-9 输入感知分支预测模块消融结果

场景	方法	全样本误预测率	投机率	条件误预测率	平均撤销数
无噪声场景	仅路径统计	0.015	0.033	0.518	0.015
	仅输入桶统计	0.000	0.998	0.000	0.000
	自适应混合	0.005	0.988	0.005	0.005
稀疏噪声场景	仅路径统计	0.018	0.037	0.609	0.018
	仅输入桶统计	0.291	0.589	0.494	0.291
	自适应混合	0.028	0.053	0.574	0.028

注：全样本误预测率为主结论指标；条件误预测率仅在已发生投机的条件下统计，因此需结合投机率共同解读。

从实验结果看，在无噪声场景下，仅输入桶统计表现最好，其全样本误预测率与平均撤销数均为 0，而自适应混合反而引入了少量额外误预测。这说明在输入模式简单、桶内样本充足时，纯桶统计已足以刻画输入与后继分支之间的对应关系，自适应混合并不带来额外收益。但在稀疏噪声场景下，仅输入桶统计的全样本误预测率与平均撤销数均升至 0.291，而自适应混合将二者同时降至 0.028。这表明，当高基数噪声键导致输入桶统计稀疏化时，单纯依赖桶级统计容易产生较多误投机，而自适应路径-输入混合机制能够更有效地抑制由稀疏性带来的无效开销。

总体上，输入感知自适应机制并非在所有简单场景下都优于仅输入桶统计，但在更具代表性和符合真实情况的稀疏噪声条件下，能够更稳定地降低全样本误预测与无效投机开销，说明在路径级统计与输入桶级统计之间进行自适应折中是有必要的。

5.5.2 分布漂移适应模块消融

为探究单一指数衰减统计（EWMA）、单一短窗统计（Window）与本文提出的自适应切换机制（Adaptive）在误预测控制与漂移恢复上的差异究竟有多大，本节沿用 5.2.2 节的实验设计，进行进一步探究实验。

实验在原来的基础上，设置了两类场景：强漂移对应 $0.9 \leftrightarrow 0.1$ ，弱漂移对应 $0.7 \leftrightarrow 0.3$ 。每类均包含两个方向相反的镜像配置与 10 个随机种子。表 5-10 给出了按漂移强度合并后的结果。图 5-9 直观地给出了一次实验中各策略的最大概率变化情况。

表 5-10 按漂移强度合并后的分支预测统计模块消融结果

范围	样本数	误预测率			误预测胜率		Squashed 胜率		恢复胜率	
		Adaptive	EWMA	Window	vs EWMA	vs Window	vs EWMA	vs Window	vs EWMA	vs Window
总体	40	0.238	0.311	0.240	0.90	0.60	0.55	0.80	1.00	0.98
强漂移	20	0.131	0.248	0.150	1.00	0.90	1.00	0.90	1.00	1.00
弱漂移	20	0.345	0.373	0.332	0.80	0.30	0.10	0.70	1.00	0.95

注：胜率表示在对应场景全部随机种子上，Adaptive 相对基线表现更优所占比例。对于误预测率与 Squashed 指标，“更优”表示数值更低；对于恢复指标，“更优”表示恢复步数更少。

从实验结果可以得出，自适应切换机制的误预测表现总体上优于单一指数衰减统计，与单一短窗统计大体接近，并在恢复相关指标上表现出更稳定的优势。从分场景结果看，其优势主要集中在强漂移条件下。但也应指出，这一结论

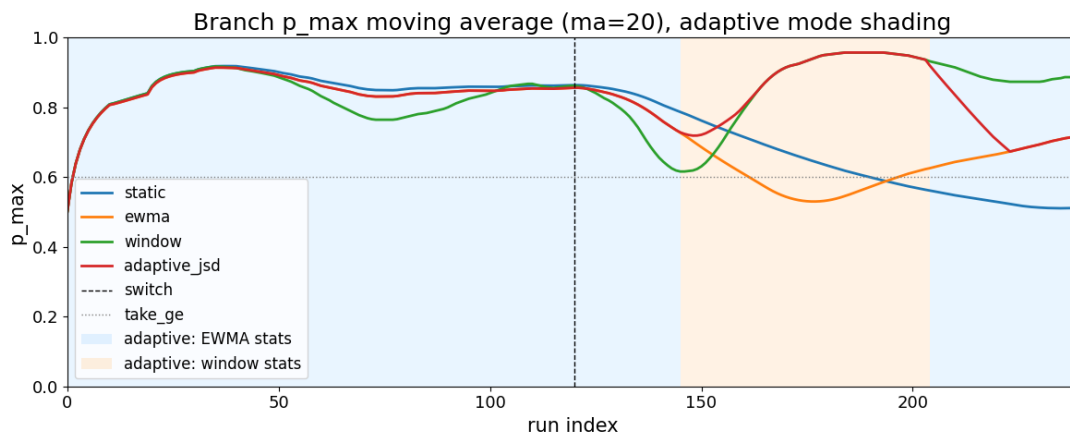


图 5-9 四种策略在分布漂移场景下的最大预测概率变化情况

仍建立在合成两阶段漂移之上，因此更适合作为机制层面的证据，而非对所有真实工作负载的直接外推。

5.5.3 联合优化效果消融

为进一步考察输入感知机制与分布漂移适应机制在同时启用时能否形成互补作用，本节在前两节单模块消融的基础上，进一步开展联合优化消融实验。与分别考察单一模块不同，本节关注的问题是：当两类机制共同参与时，联合策略是否优于“只启用其中一个模块”。

实验统一在相同工作流和相同输入序列下比较四种方法：仅路径统计(path-only)、仅输入感知(input-only)、仅漂移适应(drift-only)和联合优化(joint)。实验仍设置两类场景：一类为无噪声漂移场景，用于观察较理想条件下各方法的基本表现；另一类为稀疏噪声漂移场景，用于考察输入桶统计稀疏化时联合机制的行为。图 5-10 给出了四种方法在两类场景下的全样本误预测率、投机率和平均撤销数对比结果。

从结果可以看出，在无噪声漂移场景下，仅输入感知的全样本误预测率和平均撤销数最低；在稀疏噪声漂移场景下，仅输入感知基本退化为仅路径统计，而联合优化与仅漂移适应的表现都较好。

因此，联合优化策略并不总是绝对最优，但相较于只启用单一模块，其跨场景表现更稳健：在较干净场景下，它虽略逊于仅输入感知的最优误预测表现，却具有更短的恢复滞后；在稀疏噪声场景下，它又能够避免退化为单纯依赖输入桶

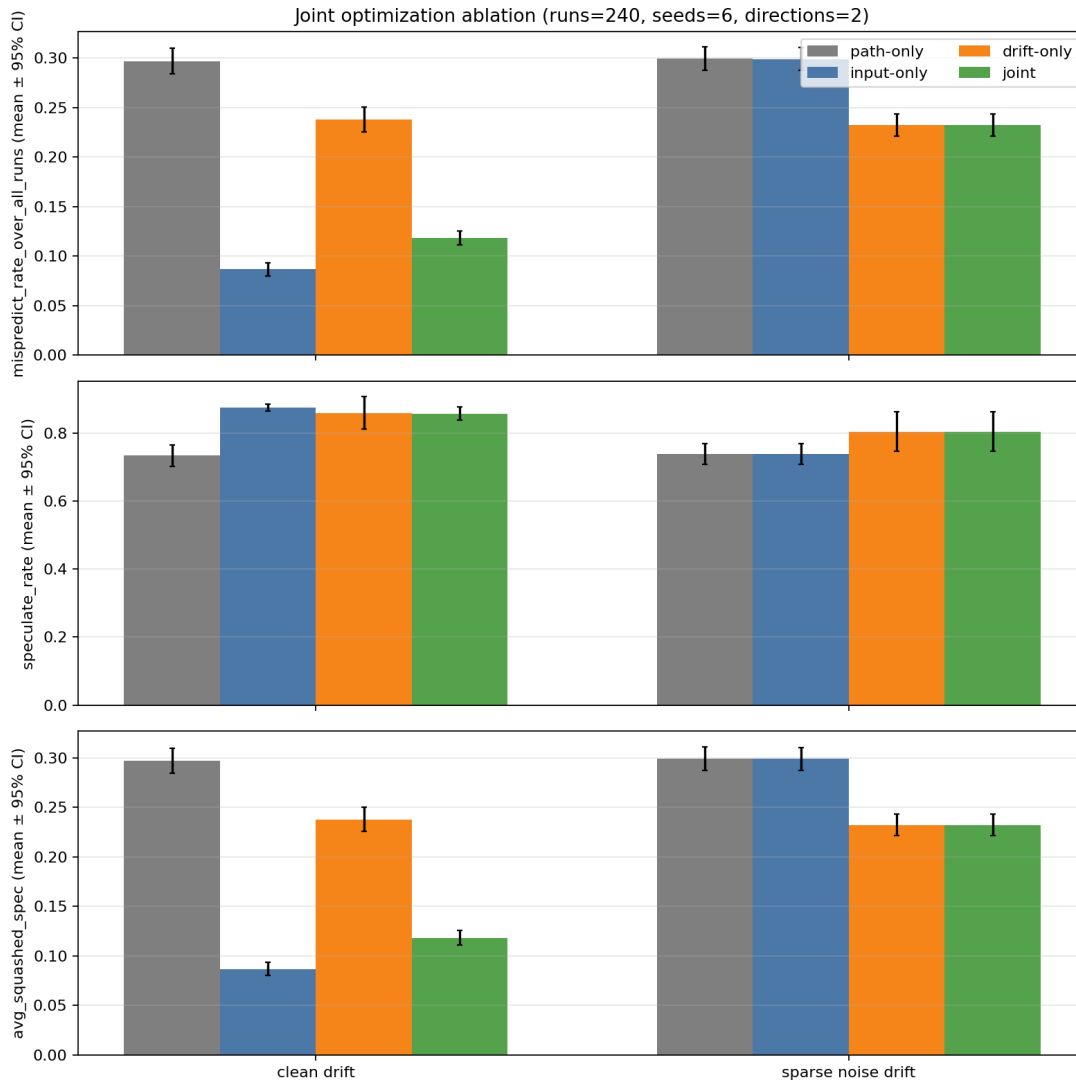


图 5-10 四种方法在两类场景下的全样本误预测率、投机率和平均撤销数

统计的脆弱方案。这说明，在输入感知与漂移适应之间进行联合设计是有意义的，但其收益取决于输入桶统计是否足够可靠。

5.6 本章小结

本章围绕本文提出的 Serverless workflow 投机执行优化方法及原型系统 SpecSW，从核心算法、系统功能、端到端性能和关键机制贡献四个方面进行了实验评估。

在算法层面，实验分别验证了输入感知分支预测、分布漂移适应以及联合优化策略的有效性。结果表明，在输入决定分支走向的场景中，输入感知策略较仅路径统计可将平均撤销量和平均浪费时间分别降低 89.47% 和 88.98%；在分布漂移场景中，漂移适应机制较长记忆统计可将平均撤销量和平均浪费时间分别降

低 40.28% 和 36.53%；在输入相关性与分布漂移并存的复合场景中，联合优化策略将总投机率由 20.83% 提升至 74.38%，并将投机条件下误预测率由 46.00% 降至 1.40%，说明本文提出的两类优化机制具有明确作用，且联合设计能够取得更好的综合效果。

在功能层面，SpecSW 通过了基础执行、分支门控、结果提交、错误路径撤销、状态一致性与副作用隔离等多项验证，并正确支持了三类真实风格端到端应用 workflow，说明系统实现具备较好的完整性与可用性。

在性能层面，两种投机方法在全部场景中均优于保守执行，其中 BaseSpec 和 SpecSW 的总体平均加速比分别达到 2.42x 和 2.97x，且 SpecSW 在 18 个场景中的 17 个场景下进一步优于 BaseSpec；在 P95 指标上，两者相对 NoSpec 的归一化时延分别约为 0.78 和 0.70。同时，SpecSW 将平均错误投机率由 19.76% 降至 13.98%，并将每请求平均额外触发数由 0.633 降至 0.563，说明其性能提升主要来源于更高质量的投机。

在消融实验层面，结果进一步表明，各关键机制均具有独立贡献，且系统整体通用性更高。其中，输入感知自适应混合机制在稀疏噪声场景下可将全样本误预测率和平均撤销数由仅输入桶统计的 0.291 同时降至 0.028；分布漂移适应模块总体误预测率为 0.238，低于单一指数衰减统计的 0.311，和单一短窗统计的 0.240，并在强漂移场景下表现出更明显的恢复优势；联合优化策略虽然并非在所有场景下都达到单项最优，但相较只启用单一模块的方案，整体表现更稳健。

综上所述，本文提出的 SpecSW 能够在不改变 workflow 外部语义的前提下，有效提升 Serverless workflow 投机执行的质量与端到端性能，验证了本文整体设计思路的有效性与合理性。

第六章 总结与展望

6.1 工作总结

随着云计算基础设施的发展和事件驱动应用的普及，Serverless 计算逐渐成为构建复杂云上应用的重要方式。相较于传统预置服务器或集群的部署模式，Serverless 以函数即服务为核心，具有按需计费、自动伸缩和托管运行时等优势，使开发者能够更加专注于业务逻辑本身。在此背景下，Serverless 工作流作为组织多函数协同执行的重要形式，在在线服务、数据处理的智能应用等场景中得到了广泛关注。

然而，现有 Serverless 工作流在执行过程中仍面临关键路径较长、分支决策等待明显以及多级函数调用开销叠加等问题。传统严格依赖驱动的执行方式要求上游节点完成后再触发下游节点，导致分支判定前后的等待时间难以隐藏；同时，冷启动、调度、通信和状态管理等平台开销也会沿工作流链路逐步累积，进一步增加端到端时延。已有工作表明，投机执行为缓解上述问题提供了一种有潜力的思路，但在真实工作流场景中，仍存在预测精度不足、动态适应性有限以及工程化支撑不够完善等问题。

为解决上述问题，本文围绕面向 Serverless 工作流的投机执行优化开展了系统研究，提出了 SpecSW 方案。本文的主要工作和结果总结如下：

- 提出了一种面向 Serverless 工作流的投机执行模型。该模型对控制依赖、数据依赖、分支预测、候选集合构造、门控与预算约束、状态隔离以及验证恢复等关键要素进行了统一描述，为后续方法设计与系统实现奠定了基础。
- 设计了 Serverless 工作流投机执行的声明式编程框架。该框架围绕工作流过程描述、运行接口和投机运行时配置等内容，提供了规范化抽象，使用户能够以声明式方式描述工作流拓扑、节点依赖关系及必要的策略注解，提高了方案的可用性与工程实现能力。
- 提出了一组面向复杂工作流场景的分支预测优化方法。针对基础方案在真

实场景下预测准确率和适应性不足的问题，本文从输入与分支走向的相关性以及时间变化两个维度对预测机制进行了改进，提出了输入感知预测方法和分布漂移适应机制，并讨论了二者的联合使用方式，以提升投机触发决策的稳定性。

- **实现了 SpecSW 原型系统。**本文基于 OpenWhisk 平台实现并评估了 SpecSW 原型系统。实验结果表明：相比基础投机方法，SpecSW 的平均端到端时延降低了 22.73%，P95 尾延迟下降了 10.3%，错误投机率平均降低了 5.78 个百分点。

6.2 研究展望

本文的未来工作主要有且不仅限于以下几个方向：

- **研究投机执行关键参数的配置方法。**投机执行效果与门控阈值、投机深度、历史窗口大小以及预算约束等参数密切相关，不同工作流结构和负载模式下的最优参数往往存在差异。未来可进一步研究面向不同场景的参数调优方法，以提高方案的稳定性和适用性。
- **扩展对隐式工作流的支持。**本文当前主要面向显式定义的 Serverless 工作流进行设计与实现，而在真实应用中，函数之间通过调用关系动态形成的隐式工作流同样广泛存在。未来可进一步推进本方案在隐式工作流的实现，以拓展方案的适用范围。
- **扩展真实平台与大规模实验验证。**目前 SpecSW 仅在 OpenWhisk 平台上实现和评测，未来可扩展到更多 Serverless 平台和更大规模部署场景中，并引入更加丰富的真实应用工作流和负载模式，进一步验证方案的通用性与工程价值。
- **开展面向性能与成本协同优化的研究。**未来可将成本感知门控、资源预算自适应调整以及性能-成本联合优化纳入统一框架，使投机执行从单纯的性能优化进一步发展为兼顾时延、吞吐与成本的综合优化机制。

参考文献

- [1] BALDINI I, CASTRO P, CHANG K, et al. Serverless computing: Current trends and open problems[G] // Research advances in cloud computing. Springer, 2017: 1-20.
- [2] SHAFIEI H, KHONSARI A, MOUSAVI P. Serverless Computing: A Survey of Opportunities, Challenges, and Applications[J/OL]. ACM Comput. Surv., 2022, 54(11s). <https://doi.org/10.1145/3510611>. DOI: 10.1145/3510611.
- [3] LI Y, LIN Y, WANG Y, et al. Serverless computing: state-of-the-art, challenges and opportunities[J]. IEEE Transactions on Services Computing, 2022, 16(2): 1522-1539.
- [4] LI Z, GUO L, CHENG J, et al. The serverless computing survey: A technical primer for design architecture[J]. ACM Computing Surveys (CSUR), 2022, 54(10s): 1-34.
- [5] MCGRATH G, BRENNER P R. Serverless computing: Design, implementation, and performance[C] // 2017 IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW). 2017: 405-410.
- [6] Amazon Web Services. AWS Lambda: Serverless Function, FaaS Serverless[EB/OL]. [2026-02-03]. <https://aws.amazon.com/lambda>.
- [7] Google Cloud. Cloud Functions[EB/OL]. [2026-02-03]. <https://cloud.google.com/functions>.
- [8] Microsoft. Azure Functions—Serverless Functions in Computing[EB/OL]. [2026-02-03]. <https://azure.microsoft.com/en-us/services/functions>.
- [9] Alibaba Cloud. 阿里云函数计算[EB/OL]. [2026-02-03]. <https://www.aliyun.com/product/fc>.

- [10] SAHA D, TALLURI S, JANSEN M, et al. Survey of Serverless Workflows[J]. AtLarge Research, 2024.
- [11] RISTOV S, PEDRATSCHER S, FAHRINGER T. AFCL: An abstract function choreography language for serverless workflow specification[J]. Future Generation Computer Systems, 2021, 114: 368-382.
- [12] EISMANN S, SCHEUNER J, VAN EYK E, et al. A review of serverless use cases and their characteristics[J]. ArXiv preprint arXiv:2008.11110, 2020.
- [13] LÓPEZ P G, ARJONA A, SAMPÉ J, et al. Triggerflow: trigger-based orchestration of serverless workflows[C] // Proceedings of the 14th ACM international conference on distributed and event-based systems. 2020: 3-14.
- [14] Amazon Web Services. AWS Step Functions: Serverless Workflow Orchestration[EB/OL]. [2026-02-13]. <https://aws.amazon.com/step-functions/>.
- [15] Google Cloud. Workflows: Orchestrate and automate Google Cloud services and HTTP-based API services[EB/OL]. [2026-02-13]. <https://docs.cloud.google.com/workflows/docs>.
- [16] Microsoft. Azure Durable Functions: Stateful functions in a serverless compute environment[EB/OL]. [2026-02-13]. <https://learn.microsoft.com/en-us/azure/azure-functions/durable/>.
- [17] KOUSIOURIS G, GIANNAKOS C, TSERPES K, et al. Measuring baseline overheads in different orchestration mechanisms for large faas workflows[C] // Companion of the 2022 ACM/SPEC International Conference on Performance Engineering. 2022: 61-68.
- [18] DEAN J, BARROSO L A. The tail at scale[J]. Communications of the ACM, 2013, 56(2): 74-80.
- [19] DU D, YU T, XIA Y, et al. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting[C] // Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. 2020: 467-481.

- [20] MAHGOUB A, SHANKAR K, MITRA S, et al. SONIC: Application-aware data passing for chained serverless applications[C]//2021 USENIX Annual Technical Conference (USENIX ATC 21). 2021: 285-301.
- [21] BHASI V M, GUNASEKARAN J R, THINAKARAN P, et al. Kraken: Adaptive container provisioning for deploying dynamic dags in serverless platforms[C]//Proceedings of the ACM Symposium on Cloud Computing. 2021: 153-167.
- [22] STOJKOVIC J, XU T, FRANKE H, et al. Mxfaas: Resource sharing in serverless environments for parallelism and efficiency[C]//Proceedings of the 50th annual international symposium on computer architecture. 2023: 1-15.
- [23] YU H, FONTENOT C, WANG H, et al. Libra: Harvesting idle resources safely and timely in serverless clusters[C]//Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing. 2023: 181-194.
- [24] STOJKOVIC J, XU T, FRANKE H, et al. Specfaas: Accelerating serverless applications with speculative function execution[C]//2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2023: 814-827.
- [25] MAHGOUB A, YIE B, SHANKAR K, et al. ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs[C]//16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 2022: 303-320.
- [26] SUNYAEV A. Cloud computing[G]//Internet computing. Springer, 2020: 195-236.
- [27] YOUNIS R, IQBAL M, MUNIR K, et al. A comprehensive analysis of cloud service models: IaaS, PaaS, and SaaS in the context of emerging technologies and trend[C]//2024 international conference on electrical, communication and computer engineering (ICECCE). 2024: 1-6.
- [28] 杨光, 刘杰, 曲慕子, 等. 服务器无感知计算系统性能优化技术研究综述[J]. 软件学报, 2025, 36(01): 47-78. DOI: 10.13328/j.cnki.jos.007190.

- [29] HASSAN H B, BARAKAT S A, SARHAN Q I. Survey on serverless computing[J]. Journal of Cloud Computing, 2021, 10(1): 39.
- [30] IBM. IBM Cloud Functions[EB/OL]. [2026-02-03]. <https://www.ibm.com/cloud/functions>.
- [31] Apache OpenWhisk. Apache OpenWhisk: Open source serverless cloud platform[EB/OL]. [2026-02-03]. <http://openwhisk.apache.org>.
- [32] HENDRICKSON S, STURDEVANT S, HARTER T, et al. Serverless Computation with OpenLambda[C]//Proceedings of the 8th USENIX Conference on Hot Topics in Cloud Computing (HotCloud '16). Denver, CO, USA: USENIX Association, 2016: 33-39.
- [33] OpenFaaS. OpenFaaS: Serverless functions, made simple[EB/OL]. [2026-02-03]. <https://www.openfaas.com>.
- [34] Fission Project. Fission/fission: Fast and simple serverless functions for Kubernetes[EB/OL]. [2026-02-03]. <https://github.com/fission/fission>.
- [35] Knative. Knative[EB/OL]. [2026-02-03]. <https://knative.dev/>.
- [36] BRANDNER L. A platform-agnostic model and benchmark suite for serverless workflows[D]. ETH Zurich, 2022.
- [37] PAUTASSO C, ZIMMERMANN O, AMUNDSEN M, et al. Microservices in practice, part 2: Service integration and sustainability[J]. IEEE Software, 2017, 34(02): 97-104.
- [38] BORGES M C, WERNER S, KILIC A. Faaster troubleshooting-evaluating distributed tracing approaches for serverless applications[C]//2021 IEEE International Conference on Cloud Engineering (IC2E). 2021: 83-90.
- [39] Van DER AALST W M, TER HOFSTEDE A H, KIEPUSZEWSKI B, et al. Workflow patterns[J]. Distributed and parallel databases, 2003, 14(1): 5-51.
- [40] DEAN J, BARROSO L A. The tail at scale[J]. Communications of the ACM, 2013, 56(2): 74-80.

- [41] SREEKANTI V, WU C, LIN X C, et al. Cloudburst: Stateful functions-as-a-service[J]. ArXiv preprint arXiv:2001.04592, 2020.
- [42] TOPCUOGLU H, HARIRI S, WU M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing[J]. IEEE transactions on parallel and distributed systems, 2002, 13(3): 260-274.
- [43] GOLEC M, WALIA G K, KUMAR M, et al. Cold start latency in serverless computing: A systematic review, taxonomy, and future directions[J]. ACM Computing Surveys, 2024, 57(3): 1-36.
- [44] YANG Y, ZHAO L, LI Y, et al. Infless: a native serverless system for low-latency, high-throughput inference[C] // Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2022: 768-781.
- [45] SILVA P, FIREMAN D, PEREIRA T E. Prebaking functions to warm the serverless cold start[C] // Proceedings of the 21st international middleware conference. 2020: 1-13.
- [46] CADDEN J, UNGER T, AWAD Y, et al. SEUSS: skip redundant paths to make serverless fast[C] // Proceedings of the Fifteenth European Conference on Computer Systems. 2020: 1-15.
- [47] TIAN H, LI S, WANG A, et al. Owl: Performance-aware scheduling for resource-efficient function-as-a-service cloud[C] // Proceedings of the 13th Symposium on Cloud Computing. 2022: 78-93.
- [48] PU Q, VENKATARAMAN S, STOICA I. Shuffling, fast and slow: Scalable analytics on serverless infrastructure[C] // 16th USENIX symposium on networked systems design and implementation (NSDI 19). 2019: 193-206.
- [49] GUNASEKARAN J R, THINAKARAN P, NACHIAPPAN N C, et al. Fifer: Tackling resource underutilization in the serverless era[C] // Proceedings of the 21st International Middleware Conference. 2020: 280-295.

- [50] SMITH J E. A study of branch prediction strategies[C]//25 years of the international symposia on Computer architecture (selected papers). 1998: 202-215.
- [51] YEHT Y, PATT Y N. Two-level adaptive training branch prediction[C]//Proceedings of the 24th annual international symposium on Microarchitecture. 1991: 51-61.
- [52] COPIK M, KWASNIEWSKI G, BESTA M, et al. Sebs: A serverless benchmark suite for function-as-a-service computing[C]//Proceedings of the 22nd International Middleware Conference. 2021: 64-78.
- [53] SCHMID L, COPIK M, CALOTOIU A, et al. SeBS-flow: Benchmarking serverless cloud function workflows[C]//Proceedings of the Twentieth European Conference on Computer Systems. 2025: 902-920.
- [54] YU T, LIU Q, DU D, et al. Characterizing serverless platforms with serverless-bench[C]//Proceedings of the 11th ACM Symposium on Cloud Computing. 2020: 30-44.
- [55] SHAHRAD M, FONSECA R, GOIRI I, et al. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider[C]//2020 USENIX annual technical conference (USENIX ATC 20). 2020: 205-218.
- [56] ZHANG Y, GOIRI Í, CHAUDHRY G I, et al. Faster and cheaper serverless computing on harvested resources[C]//Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. 2021: 724-739.

致 谢

写到这里，论文也终于告一段落。回望在南京大学攻读硕士学位的三年，有紧张、有迷茫，也有许多被人提醒、被人拉一把的时刻。研究生生涯并不总是热血与顿悟，更多时候是反复试错、缓慢推进的日常。所幸一路走来并不孤单。谨在此向所有给予我指导、陪伴与支持的师长、同门、亲友，致以最诚挚的谢意。

首先，衷心感谢我的导师马骏副教授。刚开始确定研究方向时，我对问题边界和技术路线都缺少清晰认识，是您耐心地帮我把思路一点点厘清。您在每次讨论时指出的问题一针见血，给出的建议非常中肯，提出的方案也都切实可落地。更重要的是，您始终挡在学生前面，并且给予我理解与信任，让我能稳住心态继续把事情做完。您治学的严谨、做事的实在，以及对学生的宽厚与担当，都是我今后学习和工作的榜样，也是南大校训的最好印证。也衷心感谢曹春教授在我研究生三年中给予的指导与帮助，并在补助方面提供支持，为我顺利开展研究工作提供了坚实保障。您定期组织的大组学术交流与组会讨论，使我有机会接触并学习不同研究方向的最新进展，开阔了视野，获益匪浅。

其次，感谢所有一路相伴的同学。感谢学长朱治学、程丁丁，你们给了我很多技术上的指导和就业上的帮助，我们“小团体”凝聚在一起，一起度过了很多难忘的时光。感谢东烨同学，是你从学长那里接下了 Faasit 项目和 I2EC 的大梁，为组里其他同学提供了庇护。感谢我的室友马英硕和许逸飞同学，谢谢你们带来的快乐，也谢谢你们对我的包容。另外，感谢接过项目的许世泽学弟，感谢同门许希帆、王梓璇、孙骞、柯骅、周进一同学，感谢健身搭子魏义轩、严思远、杜翰跃、蒋一杰、任天琪、黄嘉铨同学，感谢饭搭子和游戏搭子陈哲霏、曹卓、张宇晨、蒋梓栩、赖俊宇、许嘉禾同学。

第三，我要感谢在企业实习期间相伴和相助的人们。特别感谢我的师兄张浩，你给了我很多指导和帮助，让我明白了如何具备一名开发者应有的“产品感”。感谢我的主管史明伟先生、后端同事罗松、同组实习生张啸宇、阮奕捷、陈伟杰同学以及大组长杨皓然先生。

此外，要感谢我的网球、匹克球和板式网球球友们，球场上的奔跑和球场下的闲聊，都让这段时光格外闪光。感谢一开始带我在南京打球的板式网球球友飞火、矢野・治門和彭杰，感谢我的网球启蒙人鲍大卫老师，南大网球队的指导老师问梅老师以及张开翔、张蹇、杨涛、李凯宁、陈佳松、王辛、李鑫溟、孙海龙、郭航天、徐兢喆等同学，感谢常年一起打网球的李捷老师和陈建良、许潇予、陈浩然、苟煜田、刘开来、张伟、张彤彤、左楚霄等同学，感谢匹克球指导老师李俊老师。

另外，要感谢我的女朋友邓芳同学。我们在球场上相识，这份缘分奇妙而珍贵。感谢你明媚的笑容，那是我在面对压力和疲惫时最有效的治愈。感谢你教会我开朗地看待生活，与你在一起的时光，无论是球场上、操场上、健身房里的挥汗如雨，还是生活中的点滴分享，总是充满了简单的快乐。未来，我最大的愿望，就是能与你继续携手，一起开心地走下去。

最深沉的感谢，献给我的父母。二十余载求学路，你们始终是最坚实的后盾。每当我在电话里说“最近有点忙、有点累”，你们总是先关心我吃得好不好、休息够不够，从不催促我拿出什么结果，却始终相信我能把路走下去。你们用最朴素的方式给了我理解与支持，也给了我在压力面前不轻易退缩的底气。养育之恩无以为报，惟愿未来能用更踏实的成长，回报你们的付出与期待。

最后，也想感谢三年来那个没有轻易放弃的自己。感谢无数个与文献、数据和代码相伴的夜晚，感谢那些焦虑、迟疑之后依然选择继续推进的时刻。论文的完成不是终点，而是一个阶段的结束。未来我会带着这段经历给我的积累与勇气，继续前行，努力把每一步走得更稳、更扎实。

谨以此文，感谢所有未能一一提及却曾给予我善意与帮助的人。愿此去一切顺利，后会有期。

简历与科研成果

基本信息

吴晟昊，男，汉族，2001 年 6 月出生，福建宁德人。

教育背景

2023 年 9 月—2026 年 6 月	南京大学计算机学院	硕士
2019 年 9 月—2023 年 6 月	电子科技大学信息与软件工程学院	本科

攻读硕士学位期间完成的学术成果

1. 发明专利：一种基于 Serverless 的跨平台 CICD 系统与实现方法（专利号：202510253873.1）

攻读硕士学位期间参与的科研课题

1. 国家重点研发计划：服务器无感知计算系统软件技术（2022YFB4500700），2023 年 09 月 - 2026 年 01 月，负责项目总体前端与 API-Server 开发，完成子项目部署工具和编程模型核心模块的设计与开发。

攻读硕士学位期间获得的荣誉奖项

1. 2025 年度优秀研究生
2. 硕士生英才奖学金
3. 硕士生一等学业奖学金